Question 1Skipped

A digital content production company has transitioned all of its media assets to Amazon S3 in an effort to reduce storage costs. However, the rendering engine used in production continues to run in an on-premises data center and requires frequent and low-latency access to large media files. The company wants to implement a storage solution that maintains application performance while keeping costs low.

Which approach should the company choose to meet these requirements in the most cost-effective way?

Deploy an Amazon FSx for Lustre file system and sync media data from Amazon S3 into it using DataSync. Mount the FSx file system on the on-premises render servers using a VPN tunnel

Correct answer

Set up an Amazon S3 File Gateway to provide storage for the on-premises application

Set up a dedicated on-premises storage array that periodically fetches data from Amazon S3 using a custom-built application. Mount this storage volume on the rendering servers as their primary working directory

Use Mountpoint for Amazon S3 on the on-premises rendering servers to facilitate low-latency access to the S3 bucket

Overall explanation

Correct option:

Set up an Amazon S3 File Gateway to provide storage for the on-premises application

Amazon S3 File Gateway is designed specifically for on-premises applications that need low-latency access to S3 data. It caches frequently accessed data locally and exposes the S3 bucket as an NFS or SMB file share, making it ideal for performance-sensitive workloads like rendering. It is also cost-effective as the majority of data remains in S3.

What is Amazon S3 File Gateway

PDF RSS Focus mode

Amazon S3 File Gateway – Amazon S3 File Gateway supports a file interface into Amazon Simple Storage Service (Amazon S3) and combines a service and a virtual software appliance. By using this combination, you can store and retrieve objects in Amazon S3 using industry-standard file protocols such as Network File System (NFS) and Server Message Block (SMB). You deploy the gateway into your on-premises environment as a virtual machine (VM) running on VMware ESXi, Microsoft Hyper-V, or Linux Kernel-based Virtual Machine (KVM), or as a hardware appliance that you order from your preferred reseller. You can also deploy the Storage Gateway VM in VMware Cloud on AWS, or as an AMI in Amazon EC2. The gateway provides access to objects in S3 as files or file share mount points. With a S3 File Gateway, you can do the following:

- You can store and retrieve files directly using the NFS version 3 or 4.1 protocol.
- You can store and retrieve files directly using the SMB file system version, 2 and 3 protocol.
- You can access your data directly in Amazon S3 from any AWS Cloud application or service.
- You can manage your S3 data using lifecycle policies, cross-region replication, and versioning. You can think of a S3 File Gateway as a file system mount on Amazon S3.

A S3 File Gateway simplifies file storage in Amazon S3, integrates to existing applications through industry-standard file system protocols, and provides a cost-effective alternative to on-premises storage. It also provides low-latency access to data through transparent local caching. A S3 File Gateway manages data transfer to and from AWS, buffers applications from network congestion, optimizes and streams data in parallel, and manages bandwidth consumption.

S3 File Gateway integrates with other AWS services, for example:

- Common access management using AWS Identity and Access Management (IAM)
- Encryption using AWS Key Management Service (AWS KMS)
- Monitoring using Amazon CloudWatch (CloudWatch)
- Audit using AWS CloudTrail (CloudTrail)
- Operations using the AWS Management Console and AWS Command Line Interface (AWS CLI)
- Billing and cost management

In the following documentation, you can find a Getting Started section that covers setup information common to all gateways and also gateway-specific setup sections. The Getting Started section shows you how to deploy, activate, and configure storage for a gateway. The management section shows you how to manage your gateway and resources:

- provides instructions on how to create and use a S3 File Gateway. It shows you how to create a file share, map your drive to an Amazon S3 bucket, and upload files and folders to Amazon S3.
- describes how to perform management tasks for all gateway types and resources.

via - https://docs.aws.amazon.com/filegateway/latest/files3/what-is-file-s3.html

Incorrect options:

Deploy an Amazon FSx for Lustre file system and sync media data from Amazon S3 into it using DataSync. Mount the FSx file system on the on-premises render servers using a VPN tunnel - While Amazon FSx for Lustre is a high-performance file system suited for HPC and rendering workloads, it incurs higher costs than simpler alternatives and is typically optimized for cloud-based compute rather than on-premises usage. Additionally, the use of AWS DataSync and VPN tunneling introduces complexity and potential performance bottlenecks for consistent two-way file access.

Use Mountpoint for Amazon S3 on the on-premises rendering servers to facilitate low-latency access to the S3 bucket - Mountpoint for Amazon S3 is an open source file client that makes it easy for your file-aware Linux applications to connect directly to Amazon S3 buckets. It can also be installed on your existing on-premises systems, with access to S3 either directly or over an AWS Direct Connect connection via AWS PrivateLink for Amazon S3. It is not POSIX compliant. It is not intended for low-latency or local caching use-cases from on-premises environments.

Set up a dedicated on-premises storage array that periodically fetches data from Amazon S3 using a custom-built application. Mount this storage volume on the rendering servers as their primary working directory - This solution requires developing and maintaining a custom integration layer to periodically download files from S3 and sync with local storage. It increases operational burden and introduces latency and consistency issues. Additionally, the costs of maintaining on-premises hardware negate the cost benefits of moving to S3 in the first place.

References:

https://docs.aws.amazon.com/filegateway/latest/files3/what-is-file-s3.html

https://aws.amazon.com/blogs/aws/mountpoint-for-amazon-s3-generally-available-and-ready-for-production-workloads/

Domain

Design Cost-Optimized Architectures

Question 2Skipped

A financial services company runs its flagship web application on AWS. The application serves thousands of users during peak hours. The company needs a scalable near-real-time solution to share hundreds of thousands of financial transactions with multiple internal applications. The solution should also remove sensitive details from the transactions before storing the cleansed transactions in a document database for low-latency retrieval.

As an AWS Certified Solutions Architect Associate, which of the following would you recommend?

Feed the streaming transactions into Amazon Kinesis Data Firehose. Leverage AWS Lambda integration to remove sensitive data from every transaction and then store the cleansed transactions in Amazon DynamoDB. The internal applications can consume the raw transactions off the Amazon Kinesis Data Firehose

Correct answer

Feed the streaming transactions into Amazon Kinesis Data Streams. Leverage AWS Lambda integration to remove sensitive data from every transaction and then store the cleansed transactions in Amazon DynamoDB. The internal applications can consume the raw transactions off the Amazon Kinesis Data Stream

Batch process the raw transactions data into Amazon S3 flat files. Use S3 events to trigger an AWS Lambda function to remove sensitive data from the raw transactions in the flat file and then store the cleansed transactions in Amazon DynamoDB. Leverage DynamoDB Streams to share the transactions data with the internal applications

Persist the raw transactions into Amazon DynamoDB. Configure a rule in Amazon DynamoDB to update the transaction by removing sensitive data whenever any new raw transaction is written. Leverage Amazon DynamoDB Streams to share the transactions data with the internal applications

Overall explanation

Correct option:

Feed the streaming transactions into Amazon Kinesis Data Streams. Leverage AWS Lambda integration to remove sensitive data from every transaction and then store the cleansed transactions in Amazon DynamoDB. The internal applications can consume the raw transactions off the Amazon Kinesis Data Stream

You can use Amazon Kinesis Data Streams to build custom applications that process or analyze streaming data for specialized needs. Amazon Kinesis Data Streams manages the infrastructure, storage, networking, and configuration needed to stream your data at the level of your data throughput. You don't have to worry about provisioning, deployment, or ongoing maintenance of hardware, software, or other services for your data streams.

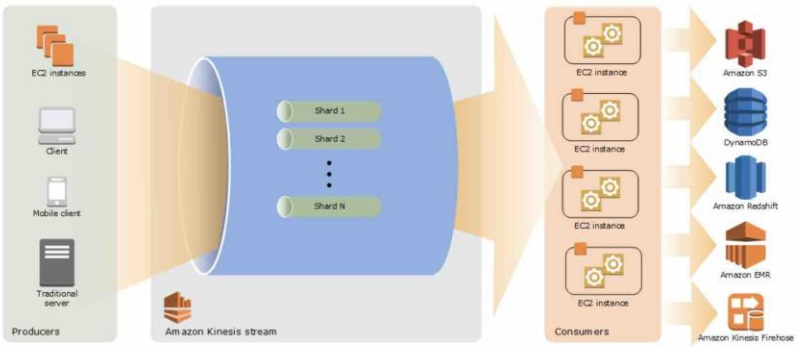
How Amazon Kinesis Data Streams Work:

via - https://aws.amazon.com/kinesis/data-streams/

Amazon Kinesis Data Streams Key Concepts:

Kinesis Data Streams High-Level Architecture

The following diagram illustrates the high-level architecture of Kinesis Data Streams. The *producers* continually push data to Kinesis Data Streams, and the *consumers* process the data in real time. Consumers (such as a custom application running on Amazon EC2 or an Amazon Kinesis Data Firehose delivery stream) can store their results using an AWS service such as Amazon DynamoDB, Amazon Redshift, or Amazon S3.



Kinesis Data Streams Terminology

Kinesis Data Stream

A *Kinesis data stream* is a set of shards. Each shard has a sequence of data records. Each data record has a sequence number that is assigned by Kinesis Data Streams.

Data Record

A *data record* is the unit of data stored in a Kinesis data stream. Data records are composed of a sequence number, a partition key, and a data blob, which is an immutable sequence of bytes. Kinesis Data Streams does not inspect, interpret, or change the data in the blob in any way. A data blob can be up to 1 MB.

Capacity Mode

A data stream *capacity mode* determines how capacity is managed and how you are charged for the usage of your data stream. Currenly, in Kinesis Data Streams, you can choose between an **on-demand** mode and a **provisioned** mode for your data streams. For more information, see Choosing the Data Stream Capacity Mode.

With the **on-demand** mode, Kinesis Data Streams automatically manages the shards in order to provide the necessary throughput. You are charged only for the actual throughput that you use and Kinesis Data Streams automatically accommodates your workloads' throughput needs as they ramp up or down. For more information, see On-demand Mode.

With the **provisioned** mode, you must specify the number of shards for the data stream. The total capacity of a data stream is the sum of the capacities of its shards. You can increase or decrease the number of shards in a data stream as needed and you are charged for the number of shards at an hourly rate. For more information, see Provisioned Mode.

Retention Period

The *retention period* is the length of time that data records are accessible after they are added to the stream. A stream's retention period is set to a default of 24 hours after creation. You can increase the retention period up to 8760 hours (365 days) using the IncreaseStreamRetentionPeriod operation, and decrease the retention period down to a minimum of 24 hours using the DecreaseStreamRetentionPeriod operation. Additional charges apply for streams with a retention period set to more than 24 hours. For more information, see Amazon Kinesis Data Streams Pricing.

Producer

Producers put records into Amazon Kinesis Data Streams. For example, a web server sending log data to a stream is a producer.

Amazon Kinesis Data Streams Application

An *Amazon Kinesis Data Streams application* is a consumer of a stream that commonly runs on a fleet of EC2 instances.

There are two types of consumers that you can develop: shared fan-out consumers and enhanced fan-out consumers. To learn about the differences between them, and to see how you can create each type of consumer, see Reading Data from Amazon Kinesis Data Streams.

The output of a Kinesis Data Streams application can be input for another stream, enabling you to create complex topologies that process data in real time. An application can also send data to a variety of other AWS services. There can be multiple applications for one stream, and each application can consume data from the stream independently and concurrently.

Shard

A *shard* is a uniquely identified sequence of data records in a stream. A stream is composed of one or more shards, each of which provides a fixed unit of capacity. Each shard can support up to 5 transactions per second for reads, up to a maximum total data read rate of 2 MB per second and up to 1,000 records per second for writes, up to a maximum total data write rate of 1 MB per second (including partition keys). The data capacity of your stream is a function of the number of shards that you specify for the stream. The total capacity of the stream is the sum of the capacities of its shards.

If your data rate increases, you can increase or decrease the number of shards allocated to your stream. For more information, see Resharding a Stream.

Partition Key

A *partition key* is used to group data by shard within a stream. Kinesis Data Streams segregates the data records belonging to a stream into multiple shards. It uses the partition key that is associated with each data record to determine which shard a given data record belongs to. Partition keys are Unicode strings, with a maximum length limit of 256 characters for each key. An MD5 hash function is used to map partition keys to 128-bit integer values and to map associated data records to shards using the hash key ranges of the shards. When an application puts data into a stream, it must specify a partition key.

Sequence Number

Each data record has a *sequence number* that is unique per partition-key within its shard. Kinesis Data Streams assigns the sequence number after you write to the stream with `client.putRecords` or `client.putRecord`. Sequence numbers for the same partition key generally increase over time. The longer the time period between write requests, the larger the sequence numbers become.

> **Note**
> Sequence numbers cannot be used as indexes to sets of data within the same stream. To logically separate sets of data, use partition keys or create a separate stream for each dataset.

Kinesis Client Library

The Kinesis Client Library is compiled into your application to enable fault-tolerant consumption of data from the stream. The Kinesis Client Library ensures that for every shard there is a record processor running and processing that shard. The library also simplifies reading data from the stream. The Kinesis Client Library uses an Amazon DynamoDB table to store control data. It creates one table per application that is processing data.

There are two major versions of the Kinesis Client Library. Which one you use depends on the type of consumer you want to create. For more information, see Reading Data from Amazon Kinesis Data Streams.

Application Name

The name of an Amazon Kinesis Data Streams application identifies the application. Each of your applications must have a unique name that is scoped to the AWS account and Region used by the application. This name is used as a name for the control table in Amazon DynamoDB and the namespace for Amazon CloudWatch metrics.

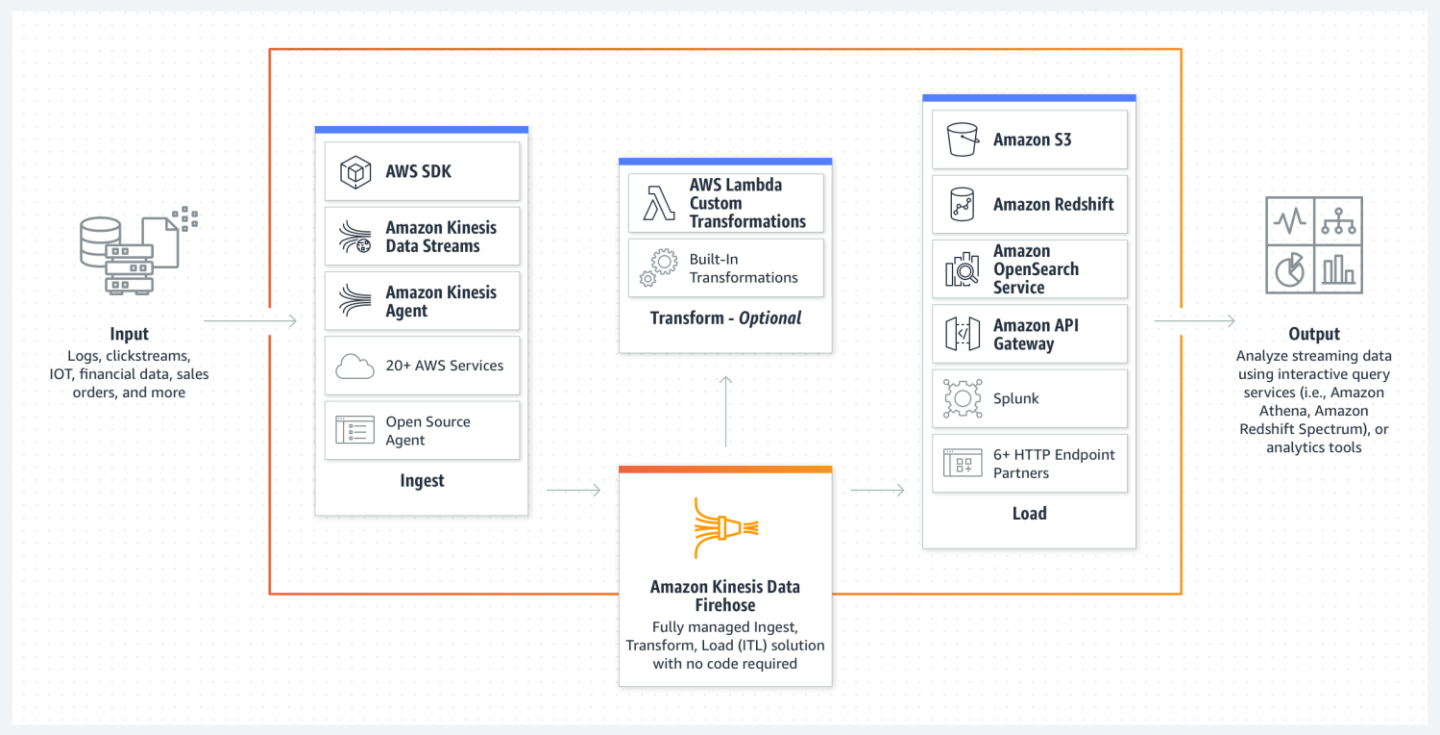
via - https://docs.aws.amazon.com/streams/latest/dev/key-concepts.html

For the given use case, you can stream the raw financial transactions into Amazon Kinesis Data Streams, which in turn, are processed by the AWS Lambda function that is set up as one of the consumers of the data stream. The Lambda would remove sensitive data from every transaction and then store the cleansed transactions in Amazon DynamoDB. The internal applications can be configured as the other consumers of the data stream and ingest the raw transactions

Incorrect options:

Batch process the raw transactions data into Amazon S3 flat files. Use S3 events to trigger an AWS Lambda function to remove sensitive data from the raw transactions in the flat file and then store the cleansed transactions in Amazon DynamoDB. Leverage DynamoDB Streams to share the transactions data with the internal applications- The use case requires a near-real-time solution for cleansing, processing and storing the transactions, so using a batch process would be incorrect.

Feed the streaming transactions into Amazon Kinesis Data Firehose. Leverage AWS Lambda integration to remove sensitive data from every transaction and then store the cleansed transactions in Amazon DynamoDB. The internal applications can consume the raw transactions off the Amazon Kinesis Data Firehose - Amazon Kinesis Data Firehose is an extract, transform, and load (ETL) service that reliably captures, transforms, and delivers streaming data to data lakes, data stores, and analytics services.



via - https://aws.amazon.com/kinesis/data-firehose/

You cannot set up multiple consumers for Amazon Kinesis Data Firehose delivery streams as it can dump data in a single data repository at a time, so this option is incorrect.

Persist the raw transactions into Amazon DynamoDB. Configure a rule in Amazon DynamoDB to update the transaction by removing sensitive data whenever any new raw transaction is written. Leverage Amazon DynamoDB Streams to share the transactions data with the internal applications - There is no such rule within Amazon DynamoDB that can auto-update every time a new item is written in a DynamoDB table. You would need to use a Amazon DynamoDB trigger to invoke an external service like a Lambda function on every new write, which can then cleanse and update the item. In addition, this process introduces inefficiency in the workflow as the same item is written and then updated for cleansing purposes. Therefore this option is incorrect.

References:

https://aws.amazon.com/kinesis/data-streams/

https://aws.amazon.com/kinesis/data-firehose/

https://docs.aws.amazon.com/streams/latest/dev/key-concepts.html

Domain

Design High-Performing Architectures

Question 3Skipped

A global enterprise is modernizing its hybrid IT infrastructure to improve both availability and network performance. The company operates a TCP-based application hosted on Amazon EC2 instances that are

deployed across multiple AWS Regions, while a secondary UDP-based component of the application is hosted in its on-premises data centers. These application components must be accessed by customers around the world with minimal latency and consistent uptime.

Which combination of options should a solutions architect implement for the given use case? (Select two)

Deploy AWS PrivateLink to connect each on-premises UDP workload to the AWS Regions through interface endpoints exposed by the Network Load Balancers

Correct selection

Create a Network Load Balancer (NLB) in each Region to handle the EC2-based TCP traffic. For the UDP-based on-premises workload, configure NLBs in each Region to route to the on-premises endpoints via IP-based target groups

Set up AWS Direct Connect connections to route all TCP and UDP traffic through a single Region, using static routes and BGP failover

Correct selection

Configure an AWS Global Accelerator standard accelerator, and register the TCP-based EC2 workloads behind the load balancers

Create a Network Load Balancer (NLB) in each Region to handle the EC2-based TCP traffic. For the UDP-based on-premises workload, configure Application Load Balancers in each Region to route to the on-premises endpoints via IP-based target groups

Overall explanation

Correct options:

Configure an AWS Global Accelerator standard accelerator, and register the TCP-based EC2 workloads behind the load balancers

AWS Global Accelerator supports improving the availability and performance of global applications with TCP-based traffic. It routes users to the closest healthy endpoint in multiple Regions via the AWS global network and supports automatic failover. Registering load balancers as endpoints ensures that the TCP workload hosted on EC2 instances benefits from low-latency access and high availability.

Create a Network Load Balancer (NLB) in each Region to handle the EC2-based TCP traffic. For the UDP-based on-premises workload, configure NLBs in each Region to route to the on-premises endpoints via IP-based target groups

Network Load Balancers support both TCP and UDP traffic. By creating NLBs in each Region, the architecture allows load balancing for EC2 workloads and can forward UDP traffic to IP-based targets in on-premises networks. This setup ensures that both stateful TCP and stateless UDP workloads are accessible and scalable from AWS.

Network Load Balancer overview

A Network Load Balancer functions at the fourth layer of the Open Systems Interconnection (OSI) model. It can handle millions of requests per second. After the load balancer receives a connection request, it selects a target from the target group for the default rule. It attempts to open a TCP connection to the selected target on the port specified in the listener configuration.

When you enable an Availability Zone for the load balancer, Elastic Load Balancing creates a load balancer node in the Availability Zone. By default, each load balancer node distributes traffic across the registered targets in its Availability Zone only. If you enable cross-zone load balancing, each load balancer node distributes traffic across the registered targets in all enabled Availability Zones. For more information, see Update the Availability Zones for your Network Load Balancer.

To increase the fault tolerance of your applications, you can enable multiple Availability Zones for your load balancer and ensure that each target group has at least one target in each enabled Availability Zone. For example, if one or more target groups does not have a healthy target in an Availability Zone, we remove the IP address for the corresponding subnet from DNS, but the load balancer nodes in the other Availability Zones are still available to route traffic. If a client doesn't honor the time-to-live (TTL) and sends requests to the IP address after it is removed from DNS, the requests fail.

For TCP traffic, the load balancer selects a target using a flow hash algorithm based on the protocol, source IP address, source port, destination IP address, destination port, and TCP sequence number. The TCP connections from a client have different source ports and sequence numbers, and can be routed to different targets. Each individual TCP connection is routed to a single target for the life of the connection.

For UDP traffic, the load balancer selects a target using a flow hash algorithm based on the protocol, source IP address, source port, destination IP address, and destination port. A UDP flow has the same source and destination, so it is consistently routed to a single target throughout its lifetime. Different UDP flows have different source IP addresses and ports, so they can be routed to different targets.

Elastic Load Balancing creates a network interface for each Availability Zone you enable. Each load balancer node in the Availability Zone uses this network interface to get a static IP address. When you create an Internet-facing load balancer, you can optionally associate one Elastic IP address per subnet.

When you create a target group, you specify its target type, which determines how you register targets. For example, you can register instance IDs, IP addresses, or an Application Load Balancer. The target type also affects whether the client IP addresses are preserved. For more information, see Client IP preservation.

You can add and remove targets from your load balancer as your needs change, without disrupting the overall flow of requests to your application. Elastic Load Balancing scales your load balancer as traffic to your application changes over time. Elastic Load Balancing can scale to the vast majority of workloads automatically.

You can configure health checks, which are used to monitor the health of the registered targets so that the load balancer can send requests only to the healthy targets.

via - https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

Incorrect Options:

Set up AWS Direct Connect connections to route all TCP and UDP traffic through a single Region, using static routes and BGP failover - AWS Direct Connect enhances dedicated network performance between AWS and on-premises infrastructure but does not inherently provide intelligent global routing, failover, or multi-region resilience. It is not optimal for distributing traffic across Regions or managing global availability.

Deploy AWS PrivateLink to connect each on-premises UDP workload to the AWS Regions through interface endpoints exposed by the Network Load Balancers - AWS PrivateLink enables secure connectivity to AWS services via VPC endpoints but supports only TCP-based traffic. It cannot be used to route or expose UDP-based services, making it unsuitable for the on-premises UDP workload.

Create a Network Load Balancer (NLB) in each Region to handle the EC2-based TCP traffic. For the UDP-based on-premises workload, configure Application Load Balancers in each Region to route to the on-premises endpoints via IP-based target groups - While the first part of the option is valid — using NLBs to handle EC2-based TCP traffic is appropriate because NLBs support TCP connections and can scale efficiently across Regions. The second part of the option is flawed. The issue lies in attempting to use Application Load Balancers (ALBs) for UDP-based traffic. ALBs only support HTTP/HTTPS (Layer 7) and WebSocket protocols. They do not support Layer 4 protocols such as TCP or UDP.

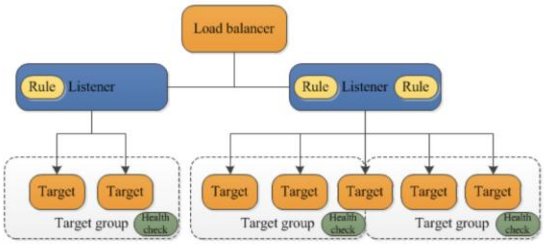
Application Load Balancer components

A *load balancer* serves as the single point of contact for clients. The load balancer distributes incoming application traffic across multiple targets, such as EC2 instances, in multiple Availability Zones. This increases the availability of your application. You add one or more listeners to your load balancer.

A *listener* checks for connection requests from clients, using the protocol and port that you configure. The rules that you define for a listener determine how the load balancer routes requests to its registered targets. Each rule consists of a priority, one or more actions, and one or more conditions. When the conditions for a rule are met, then its actions are performed. You must define a default rule for each listener, and you can optionally define additional rules.

Each *target group* routes requests to one or more registered targets, such as EC2 instances, using the protocol and port number that you specify. You can register a target with multiple target groups. You can configure health checks on a per target group basis. Health checks are performed on all targets registered to a target group that is specified in a listener rule for your load balancer.

The following diagram illustrates the basic components. Notice that each listener contains a default rule, and one listener contains another rule that routes requests to a different target group. One target is registered with two target groups.



For more information, see the following documentation:

- Load balancers
- Listeners
- Target groups

Application Load Balancer overview

An Application Load Balancer functions at the application layer, the seventh layer of the Open Systems Interconnection (OSI) model. After the load balancer receives a request, it evaluates the listener rules in priority order to determine which rule to apply, and then selects a target from the target group for the rule action. You can configure listener rules to route requests to different target groups based on the content of the application traffic. Routing is performed independently for each target group, even when a target is registered with multiple target groups. You can configure the routing algorithm used at the target group level. The default routing algorithm is round robin; alternatively, you can specify the least outstanding requests routing algorithm.

You can add and remove targets from your load balancer as your needs change, without disrupting the overall flow of requests to your application. Elastic Load Balancing scales your load balancer as traffic to your application changes over time. Elastic Load Balancing can scale to the vast majority of workloads automatically.

You can configure health checks, which are used to monitor the health of the registered targets so that the load balancer can send requests only to the healthy targets.

via - https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

References:

https://docs.aws.amazon.com/global-accelerator/latest/dg/what-is-global-accelerator.html

https://docs.aws.amazon.com/elasticloadbalancing/latest/network/introduction.html

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/introduction.html

Domain

Design High-Performing Architectures

Question 4Skipped

A company wants to ensure high availability for its Amazon RDS database. The development team wants to opt for Multi-AZ deployment and they would like to understand what happens when the primary instance of the Multi-AZ configuration goes down.

As a Solutions Architect, which of the following will you identify as the outcome of the scenario?

Correct answer

The CNAME record will be updated to point to the standby database

An email will be sent to the System Administrator asking for manual intervention

The URL to access the database will change to the standby database

The application will be down until the primary database has recovered itself

Overall explanation

Correct option:

The CNAME record will be updated to point to the standby database

Amazon RDS provides high availability and failover support for DB instances using Multi-AZ deployments. Amazon RDS uses several different technologies to provide failover support. Multi-AZ deployments for MariaDB, MySQL, Oracle, and PostgreSQL DB instances use Amazon's failover technology. SQL Server DB instances use SQL Server Database Mirroring (DBM) or Always On Availability Groups (AGs).

In a Multi-AZ deployment, Amazon RDS automatically provisions and maintains a synchronous standby replica in a different Availability Zone. The primary DB instance is synchronously replicated across Availability Zones to a standby replica to provide data redundancy, eliminate I/O freezes, and minimize latency spikes during system backups. Running a DB instance with high availability can enhance availability during planned system maintenance, and help protect your databases against DB instance failure and Availability Zone disruption.

Failover is automatically handled by Amazon RDS so that you can resume database operations as quickly as possible without administrative intervention. When failing over, Amazon RDS simply flips the canonical name record (CNAME) for your DB instance to point at the standby, which is in turn promoted to become the new primary. Multi-AZ means the URL is the same, the failover is automated, and the CNAME will automatically be updated to point to the standby database.

Incorrect options:

The URL to access the database will change to the standby database - As discussed above, URL remains the same.

An email will be sent to the System Administrator asking for manual intervention - This option is incorrect and it has been added as a distractor.

The application will be down until the primary database has recovered itself - This option is incorrect and it has been added as a distractor.

References:

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Concepts.MultiAZ.html

https://aws.amazon.com/rds/faqs/

Domain

Design Resilient Architectures

Question 5Skipped

A big data analytics company is using Amazon Kinesis Data Streams (KDS) to process IoT data from the field devices of an agricultural sciences company. Multiple consumer applications are using the incoming data streams and the engineers have noticed a performance lag for the data delivery speed between producers and consumers of the data streams.

As a solutions architect, which of the following would you recommend for improving the performance for the given use-case?

Swap out Amazon Kinesis Data Streams with Amazon SQS FIFO queues

Correct answer

Use Enhanced Fanout feature of Amazon Kinesis Data Streams

Swap out Amazon Kinesis Data Streams with Amazon Kinesis Data Firehose

Swap out Amazon Kinesis Data Streams with Amazon SQS Standard queues

Overall explanation

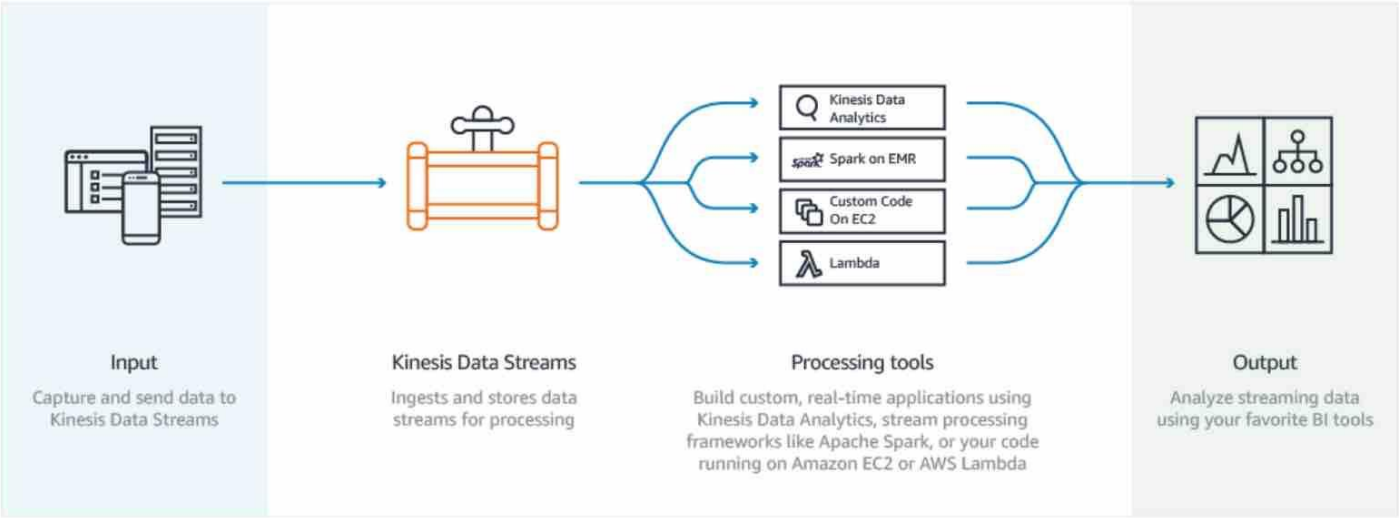
Correct option:

Use Enhanced Fanout feature of Amazon Kinesis Data Streams

Amazon Kinesis Data Streams (KDS) is a massively scalable and durable real-time data streaming service. KDS can continuously capture gigabytes of data per second from hundreds of thousands of sources such as website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events.

By default, the 2MB/second/shard output is shared between all of the applications consuming data from the stream. You should use enhanced fan-out if you have multiple consumers retrieving data from a stream in parallel. With enhanced fan-out developers can register stream consumers to use enhanced fan-out and receive their own 2MB/second pipe of read throughput per shard, and this throughput automatically scales with the number of shards in a stream.

Amazon Kinesis Data Streams Fanout:



Kinesis Data Streams are scaled using the concept of a **shard**. One shard provides an ingest capacity of 1MB/second or 1000 records/second and an output capacity of 2MB/second. It's not uncommon for customers to have thousands or tens of thousands of shards supporting 10s of GB/sec of ingest and egress. Before the enhanced fan-out capability, that 2MB/second/shard output was shared between all of the applications consuming data from the stream. With enhanced fan-out developers can register stream consumers to use enhanced fan-out and receive their own 2MB/second pipe of read throughput per shard, and this throughput automatically scales with the number of shards in a stream. Prior to the launch of Enhanced Fan-out customers would frequently fan-out their data out to multiple streams to support their desired read throughput for their downstream applications. That sounds like undifferentiated heavy lifting to us, and that's something we decided our customers shouldn't need to worry about. Customers pay for enhanced fan-out based on the amount of data retrieved from the stream using enhanced fan-out and the number of consumers registered per-shard. You can find additional info on the pricing page.

via - https://aws.amazon.com/blogs/aws/kds-enhanced-fanout/

Incorrect options:

Swap out Amazon Kinesis Data Streams with Amazon Kinesis Data Firehose - Amazon Kinesis Data Firehose is the easiest way to reliably load streaming data into data lakes, data stores, and analytics tools. It is a fully managed service that automatically scales to match the throughput of your data and requires no ongoing administration. It can also batch, compress, transform, and encrypt the data before loading it, minimizing the amount of storage used at the destination and increasing security. Amazon Kinesis Data Firehose can only write to Amazon S3, Amazon Redshift, Amazon Elasticsearch or Splunk. You can't have applications consuming data streams from Amazon Kinesis Data Firehose, that's the job of Amazon Kinesis Data Streams. Therefore this option is not correct.

Swap out Amazon Kinesis Data Streams with Amazon SQS Standard queues

Swap out Amazon Kinesis Data Streams with Amazon SQS FIFO queues

Amazon Simple Queue Service (Amazon SQS) is a fully managed message queuing service that enables you to decouple and scale microservices, distributed systems, and serverless applications. Amazon SQS offers two types of message queues. Standard queues offer maximum throughput, best-effort ordering, and at-least-once delivery. Amazon SQS FIFO queues are designed to guarantee that messages are processed exactly once, in the exact order that they are sent. As multiple applications are consuming the

same stream concurrently, both Amazon SQS Standard and Amazon SQS FIFO are not the right fit for the given use-case.

Exam Alert:

Please understand the differences between the capabilities of Amazon Kinesis Data Streams vs Amazon SQS, as you may be asked scenario-based questions on this topic in the exam.

Q: When should I use Amazon Kinesis Data Streams, and when should I use Amazon SQS?

We recommend Amazon Kinesis Data Streams for use cases with requirements that are similar to the following:

- Routing related records to the same record processor (as in streaming MapReduce). For example, counting and aggregation are simpler when all records for a given key are routed to the same record processor.
- Ordering of records. For example, you want to transfer log data from the application host to the processing/archival host while maintaining the order of log statements.
- Ability for multiple applications to consume the same stream concurrently. For example, you have one application that updates a real-time dashboard and another that archives data to Amazon Redshift. You want both applications to consume data from the same stream concurrently and independently.
- Ability to consume records in the same order a few hours later. For example, you have a billing application and an audit application that runs a few hours behind the billing application. Because Amazon Kinesis Data Streams stores data for up to 7 days, you can run the audit application up to 7 days behind the billing application.

We recommend Amazon SQS for use cases with requirements that are similar to the following:

- Messaging semantics (such as message-level ack/fail) and visibility timeout. For example, you have a queue of work items and want to track the successful completion of each item independently. Amazon SQS tracks the ack/fail, so the application does not have to maintain a persistent checkpoint/cursor. Amazon SQS will delete acked messages and redeliver failed messages after a configured visibility timeout.
- Individual message delay. For example, you have a job queue and need to schedule individual jobs with a delay. With Amazon SQS, you can configure individual messages to have a delay of up to 15 minutes.
- Dynamically increasing concurrency/throughput at read time. For example, you have a work queue and want to add more readers until the backlog is cleared. With Amazon Kinesis Data Streams, you can scale up to a sufficient number of shards (note, however, that you'll need to provision enough shards ahead of time).
- Leveraging Amazon SQS's ability to scale transparently. For example, you buffer requests and the load changes as a result of occasional load spikes or the natural growth of your business. Because each buffered request can be processed independently, Amazon SQS can scale transparently to handle the load without any provisioning instructions from you.

via - https://aws.amazon.com/kinesis/data-streams/faqs/

References:

https://aws.amazon.com/blogs/aws/kds-enhanced-fanout/

https://aws.amazon.com/kinesis/data-streams/faqs/

Domain

Design High-Performing Architectures

Question 6Skipped

A financial data processing company runs a workload on Amazon EC2 instances that fetch and process real-time transaction batches from an Amazon SQS queue. The application needs to scale based on unpredictable message volume, which fluctuates significantly throughout the day. The system must process messages with minimal delay and no downtime, even during peak spikes. The company is seeking a solution that balances cost-efficiency with availability and elasticity.

Which EC2 purchasing strategy best meets these requirements in the most cost-effective manner?

Use EC2 Reserved Instances for the baseline workload and configure EC2 Auto Scaling to launch On-Demand Instances for all traffic spikes

Use EC2 Spot Instances exclusively with Auto Scaling enabled to match message volume fluctuations and save on compute costs

Correct answer

Use Reserved Instances for the baseline level of traffic and configure EC2 Auto Scaling with Spot Instances to handle spikes in message volume

Purchase EC2 Reserved Instances to match peak capacity and assign all message processing tasks to these instances regardless of load variations

Overall explanation

Correct option:

Use Reserved Instances for the baseline level of traffic and configure EC2 Auto Scaling with Spot Instances to handle spikes in message volume

This blended approach is ideal for workloads that are partially predictable and partially variable. Reserved Instances ensure cost-efficient baseline capacity for steady message ingestion, while Spot Instances are used to handle unpredictable surges in traffic, significantly reducing costs compared to On-Demand. If Spot capacity is unavailable, Auto Scaling can be configured with fallback instance types or prioritized mixed instance policies. This model maximizes cost savings without compromising on availability.

Incorrect options:

Purchase EC2 Reserved Instances to match peak capacity and assign all message processing tasks to these instances regardless of load variations - While Reserved Instances (RIs) offer significant cost savings compared to On-Demand pricing, they are best suited for predictable, steady-state workloads. Using RIs to provision maximum capacity at all times, regardless of actual load, results in over-provisioning and wasted cost during low-traffic periods. This approach lacks elasticity and is not suitable for workloads with unpredictable and bursty traffic patterns.

Use EC2 Spot Instances exclusively with Auto Scaling enabled to match message volume fluctuations and save on compute costs - Spot Instances provide deep cost savings, but they can be

interrupted by AWS with only two minutes of warning if capacity is reclaimed. Using them exclusively for a mission-critical workload with real-time, parallel processing introduces risk of downtime and message loss, especially if traffic spikes coincide with Spot interruptions. For fault-tolerant or batch jobs, this could work—but not for always-on processing with strict uptime requirements.

Use EC2 Reserved Instances for the baseline workload and configure EC2 Auto Scaling to launch On-Demand Instances for all traffic spikes - While combining Reserved Instances for baseline capacity with On-Demand Instances for additional traffic is a functional and reliable solution, it is not the most cost-effective choice for a workload with unpredictable, intermittent spikes like real-time message processing. On-Demand Instances are priced at a premium and can lead to significantly higher costs when scaling frequently. A blended model using Reserved + Spot Instances would better balance availability and cost.

References:

https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-mixed-instances-groups.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/using-spot-instances.html

https://docs.aws.amazon.com/savingsplans/latest/userguide/what-is-savings-plans.html

Domain

Design High-Performing Architectures

Question 7Skipped

An e-commerce company uses a two-tier architecture with application servers in the public subnet and an Amazon RDS MySQL DB in a private subnet. The development team can use a bastion host in the public subnet to access the MySQL database and run queries from the bastion host. However, end-users are reporting application errors. Upon inspecting application logs, the team notices several "could not connect to server: connection timed out" error messages.

Which of the following options represent the root cause for this issue?

The database user credentials (username and password) configured for the application do not have the required privilege for the given database

The database user credentials (username and password) configured for the application are incorrect

Correct answer

The security group configuration for the database instance does not have the correct rules to allow inbound connections from the application servers

The security group configuration for the application servers does not have the correct rules to allow inbound connections from the database instance

Overall explanation

Correct option:

The security group configuration for the database instance does not have the correct rules to allow inbound connections from the application servers

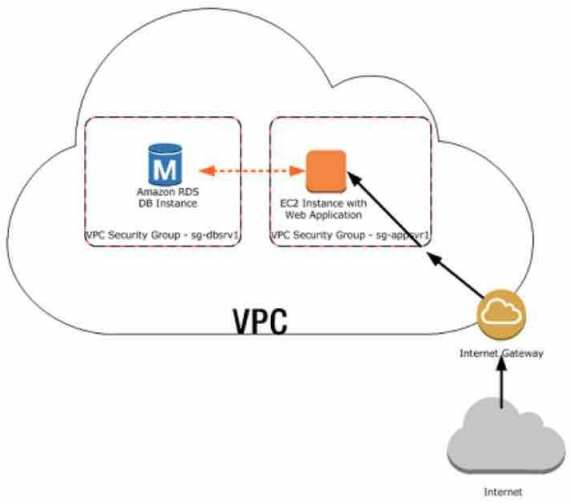
You should use security groups to control the inbound and outbound traffic for your database instance. For your application servers, create a security group with inbound rules that use the IP addresses of the client application as the source. This security group allows your client application to connect to your application servers. Then create a second security group for your database instance and create a new rule by specifying the security group that you created earlier as the source for this database-specific security group.

Security Group Scenario

A common use of a DB instance in a VPC is to share data with an application server running in an Amazon EC2 instance in the same VPC, which is accessed by a client application outside the VPC. For this scenario, you use the RDS and VPC pages on the AWS Management Console or the RDS and EC2 API operations to create the necessary instances and security groups:

1. Create a VPC security group (for example, `sg-appsrv1`) and define inbound rules that use the IP addresses of the client application as the source. This security group allows your client application to connect to EC2 instances in a VPC that uses this security group.
2. Create an EC2 instance for the application and add the EC2 instance to the VPC security group (`sg-appsrv1`) that you created in the previous step. The EC2 instance in the VPC shares the VPC security group with the DB instance.
3. Create a second VPC security group (for example, `sg-dbsrv1`) and create a new rule by specifying the VPC security group that you created in step 1 (`sg-appsrv1`) as the source.
4. Create a new DB instance and add the DB instance to the VPC security group (`sg-dbsrv1`) that you created in the previous step. When you create the DB instance, use the same port number as the one specified for the VPC security group (`sg-dbsrv1`) rule that you created in step 3.

The following diagram shows this scenario.



via - https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.RDSSecurityGroups.html

Incorrect options:

The security group configuration for the application servers does not have the correct rules to allow inbound connections from the database instance - As mentioned in the explanation above, the

application servers don't need inbound connections from the database instance, rather the database instance needs the correct inbound rule with application servers' security group as the source.

The database user credentials (username and password) configured for the application are incorrect

The database user credentials (username and password) configured for the application do not have the required privilege for the given database

These two options have been added as a distractor since the error mentions a "connection timeout" issue rather than an "access denied" error.

References:

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/Overview.RDSSecurityGroups.html

https://aws.amazon.com/premiumsupport/knowledge-center/rds-cannot-connect/

Domain

Design Secure Architectures

Question 8Skipped

A tech company runs a web application that includes multiple internal services deployed across Amazon EC2 instances within a VPC. These services require communication with a third-party SaaS provider's API for analytics and billing, which is also hosted on the AWS infrastructure. The company is concerned about minimizing public internet exposure while maintaining secure and reliable connectivity. The solution must ensure private access without allowing unsolicited incoming traffic from the SaaS provider.

Which solution will best meet these requirements?

Correct answer

Use AWS PrivateLink to create a private endpoint within the application’s VPC that connects securely to the SaaS provider’s VPC

Use AWS CloudFront to route requests from the application’s internal services to the SaaS provider through edge locations

Set up VPC peering between the application VPC and the SaaS provider’s VPC to allow direct communication

Establish a VPN connection using AWS Site-to-Site VPN to create a secure tunnel between the internal services and the third-party SaaS provider

Overall explanation

Correct option:

Use AWS PrivateLink to create a private endpoint within the application’s VPC that connects securely to the SaaS provider’s VPC

AWS PrivateLink is a fully managed service that enables private connectivity between Virtual Private Clouds (VPCs), AWS services, and supported third-party SaaS applications over the AWS network, without exposing traffic to the public internet. In this case, the internal service components of the application can securely access the third-party SaaS APIs by creating an interface VPC endpoint for the SaaS provider's service.

This endpoint appears as an elastic network interface (ENI) in the application's VPC, allowing internal services to communicate with the SaaS API using private IP addresses. Importantly, the traffic never leaves the Amazon network, which significantly reduces the risk of data exposure or interception. Additionally, the SaaS provider cannot initiate connections to the application's services because PrivateLink is inherently unidirectional from the service consumer (the company's VPC) to the service provider (the SaaS provider's VPC). This architecture helps meet strict security and compliance requirements, such as zero-trust networking and least privilege access.

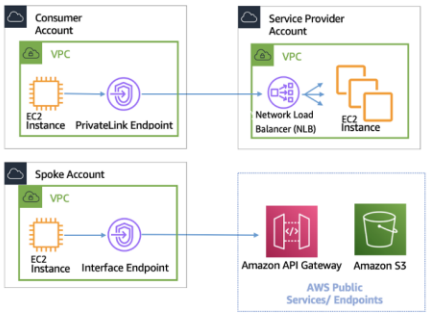
AWS PrivateLink

PDF | RSS | Focus mode

AWS PrivateLink provides private connectivity between VPCs, AWS services, and your on-premises networks without exposing your traffic to the public internet. Interface VPC endpoints, powered by AWS PrivateLink, make it easy to connect to AWS and other services across different accounts and VPCs to significantly simplify your network architecture. This allows customers who may want to privately expose a service/application residing in one VPC (service provider) to other VPCs (consumer) within an AWS Region in a way that only consumer VPCs initiate connections to the service provider VPC. An example of this is the ability for your private applications to access service provider APIs.

To use AWS PrivateLink, create a Network Load Balancer for your application in your VPC, and create a VPC endpoint service configuration pointing to that load balancer. A service consumer then creates an interface endpoint to your service. This creates an elastic network interface (ENI) in the consumer subnet with a private IP address that serves as an entry point for traffic destined for the service. The consumer and service are not required to be in the same VPC. If the VPC is different, the consumer and service provider VPCs can have overlapping IP address ranges. In addition to creating the interface VPC endpoint to access services in other VPCs, you can create interface VPC endpoints to privately access supported AWS services through AWS PrivateLink, as shown in the following figure.

With Application Load Balancer (ALB) as target of NLB, you can now combine ALB advanced routing capabilities with AWS PrivateLink. Refer to Application Load Balancer-type Target Group for Network Load Balancer for reference architectures and detailed configuration.



AWS PrivateLink for connectivity to other VPCs and AWS Services

The choice between Transit Gateway, VPC peering, and AWS PrivateLink is dependent on connectivity.

- **AWS PrivateLink** — Use AWS PrivateLink when you have a client/server set up where you want to allow one or more consumer VPCs unidirectional access to a specific service or set of instances in the service provider VPC or certain AWS services. Only the clients with access in the consumer VPC can initiate a connection to the service in the service provider VPC or AWS service. This is also a good option when client and servers in the two VPCs have overlapping IP addresses because AWS PrivateLink uses ENIs within the client VPC in a manner that ensures that are no IP conflicts with the service provider. You can access AWS PrivateLink endpoints over VPC peering, VPN, Transit Gateway, Cloud WAN, and AWS Direct Connect.
- **VPC peering and Transit Gateway** — Use VPC peering and Transit Gateway when you want to enable layer-3 IP connectivity between VPCs.

  Your architecture will contain a mix of these technologies in order to fulfill different use cases. All of these services can be combined and operated with each other. For example, AWS PrivateLink handling API style client-server connectivity, VPC peering for handling direct connectivity requirements where placement groups may still be desired within the Region or inter-Region connectivity is needed, and Transit Gateway to simplify connectivity of VPCs at scale as well as edge consolidation for hybrid connectivity.

via - https://docs.aws.amazon.com/whitepapers/latest/building-scalable-secure-multi-vpc-network-infrastructure/aws-privatelink.html

Incorrect options:

Establish a VPN connection using AWS Site-to-Site VPN to create a secure tunnel between the internal services and the third-party SaaS provider - While Site-to-Site VPN can establish a secure connection, it typically requires IPsec-compatible devices and may route traffic over the internet, depending on the SaaS provider's setup. It also introduces more operational complexity and latency compared to AWS-native private connectivity options.

Set up VPC peering between the application VPC and the SaaS provider's VPC to allow direct communication - VPC peering enables direct network connectivity between VPCs; however, it lacks the granular access controls that AWS PrivateLink provides. Unlike PrivateLink, VPC peering allows broader network-level access, which can potentially permit unsolicited inbound traffic from the SaaS provider. Additionally, due to limitations in scalability and security management, SaaS providers typically do not support peering with individual customer VPCs.

Use AWS CloudFront to route requests from the application's internal services to the SaaS provider through edge locations - CloudFront is designed for content distribution and caching at edge locations. It does not establish private or secure backend connectivity between internal services and SaaS APIs, and traffic still traverses the public internet.

References:

https://docs.aws.amazon.com/vpc/latest/privatelink/what-is-privatelink.html

https://docs.aws.amazon.com/whitepapers/latest/building-scalable-secure-multi-vpc-network-infrastructure/aws-privatelink.html

Domain

Design Secure Architectures

Question 9Skipped

An organization operates a legacy reporting tool hosted on an Amazon EC2 instance located within a public subnet of a VPC. This tool aggregates scanned PDF reports from field devices and temporarily stores them on an attached Amazon EBS volume. At the end of each day, the tool transfers the accumulated files to an Amazon S3 bucket for archival. A solutions architect identifies that the files are being uploaded over the internet using S3's public endpoint. To improve security and avoid exposing data traffic to the public internet, the architect needs to reconfigure the setup so that uploads to Amazon S3 occur privately without using the public S3 endpoint.

Which solution will fulfill these requirements?

Provision a dedicated AWS Direct Connect link to route traffic from the VPC to Amazon S3 privately

Set up a NAT gateway in the public subnet and modify the route table of the EC2 instance's subnet to direct Amazon S3 traffic through the NAT gateway

Correct answer

Create a gateway VPC endpoint for Amazon S3 in the VPC. Ensure that the EC2 instance's subnet route table is updated to route S3 traffic through the endpoint. Confirm that appropriate IAM policies are in place to permit access via the VPC endpoint

Create an S3 access point within the same Region and attach a policy that grants the EC2 instance access. Update the application to use the access point alias to upload data

Overall explanation

Correct option:

Create a gateway VPC endpoint for Amazon S3 in the VPC. Ensure that the EC2 instance's subnet route table is updated to route S3 traffic through the endpoint. Confirm that appropriate IAM policies are in place to permit access via the VPC endpoint

A gateway VPC endpoint for Amazon S3 enables private connectivity between your Amazon VPC and S3 without routing traffic over the public internet. By creating a gateway VPC endpoint, the EC2 instance uploads data to S3 over AWS's internal network, ensuring the traffic remains entirely within the AWS infrastructure. This approach satisfies the requirement to avoid using the public endpoint, enhances security, and minimizes exposure to external threats. Additionally, route table entries in the public subnet can be updated to direct all S3-bound traffic through the VPC endpoint, and IAM or bucket policies can further restrict access to only traffic coming from the VPC endpoint. This solution requires no application changes and involves minimal operational overhead, making it both secure and efficient.

There is no additional charge for using gateway endpoints. Amazon S3 supports both gateway endpoints and interface endpoints. With a gateway endpoint, you can access Amazon S3 from your VPC, without requiring an internet gateway or NAT device for your VPC, and with no additional cost. However, gateway endpoints do not allow access from on-premises networks, from peered VPCs in other AWS Regions, or through a transit gateway.

Incorrect options:

Create an S3 access point within the same Region and attach a policy that grants the EC2 instance access. Update the application to use the access point alias to upload data - While Amazon S3 access points simplify managing access to shared buckets across many applications or teams, they do not change the network path used to reach Amazon S3. Without a VPC endpoint, traffic using an access point still flows over the public internet. This option does not meet the requirement to avoid using S3's public endpoint.

Set up a NAT gateway in the public subnet and modify the route table of the EC2 instance's subnet to direct Amazon S3 traffic through the NAT gateway - Although NAT gateways can route traffic to the internet, they do not provide private access to S3. Traffic sent through a NAT gateway to S3 still uses public endpoints and incurs data transfer charges. This does not satisfy the requirement to avoid public endpoint usage.

Provision a dedicated AWS Direct Connect link to route traffic from the VPC to Amazon S3 privately - Direct Connect is ideal for high-throughput, low-latency, hybrid workloads where traffic flows from on-premises to AWS. However, it is unnecessary for this use case, which involves internal traffic between EC2 and S3 within AWS.

References:

https://docs.aws.amazon.com/vpc/latest/privatelink/vpc-endpoints-s3.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-points.html

Domain

Design Secure Architectures

Question 10Skipped

A fintech company currently operates a real-time search and analytics platform on-premises. This platform ingests streaming data from multiple data-producing systems and provides immediate search capabilities and interactive visualizations for end users. As part of its cloud migration strategy, the company wants to rearchitect the solution using AWS-native services.

Which of the following represents the most efficient solution?

Deploy Amazon EC2 instances to handle the ingestion and processing of streaming data, storing the results in Amazon S3. Utilize Amazon Athena to search the stored data, and use Amazon Managed Grafana to generate dashboards and visual insights

Correct answer

Ingest and process the streaming data using Amazon Kinesis Data Streams, then index the data with Amazon OpenSearch Service for real-time search capabilities. Use Amazon QuickSight to build interactive dashboards and visualizations based on the indexed data

Use AWS Glue streaming ETL to process data streams and load the data into Amazon Redshift. Use Amazon Redshift's full-text search capabilities for querying. Use Amazon QuickSight for data visualizations

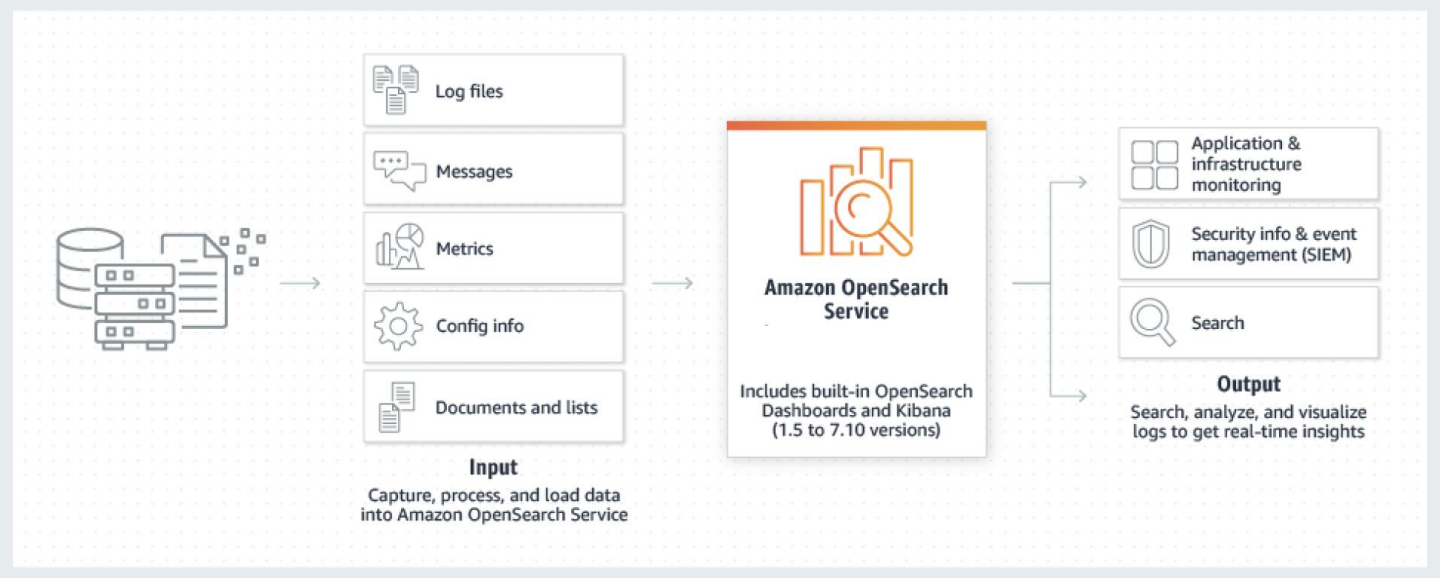
Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate to ingest the data into Amazon DynamoDB to facilitate full text search. Use Amazon CloudWatch to create dashboards and query the data

Overall explanation

Correct option:

Ingest and process the streaming data using Amazon Kinesis Data Streams, then index the data with Amazon OpenSearch Service for real-time search capabilities. Use Amazon QuickSight to build interactive dashboards and visualizations based on the indexed data

This solution leverages Amazon Kinesis Data Streams for high-throughput, low-latency stream ingestion, and Amazon OpenSearch Service for full-text indexing and real-time search. OpenSearch is purpose-built for search and analytics workloads. For visualization, Amazon QuickSight can be connected to OpenSearch or other data sources to build rich dashboards. This architecture provides a scalable, fully managed, real-time search and analytics platform using AWS-native services.



via - https://docs.aws.amazon.com/opensearch-service/latest/developerguide/what-is.html

Incorrect options:

Deploy Amazon EC2 instances to handle the ingestion and processing of streaming data, storing the results in Amazon S3. Utilize Amazon Athena to search the stored data, and use Amazon Managed Grafana to generate dashboards and visual insights - While this solution uses serverless analytics via Amazon Athena and integrates with visualization tools like Amazon Managed Grafana, the use of Amazon EC2 to ingest and process real-time streaming data adds unnecessary operational overhead. Additionally, Athena is optimized for querying data on Amazon S3 rather than low-latency, real-time search requirements. So, this architecture lacks the responsiveness and indexing performance needed for real-time search applications.

Use AWS Glue streaming ETL to process data streams and load the data into Amazon Redshift. Use Amazon Redshift's full-text search capabilities for querying. Use Amazon QuickSight for data visualizations - Although AWS Glue streaming ETL can handle data ingestion and Redshift supports powerful analytics, Redshift is primarily a data warehouse and not optimized for low-latency, full-text search. Redshift's search capabilities are limited compared to dedicated search engines like OpenSearch. Furthermore, the combination of Glue and Redshift adds complexity and may introduce latency for real-time applications.

Use Amazon Elastic Container Service (Amazon ECS) with AWS Fargate to ingest the data into Amazon DynamoDB to facilitate full text search. Use Amazon CloudWatch to create dashboards and query the data - DynamoDB is a NoSQL database optimized for key-value access, not full-text search. Additionally, Amazon CloudWatch is designed for infrastructure and application monitoring, not advanced search or visualization of dynamic user data. This setup lacks the native indexing and real-time query capabilities required for a robust search experience.

References:

https://docs.aws.amazon.com/streams/latest/dev/introduction.html

https://docs.aws.amazon.com/opensearch-service/latest/developerguide/what-is.html

https://docs.aws.amazon.com/quicksight/latest/user/welcome.html

Domain

Design Resilient Architectures

Question 11Skipped

You are a cloud architect at an IT company. The company has multiple enterprise customers that manage their own mobile applications that capture and send data to Amazon Kinesis Data Streams. They have been getting a ProvisionedThroughputExceededException exception. You have been contacted to help and upon analysis, you notice that messages are being sent one by one at a high rate.

Which of the following options will help with the exception while keeping costs at a minimum?

Use Exponential Backoff

Correct answer

Use batch messages

Decrease the Stream retention duration

Increase the number of shards

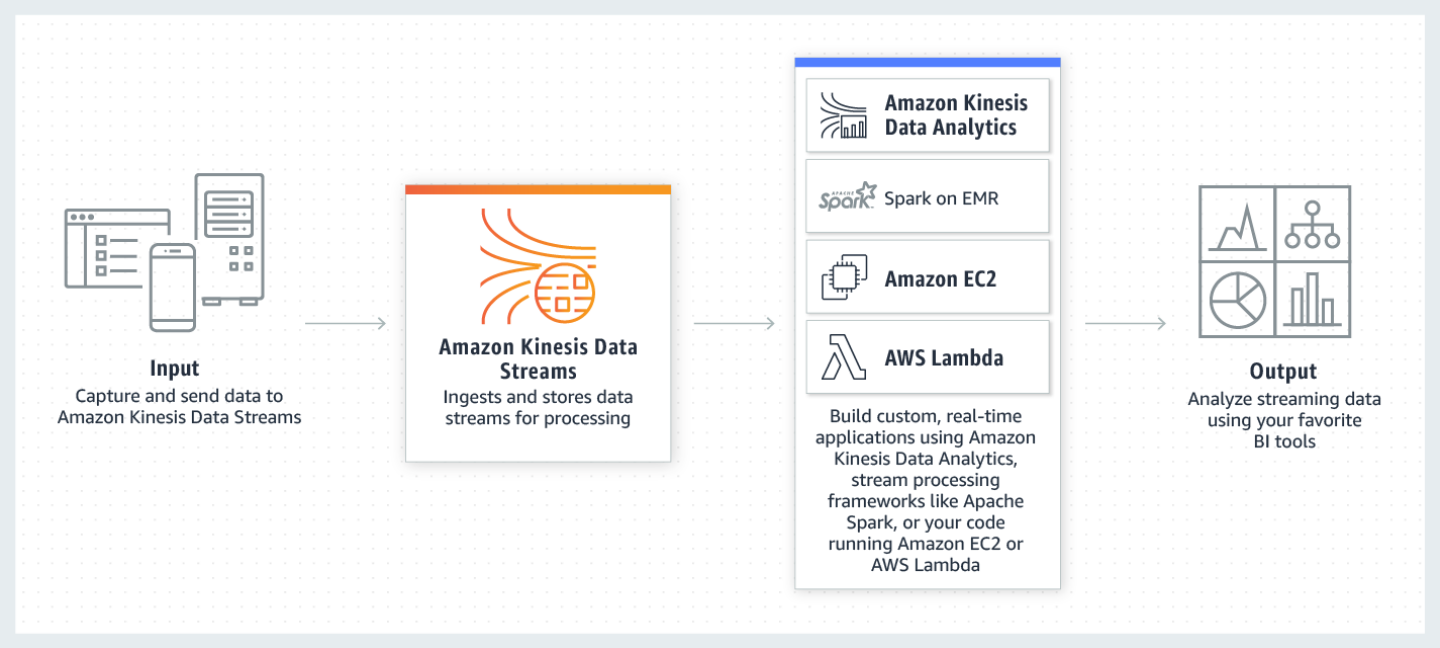
Overall explanation

Correct option:

Use batch messages

Amazon Kinesis Data Streams (KDS) is a massively scalable and durable real-time data streaming service. KDS can continuously capture gigabytes of data per second from hundreds of thousands of sources such as website clickstreams, database event streams, financial transactions, social media feeds, IT logs, and location-tracking events. The data collected is available in milliseconds to enable real-time analytics use cases such as real-time dashboards, real-time anomaly detection, dynamic pricing, and more.

Amazon Kinesis Data Streams Overview:



via - https://aws.amazon.com/kinesis/data-streams/

When a host needs to send many records per second (RPS) to Amazon Kinesis, simply calling the basic PutRecord API action in a loop is inadequate. To reduce overhead and increase throughput, the application must batch records and implement parallel HTTP requests. This will increase the efficiency overall and ensure you are optimally using the shards.

Incorrect options:

Use Exponential Backoff - While this may help in the short term, as soon as the request rate increases, you will see the ProvisionedThroughputExceededException exception again.

Increase the number of shards - Increasing shards could be a short term fix but will substantially increase the cost, so this option is ruled out.

Decrease the Stream retention duration - This operation may result in data loss and won't help with the exceptions, so this option is incorrect.

References:

https://aws.amazon.com/blogs/big-data/implementing-efficient-and-reliable-producers-with-the-amazon-kinesis-producer-library/

https://aws.amazon.com/kinesis/data-streams/

Domain

Design High-Performing Architectures

Question 12Skipped

A SaaS analytics company is deploying a microservices-based application on Amazon ECS using the Fargate launch type. The application requires access to a shared, POSIX-compliant file system that is

available across multiple Availability Zones for redundancy and availability. To meet compliance requirements, the system must support regional backups and cross-Region data recovery with a recovery point objective (RPO) of no more than 8 hours. A backup strategy will be implemented using AWS Backup to automate replication across Regions. As the lead cloud architect, you are evaluating file storage solutions that align with these requirements.

Which option best meets the application's availability, durability, and RPO objectives?

Deploy Amazon FSx for Lustre and configure a backup plan using AWS Backup for cross-Region replication of the file system metadata

Correct answer

Use Amazon Elastic File System (Amazon EFS) with the Standard storage class and configure AWS Backup to create cross-Region backups on a scheduled basis

Use Amazon FSx for NetApp ONTAP with a Multi-AZ deployment and rely on its native high availability and AWS Backup integration to replicate the file system to another Region automatically

Configure Amazon S3 with the S3 Standard storage class and mount it in containers using Mountpoint for Amazon S3. Use AWS Backup to replicate objects to another Region

Overall explanation

Correct option:

Use Amazon Elastic File System (Amazon EFS) with the Standard storage class and configure AWS Backup to create cross-Region backups on a scheduled basis

Amazon EFS is a fully managed, scalable, NFS-compatible file system ideal for containerized workloads. It provides multi-AZ mount targets by default and supports integration with AWS Backup for automated cross-Region backup and restore. When used with AWS Backup, EFS can achieve an RPO of 8 hours or less through scheduled backups. The Standard storage class offers high durability and low latency, making it a strong fit for active workloads that span Availability Zones.

Incorrect options:

Deploy Amazon FSx for Lustre and configure a backup plan using AWS Backup for cross-Region replication of the file system metadata - Although Amazon FSx for Lustre supports integration with AWS Backup, it is optimized for high-performance compute (HPC) workloads, not general-purpose container applications. It lacks native cross-AZ mount targets and is not a good fit for persistent shared storage across AZs. In addition, data durability and recovery use cases are not ideal with Lustre unless linked to Amazon S3, which adds architectural complexity and delay.

Configure Amazon S3 with the S3 Standard storage class and mount it in containers using Mountpoint for Amazon S3. Use AWS Backup to replicate objects to another Region - While Amazon S3 is durable and supports cross-Region replication, it is an object storage service, not a file system. It lacks POSIX compliance even when used with Mountpoint for Amazon S3. So, this option is incorrect.

Use Amazon FSx for NetApp ONTAP with a Multi-AZ deployment and rely on its native high availability and AWS Backup integration to replicate the file system to another Region automatically - While Amazon FSx for NetApp ONTAP does support Multi-AZ deployments for high availability within a Region and integrates with AWS Backup for local backups, cross-Region replication is not managed automatically by AWS Backup for FSx for ONTAP. Instead, it requires manual configuration using NetApp SnapMirror, which introduces additional operational overhead and does not provide a managed RPO guarantee like EFS does when used with AWS Backup. Therefore, although the storage service is resilient and enterprise-grade, this setup does not fully meet the requirement for an RPO of 8 hours using AWS Backup for seamless cross-Region recovery.

References:

https://docs.aws.amazon.com/aws-backup/latest/devguide/cross-region-backup.html

https://docs.aws.amazon.com/aws-backup/latest/devguide/backup-feature-availability.html#features-by-resource

https://aws.amazon.com/blogs/aws/mountpoint-for-amazon-s3-generally-available-and-ready-for-production-workloads/

https://www.amazonaws.cn/en/fsx/lustre/faqs/#Availability_and_durability

https://docs.aws.amazon.com/fsx/latest/ONTAPGuide/what-is-fsx-ontap.html

Domain

Design Resilient Architectures

Question 13Skipped

A digital design company has migrated its project archiving platform to AWS. The application runs on Amazon EC2 Linux instances in an Auto Scaling group that spans multiple Availability Zones. Designers upload and retrieve high-resolution image files from a shared file system, which is currently configured to use Amazon EFS Standard-IA. Metadata for these files is stored and indexed in an Amazon RDS for PostgreSQL database. The company's cloud engineering team has been asked to optimize storage costs for the image archive without compromising reliability. They are open to refactoring the application to use managed AWS services when necessary.

Which solution offers the most cost-effective architecture?

Replace the EFS file system with Amazon FSx for NetApp ONTAP. Use volume tiering to move cold data to lower-cost capacity pool storage. Update the application to use the ONTAP mount path

Replace the EFS file system with Amazon FSx for Lustre. Mount the file system to EC2 instances and store project files there to reduce access latency and cost

Correct answer

Create an Amazon S3 bucket with Intelligent-Tiering enabled. Update the application to store and retrieve project files using the Amazon S3 API

Use AWS Backup to export all EFS files daily to an Amazon S3 bucket. Retain the EFS file system in Standard-IA class for occasional real-time access and route all archival queries to the S3 export

Overall explanation

Correct option:

Create an Amazon S3 bucket with Intelligent-Tiering enabled. Update the application to store and retrieve project files using the Amazon S3 API

Amazon S3 Intelligent-Tiering is an ideal cost-optimization choice for datasets with unpredictable access patterns. It automatically moves objects between frequent and infrequent access tiers, optimizing cost without compromising latency or availability. Compared to EFS Standard-IA, S3 Intelligent-Tiering offers lower cost per GB for archival-like workloads and doesn't require manual monitoring or tuning. Refactoring the application to use the S3 API introduces some development overhead but enables the most cost-effective, durable, and scalable storage.

Incorrect options:

Replace the EFS file system with Amazon FSx for Lustre. Mount the file system to EC2 instances and store project files there to reduce access latency and cost - Amazon FSx for Lustre is designed for high-performance compute (HPC) use cases such as machine learning, simulations, or big data analytics—not general-purpose file storage or archival. While it offers low-latency access to frequently accessed files, it does not offer automated lifecycle management, nor is it optimized for cost reduction in infrequently accessed workloads. It also lacks direct integration with database indexing use cases unless paired with Amazon S3.

Use AWS Backup to export all EFS files daily to an Amazon S3 bucket. Retain the EFS file system in Standard-IA class for occasional real-time access and route all archival queries to the S3 export - While AWS Backup can export EFS file system backups to S3, this approach introduces duplication and complexity rather than eliminating cost. The company would now pay for EFS storage, S3 storage, and backup operations, which could increase rather than reduce overall storage costs. Additionally, this setup would require the application to differentiate between S3 and EFS reads, complicating data retrieval paths.

Replace the EFS file system with Amazon FSx for NetApp ONTAP. Use volume tiering to move cold data to lower-cost capacity pool storage. Update the application to use the ONTAP mount path - Amazon FSx for NetApp ONTAP supports volume tiering to cost-efficient storage pools, which may sound appealing for cold data. However, FSx for ONTAP is more suited for enterprise NAS use cases, often tied to lift-and-shift migrations of existing NetApp systems. Its pricing model and operational complexity are less cost-effective for infrequent file access when compared to S3 Intelligent-Tiering. It also introduces the need to manage storage efficiency features (e.g., snapshots, quotas) that may be unnecessary in this context.

References:

https://docs.aws.amazon.com/AmazonS3/latest/userguide/intelligent-tiering-overview.html

https://docs.aws.amazon.com/fsx/latest/LustreGuide/what-is.html

https://docs.aws.amazon.com/aws-backup/latest/devguide/whatisbackup.html

https://docs.aws.amazon.com/fsx/latest/ONTAPGuide/what-is-fsx-ontap.html

Domain

Design Cost-Optimized Architectures

Question 14Skipped

A DevOps team is tasked with enabling secure and temporary SSH access to Amazon EC2 instances for developers during deployments. The team wants to avoid distributing long-term SSH key pairs and instead prefers ephemeral access that can be audited and revoked immediately after the session ends. The team wants direct access via the AWS Management Console.

What do you recommend?

Use an EC2 Instance Connect Endpoint to reach the instances even though they already have public IP addresses, because Instance Connect requires an endpoint for all SSH sessions

Use EC2 Instance Connect with Systems Manager Agent disabled, and connect via private IP using an internal proxy endpoint

Use EC2 Instance Connect to inject a static SSH key and connect via the instance's private IP address directly from the internet

Correct answer

Use EC2 Instance Connect to inject a temporary public key and establish SSH access using the instance’s public IP address

Overall explanation

Correct option:

Use EC2 Instance Connect to inject a temporary public key and establish SSH access using the instance’s public IP address

Amazon EC2 Instance Connect provides a secure and auditable way to establish temporary SSH sessions to EC2 instances by injecting a one-time-use public key into the instance at connection time. When used without Session Manager, EC2 Instance Connect requires the instance to have a public IP address, and connections are made over the internet via the public IP. This setup is ideal for temporary administrative access to public instances without the need to distribute or manage long-term SSH key pairs. The session is short-lived, and logs are available via CloudTrail for auditing purposes.

Incorrect options:

Use EC2 Instance Connect to inject a static SSH key and connect via the instance's private IP address directly from the internet - EC2 Instance Connect does not use static SSH keys and does not support connecting over private IPs from the internet unless paired with AWS Systems Manager Session

Manager. This option incorrectly assumes that private IPs are routable from the public internet, which is not the case in VPC networking. Additionally, using a static key contradicts the security design of EC2 Instance Connect, which relies on ephemeral keys to minimize risk.

Use EC2 Instance Connect with Systems Manager Agent disabled, and connect via private IP using an internal proxy endpoint - EC2 Instance Connect does not support connecting via private IP unless integrated with Session Manager, which requires that the SSM Agent is installed and running on the instance. This option incorrectly states that you can connect via private IP without SSM Agent or Session Manager, which is technically invalid. Instance Connect without Session Manager only supports public IP-based access.

Use an EC2 Instance Connect Endpoint to reach the instances even though they already have public IP addresses, because Instance Connect requires an endpoint for all SSH sessions - EC2 Instance Connect does not require an EC2 Instance Connect Endpoint when the target instance has a public IPv4 address; in that case you can connect directly over the internet and have a temporary SSH public key injected at connection time. EC2 Instance Connect Endpoints are specifically intended to let you connect to instances in private subnets without public IPs or direct internet connectivity, providing a managed entry point within your VPC. Adding an endpoint for publicly reachable instances adds unnecessary operational overhead and cost, and it misinterprets the feature's purpose.

References:

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-instance-connect-methods.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect-with-ec2-instance-connect-endpoint.html

https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html

Domain

Design High-Performing Architectures

Question 15Skipped

The engineering team at an e-commerce company uses an AWS Lambda function to write the order data into a single DB instance Amazon Aurora cluster. The team has noticed that many order- writes to its Aurora cluster are getting missed during peak load times. The diagnostics data has revealed that the database is experiencing high CPU and memory consumption during traffic spikes. The team also wants to enhance the availability of the Aurora DB.

Which of the following steps would you combine to address the given scenario? (Select two)

Correct selection

Handle all read operations for your application by connecting to the reader endpoint of the Amazon Aurora cluster so that Aurora can spread the load for read-only connections across the Aurora replica

Create a standby Aurora instance in another Availability Zone to improve the availability as the standby can serve as a failover target

Use Amazon EC2 instances behind an Application Load Balancer to write the order data into Amazon Aurora cluster

Increase the concurrency of the AWS Lambda function so that the order-writes do not get missed during traffic spikes

Correct selection

Create a replica Aurora instance in another Availability Zone to improve the availability as the replica can serve as a failover target

Overall explanation

Correct options:

Handle all read operations for your application by connecting to the reader endpoint of the Amazon Aurora cluster so that Aurora can spread the load for read-only connections across the Aurora replica

When you create a second, third, and so on DB instance in an Aurora-provisioned DB cluster, Aurora automatically sets up replication from the writer DB instance to all the other DB instances. These other DB instances are read-only and are known as Aurora Replicas.

Aurora Replicas have two main purposes. You can issue queries to them to scale the read operations for your application. You typically do so by connecting to the reader endpoint of the cluster. That way, Aurora can spread the load for read-only connections across as many Aurora Replicas as you have in the cluster. Aurora Replicas also help to increase availability. If the writer instance in a cluster becomes unavailable, Aurora automatically promotes one of the reader instances to take its place as the new writer.

Aurora Replicas

When you create a second, third, and so on DB instance in an Aurora provisioned DB cluster, Aurora automatically sets up replication from the writer DB instance to all the other DB instances. These other DB instances are read-only and are known as Aurora Replicas. We also refer to them as reader instances when discussing the ways that you can combine writer and reader DB instances within a cluster.

Aurora Replicas have two main purposes. You can issue queries to them to scale the read operations for your application. You typically do so by connecting to the reader endpoint of the cluster. That way, Aurora can spread the load for read-only connections across as many Aurora Replicas as you have in the cluster. Aurora Replicas also help to increase availability. If the writer instance in a cluster becomes unavailable, Aurora automatically promotes one of the reader instances to take its place as the new writer.

An Aurora DB cluster can contain up to 15 Aurora Replicas. The Aurora Replicas can be distributed across the Availability Zones that a DB cluster spans within an AWS Region.

The data in your DB cluster has its own high availability and reliability features, independent of the DB instances in the cluster. If you aren't familiar with Aurora storage features, see Overview of Aurora storage. The DB cluster volume is physically made up of multiple copies of the data for the DB cluster. The primary instance and the Aurora Replicas in the DB cluster all see the data in the cluster volume as a single logical volume.

As a result, all Aurora Replicas return the same data for query results with minimal replica lag. This lag is usually much less than 100 milliseconds after the primary instance has written an update. Replica lag varies depending on the rate of database change. That is, during periods where a large amount of write operations occur for the database, you might see an increase in replica lag.

Aurora Replicas work well for read scaling because they are fully dedicated to read operations on your cluster volume. Write operations are managed by the primary instance. Because the cluster volume is shared among all DB instances in your DB cluster, minimal additional work is required to replicate a copy of the data for each Aurora Replica.

To increase availability, you can use Aurora Replicas as failover targets. That is, if the primary instance fails, an Aurora Replica is promoted to the primary instance. There is a brief interruption during which read and write requests made to the primary instance fail with an exception. When this happens, some of the Aurora Replicas might be rebooted, depending on the DB engine version. For information about the rebooting behavior of different Aurora DB engine versions, see Rebooting an Amazon Aurora DB cluster or Amazon Aurora DB instance. Promoting an Aurora Replica this way is much faster than recreating the primary instance. If your Aurora DB cluster doesn't include any Aurora Replicas, then your DB cluster isn't available while your DB instance is recovering from the failure.

For high-availability scenarios, we recommend that you create one or more Aurora Replicas. These should be of the same DB instance class as the primary instance and in different Availability Zones for your Aurora DB cluster. For more information about Aurora Replicas as failover targets, see Fault tolerance for an Aurora DB cluster.

You can't create an encrypted Aurora Replica for an unencrypted Aurora DB cluster. You can't create an unencrypted Aurora Replica for an encrypted Aurora DB cluster.

via - https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Replication.html

Create a replica Aurora instance in another Availability Zone to improve the availability as the replica can serve as a failover target

If the primary instance in a DB cluster using single-master replication fails, Aurora automatically fails over to a new primary instance in one of two ways:

By promoting an existing Aurora Replica to the new primary instance By creating a new primary instance

Fault tolerance for an Aurora DB cluster

An Aurora DB cluster is fault tolerant by design. The cluster volume spans multiple Availability Zones (AZs) in a single AWS Region, and each Availability Zone contains a copy of the cluster volume data. This functionality means that your DB cluster can tolerate a failure of an Availability Zone without any loss of data and only a brief interruption of service.

If the primary instance in a DB cluster using single-master replication fails, Aurora automatically fails over to a new primary instance in one of two ways:

- By promoting an existing Aurora Replica to the new primary instance
- By creating a new primary instance

If the DB cluster has one or more Aurora Replicas, then an Aurora Replica is promoted to the primary instance during a failure event. A failure event results in a brief interruption, during which read and write operations fail with an exception. However, service is typically restored in less than 120 seconds, and often less than 60 seconds. To increase the availability of your DB cluster, we recommend that you create at least one or more Aurora Replicas in two or more different Availability Zones.

> **Tip**
>
> In Aurora MySQL 2.10 and higher, you can improve availability during a failover by having more than one reader DB instance in a cluster. In Aurora MySQL 2.10 and higher, Aurora restarts only the writer DB instance and the failover target during a failover. Other reader DB instances in the cluster remain available to continue processing queries through connections to the reader endpoint.

You can customize the order in which your Aurora Replicas are promoted to the primary instance after a failure by assigning each replica a priority. Priorities range from 0 for the first priority to 15 for the last priority. If the primary instance fails, Amazon RDS promotes the Aurora Replica with the better priority to the new primary instance. You can modify the priority of an Aurora Replica at any time. Modifying the priority doesn't trigger a failover.

More than one Aurora Replica can share the same priority, resulting in promotion tiers. If two or more Aurora Replicas share the same priority, then Amazon RDS promotes the replica that is largest in size. If two or more Aurora Replicas share the same priority and size, then Amazon RDS promotes an arbitrary replica in the same promotion tier.

If the DB cluster doesn't contain any Aurora Replicas, then the primary instance is recreated in the same AZ during a failure event. A failure event results in an interruption during which read and write operations fail with an exception. Service is restored when the new primary instance is created, which typically takes less than 10 minutes. Promoting an Aurora Replica to the primary instance is much faster than creating a new primary instance.

Suppose that the primary instance in your cluster is unavailable because of an outage that affects an entire AZ. In this case, the way to bring a new primary instance online depends on whether your cluster uses a Multi-AZ configuration:

- If your provisioned or Aurora Serverless v2 cluster contains any reader instances in other AZs, Aurora uses the failover mechanism to promote one of those reader instances to be the new primary instance.
- If your provisioned or Aurora Serverless v2 cluster only contains a single DB instance, or if the primary instance and all reader instances are in the same AZ, make sure to manually create one or more new DB instances in another AZ.
- If your cluster uses Aurora Serverless v1, Aurora automatically creates a new DB instance in another AZ. However, this process involves a host replacement and thus takes longer than a failover.

via
- https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Concepts.AuroraHighAvailability.html

Incorrect options:

Create a standby Aurora instance in another Availability Zone to improve the availability as the standby can serve as a failover target - There are no standby instances in Aurora. Aurora performs an automatic failover to a read replica when a problem is detected. So this option is incorrect.

Read replicas, Multi-AZ deployments, and multi-region deployments:

Multi-AZ deployments	Multi-Region deployments	Read replicas
Main purpose is high availability	Main purpose is disaster recovery and local performance	Main purpose is scalability
Non-Aurora: synchronous replication; Aurora: asynchronous replication	Asynchronous replication	Asynchronous replication
Non-Aurora: only the primary instance is active; Aurora: all instances are active	All regions are accessible and can be used for reads	All read replicas are accessible and can be used for read scaling
Non-Aurora: automated backups are taken from standby; Aurora: automated backups are taken from shared storage layer	Automated backups can be taken in each region	No backups configured by default
Always span at least two Availability Zones within a single region	Each region can have a Multi-AZ deployment	Can be within an Availability Zone, Cross-AZ, or Cross-Region
Non-Aurora: database engine version upgrades happen on primary; Aurora: all instances are updated together	Non-Aurora: database engine version upgrade is independent in each region; Aurora: all instances are updated together	Non-Aurora: database engine version upgrade is independent from source instance; Aurora: all instances are updated together
Automatic failover to standby (non-Aurora) or read replica (Aurora) when a problem is detected	Aurora allows promotion of a secondary region to be the primary	Can be manually promoted to a standalone database instance (non-Aurora) or to be the primary instance (Aurora)

via - https://aws.amazon.com/rds/features/read-replicas/

Increase the concurrency of the AWS Lambda function so that the order-writes do not get missed during traffic spikes - Increasing the concurrency of the AWS Lambda function would not resolve the issue since the bottleneck is at the database layer, as exhibited by the high CPU and memory consumption for the Aurora instance. This option has been added as a distractor.

Use Amazon EC2 instances behind an Application Load Balancer to write the order data into Amazon Aurora cluster - Using Amazon EC2 instances behind an Application Load Balancer would not resolve the issue since the bottleneck is at the database layer, as exhibited by the high CPU and memory consumption for the Aurora instance. This option has been added as a distractor.

References:

https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Replication.html

https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Concepts.AuroraHighAvailability.html

https://aws.amazon.com/rds/features/read-replicas/

Domain

Design Resilient Architectures

Question 16Skipped

Reporters at a news agency upload/download video files (about 500 megabytes each) to/from an Amazon S3 bucket as part of their daily work. As the agency has started offices in remote locations, it has resulted in poor latency for uploading and accessing data to/from the given Amazon S3 bucket. The agency wants to continue using a serverless storage solution such as Amazon S3 but wants to improve the performance.

As a solutions architect, which of the following solutions do you propose to address this issue? (Select two)

Move Amazon S3 data into Amazon Elastic File System (Amazon EFS) created in a US region, connect to Amazon EFS file system from Amazon EC2 instances in other AWS regions using an inter-region VPC peering connection

Spin up Amazon EC2 instances in each region where the agency has a remote office. Create a daily job to transfer Amazon S3 data into Amazon EBS volumes attached to the Amazon EC2 instances

Correct selection

Use Amazon CloudFront distribution with origin as the Amazon S3 bucket. This would speed up uploads as well as downloads for the video files

Correct selection

Enable Amazon S3 Transfer Acceleration (Amazon S3TA) for the Amazon S3 bucket. This would speed up uploads as well as downloads for the video files

Create new Amazon S3 buckets in every region where the agency has a remote office, so that each office can maintain its storage for the media assets

Overall explanation

Correct options:

Use Amazon CloudFront distribution with origin as the Amazon S3 bucket. This would speed up uploads as well as downloads for the video files

Amazon CloudFront is a fast content delivery network (CDN) service that securely delivers data, videos, applications, and APIs to customers globally with low latency, high transfer speeds, within a developer-friendly environment. When an object from Amazon S3 that is set up with Amazon CloudFront CDN is requested, the request would come through the Edge Location transfer paths only for the first request. Thereafter, it would be served from the nearest edge location to the users until it expires. So in this way, you can speed up uploads as well as downloads for the video files.

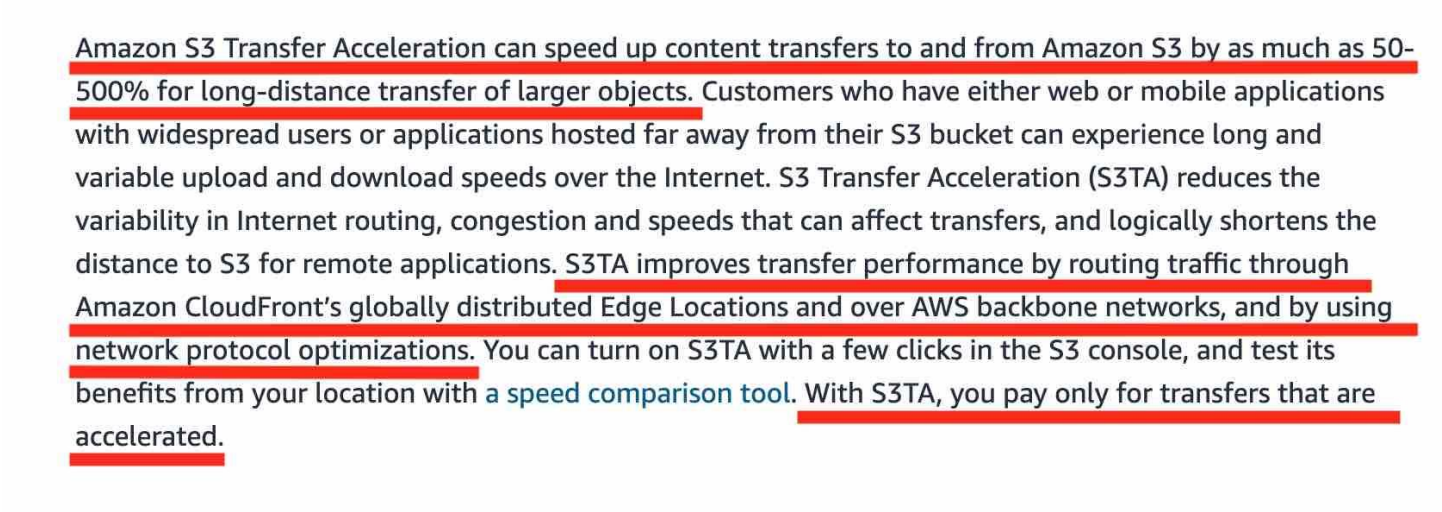
Following is a good reference blog for a deep-dive:

https://aws.amazon.com/blogs/aws/amazon-cloudfront-content-uploads-post-put-other-methods/

Enable Amazon S3 Transfer Acceleration (Amazon S3TA) for the Amazon S3 bucket. This would speed up uploads as well as downloads for the video files

Amazon S3 Transfer Acceleration (Amazon S3TA) can speed up content transfers to and from Amazon S3 by as much as 50-500% for long-distance transfer of larger objects. Transfer Acceleration takes advantage of Amazon CloudFront's globally distributed edge locations. As the data arrives at an edge location, data is routed to Amazon S3 over an optimized network path. So this option is also correct.

Amazon S3TA:



via - https://aws.amazon.com/s3/transfer-acceleration/

Incorrect options:

Create new Amazon S3 buckets in every region where the agency has a remote office, so that each office can maintain its storage for the media assets - Creating new Amazon S3 buckets in every region is not an option, since the agency maintains centralized storage. Hence this option is incorrect.

Move Amazon S3 data into Amazon Elastic File System (Amazon EFS) created in a US region, connect to Amazon EFS file system from Amazon EC2 instances in other AWS regions using an inter-region VPC peering connection

Spin up Amazon EC2 instances in each region where the agency has a remote office. Create a daily job to transfer Amazon S3 data into Amazon EBS volumes attached to the Amazon EC2 instances

Both these options using Amazon EC2 instances are not correct for the given use-case, as the agency wants a serverless storage solution.

References:

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/DownloadDistS3AndCustomOrigins.html

https://docs.aws.amazon.com/AmazonS3/latest/dev/transfer-acceleration.html

https://aws.amazon.com/s3/transfer-acceleration/

https://aws.amazon.com/blogs/aws/amazon-cloudfront-content-uploads-post-put-other-methods/

Domain

Design High-Performing Architectures

Question 17Skipped

An application hosted on Amazon EC2 contains sensitive personal information about all its customers and needs to be protected from all types of cyber-attacks. The company is considering using the AWS Web Application Firewall (AWS WAF) to handle this requirement.

Can you identify the correct solution leveraging the capabilities of AWS WAF?

AWS WAF can be directly configured only on an Application Load Balancer or an Amazon API Gateway. One of these two services can then be configured with Amazon EC2 to build the needed secure architecture

Configure an Application Load Balancer (ALB) to balance the workload for all the Amazon EC2 instances. Configure Amazon CloudFront to distribute from an Application Load Balancer since AWS WAF cannot be directly configured on ALB. This configuration not only provides necessary safety but is scalable too

AWS WAF can be directly configured on Amazon EC2 instances for ensuring the security of the underlying application data

Correct answer

Create Amazon CloudFront distribution for the application on Amazon EC2 instances. Deploy AWS WAF on Amazon CloudFront to provide the necessary safety measures

Overall explanation

Correct option:

Create Amazon CloudFront distribution for the application on Amazon EC2 instances. Deploy AWS WAF on Amazon CloudFront to provide the necessary safety measures

When you use AWS WAF with Amazon CloudFront, you can protect your applications running on any HTTP webserver, whether it's a webserver that's running in Amazon Elastic Compute Cloud (Amazon EC2) or a web server that you manage privately. You can also configure Amazon CloudFront to require HTTPS between CloudFront and your own webserver, as well as between viewers and Amazon CloudFront.

AWS WAF is tightly integrated with Amazon CloudFront and the Application Load Balancer (ALB), services that AWS customers commonly use to deliver content for their websites and applications. When you use AWS WAF on Amazon CloudFront, your rules run in all AWS Edge Locations, located around the world close to your end-users. This means security doesn’t come at the expense of performance. Blocked requests are stopped before they reach your web servers. When you use AWS WAF on Application Load Balancer, your rules run in the region and can be used to protect internet-facing as well as internal load balancers.

Incorrect options:

Configure an Application Load Balancer (ALB) to balance the workload for all the Amazon EC2 instances. Configure Amazon CloudFront to distribute from an Application Load Balancer since AWS WAF cannot be directly configured on ALB. This configuration not only provides necessary safety but is scalable too - This statement is wrong. You can configure AWS WAF on Application Load Balancers (ALB).

AWS WAF can be directly configured on Amazon EC2 instances for ensuring the security of the underlying application data - AWS WAF can be deployed on Amazon CloudFront, the Application Load Balancer (ALB), and Amazon API Gateway. It cannot be configured directly on an Amazon EC2 instance.

AWS WAF can be directly configured only on an Application Load Balancer or an Amazon API Gateway. One of these two services can then be configured with Amazon EC2 to build the needed secure architecture - This statement is only partially correct. AWS WAF can also be deployed on Amazon CloudFront service.

References:

https://aws.amazon.com/waf/faqs/

https://docs.aws.amazon.com/waf/latest/developerguide/cloudfront-features.html

Domain

Design Secure Architectures

Question 18Skipped

A data analytics team at a global media firm is building a new analytics platform to process large volumes of both historical and real-time data. This data is stored in Amazon S3. The team wants to implement a serverless solution that allows them to query the data directly using SQL. Additionally, the solution must ensure that all data is encrypted at rest and automatically replicated to another AWS Region to support business continuity.

Which solution will meet these requirements with the LEAST operational overhead?

Correct answer

Create an Amazon S3 bucket configured with server-side encryption using AWS KMS multi-Region keys (SSE-KMS). Enable cross-Region replication (CRR) on the source bucket. Use Amazon Athena to run SQL queries on the data

Enable Cross-Region Replication (CRR) on the existing Amazon S3 bucket. Apply server-side encryption using AWS KMS multi-Region keys (SSE-KMS). Use Amazon Athena to run SQL queries on the replicated data

Create an Amazon S3 bucket configured with server-side encryption using Amazon S3 managed keys (SSE-S3). Enable cross-Region replication (CRR) on the source bucket. Use Amazon Redshift Spectrum to query the S3 data using SQL

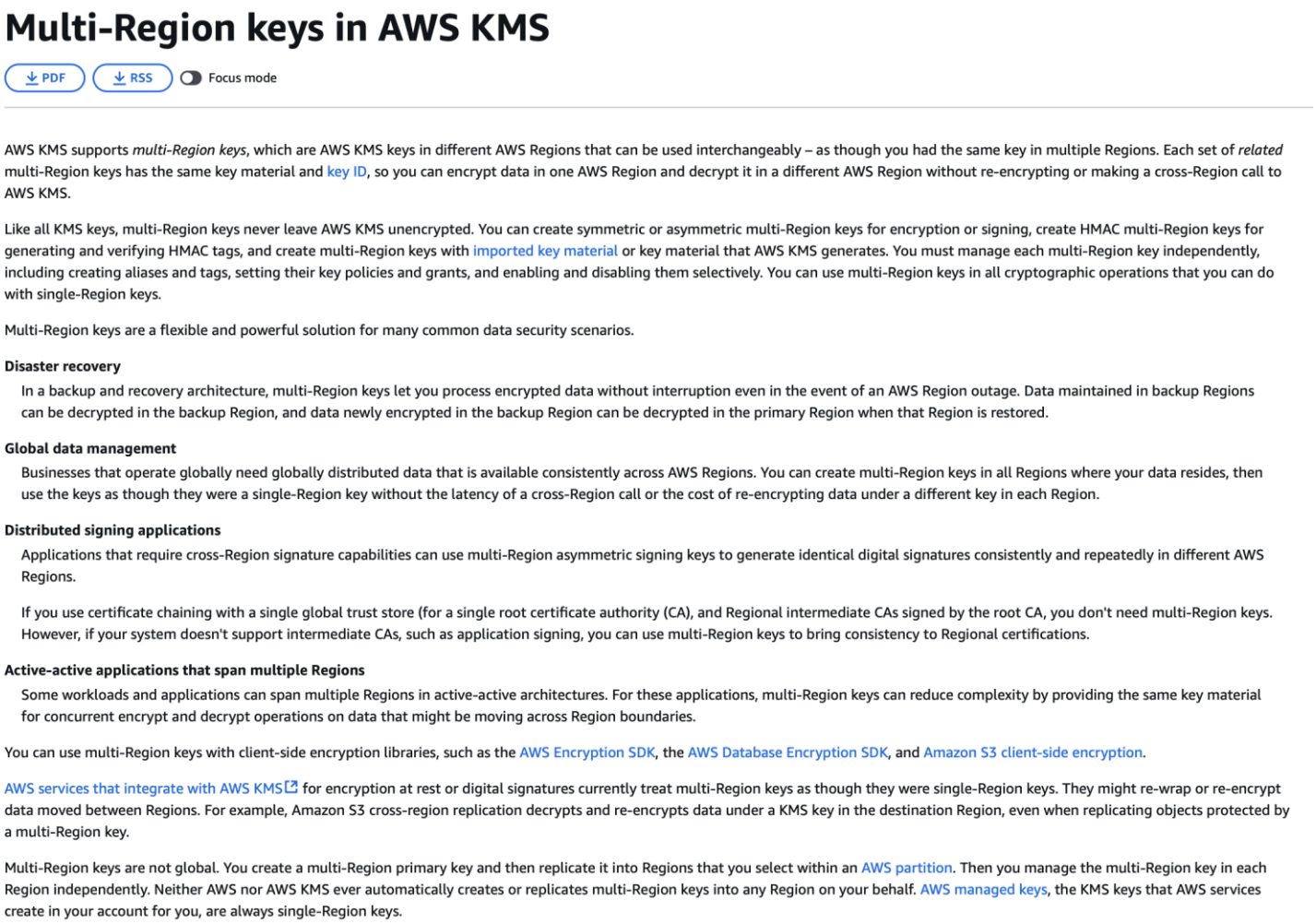
Enable Cross-Region Replication (CRR) on the existing Amazon S3 bucket. Apply server-side encryption using Amazon S3 managed keys (SSE-S3). Use Amazon Athena to run SQL queries on the replicated data

Overall explanation

Correct option:

Create an Amazon S3 bucket configured with server-side encryption using AWS KMS multi-Region keys (SSE-KMS). Enable cross-Region replication (CRR) on the source bucket. Use Amazon Athena to run SQL queries on the data

This option is the most efficient and serverless approach that meets all of the stated requirements. Amazon S3 supports server-side encryption using AWS KMS multi-Region keys, which allows secure data storage and supports automatic replication to a secondary Region via CRR. Amazon Athena is a serverless query service that can run SQL queries directly against S3 data without requiring a database engine or managing infrastructure. This solution supports encryption at rest, regional redundancy, and SQL-based querying with minimal overhead.



Multi-Region keys in AWS KMS

PDF RSS Focus mode

AWS KMS supports *multi-Region keys*, which are AWS KMS keys in different AWS Regions that can be used interchangeably – as though you had the same key in multiple Regions. Each set of *related* multi-Region keys has the same key material and key ID, so you can encrypt data in one AWS Region and decrypt it in a different AWS Region without re-encrypting or making a cross-Region call to AWS KMS.

Like all KMS keys, multi-Region keys never leave AWS KMS unencrypted. You can create symmetric or asymmetric multi-Region keys for encryption or signing, create HMAC multi-Region keys for generating and verifying HMAC tags, and create multi-Region keys with imported key material or key material that AWS KMS generates. You must manage each multi-Region key independently, including creating aliases and tags, setting their key policies and grants, and enabling and disabling them selectively. You can use multi-Region keys in all cryptographic operations that you can do with single-Region keys.

Multi-Region keys are a flexible and powerful solution for many common data security scenarios.

Disaster recovery

In a backup and recovery architecture, multi-Region keys let you process encrypted data without interruption even in the event of an AWS Region outage. Data maintained in backup Regions can be decrypted in the backup Region, and data newly encrypted in the backup Region can be decrypted in the primary Region when that Region is restored.

Global data management

Businesses that operate globally need globally distributed data that is available consistently across AWS Regions. You can create multi-Region keys in all Regions where your data resides, then use the keys as though they were a single-Region key without the latency of a cross-Region call or the cost of re-encrypting data under a different key in each Region.

Distributed signing applications

Applications that require cross-Region signature capabilities can use multi-Region asymmetric signing keys to generate identical digital signatures consistently and repeatedly in different AWS Regions.

If you use certificate chaining with a single global trust store (for a single root certificate authority (CA), and Regional intermediate CAs signed by the root CA, you don't need multi-Region keys. However, if your system doesn't support intermediate CAs, such as application signing, you can use multi-Region keys to bring consistency to Regional certifications.

Active-active applications that span multiple Regions

Some workloads and applications can span multiple Regions in active-active architectures. For these applications, multi-Region keys can reduce complexity by providing the same key material for concurrent encrypt and decrypt operations on data that might be moving across Region boundaries.

You can use multi-Region keys with client-side encryption libraries, such as the AWS Encryption SDK, the AWS Database Encryption SDK, and Amazon S3 client-side encryption.

AWS services that integrate with AWS KMS for encryption at rest or digital signatures currently treat multi-Region keys as though they were single-Region keys. They might re-wrap or re-encrypt data moved between Regions. For example, Amazon S3 cross-region replication decrypts and re-encrypts data under a KMS key in the destination Region, even when replicating objects protected by a multi-Region key.

Multi-Region keys are not global. You create a multi-Region primary key and then replicate it into Regions that you select within an AWS partition. Then you manage the multi-Region key in each Region independently. Neither AWS nor AWS KMS ever automatically creates or replicates multi-Region keys into any Region on your behalf. AWS managed keys, the KMS keys that AWS services create in your account for you, are always single-Region keys.

via - https://docs.aws.amazon.com/kms/latest/developerguide/multi-region-keys-overview.html

Incorrect options:

Enable Cross-Region Replication (CRR) on the existing Amazon S3 bucket. Apply server-side encryption using Amazon S3 managed keys (SSE-S3). Use Amazon Athena to run SQL queries on the replicated data

Enable Cross-Region Replication (CRR) on the existing Amazon S3 bucket. Apply server-side encryption using AWS KMS multi-Region keys (SSE-KMS). Use Amazon Athena to run SQL queries on the replicated data

Both of these options skip the step of creating a new bucket in the target AWS Region. You cannot enable CRR on a bucket without specifying the target bucket for replication in another Region. So, these options are incorrect.

Create an Amazon S3 bucket configured with server-side encryption using Amazon S3 managed keys (SSE-S3). Enable cross-Region replication (CRR) on the source bucket. Use Amazon Redshift Spectrum to query the S3 data using SQL - While Redshift Spectrum can query data stored in S3 using SQL, this solution introduces unnecessary operational complexity because Amazon Redshift requires cluster management. It is not entirely serverless. Although Redshift Spectrum itself is serverless, it still depends on a provisioned Redshift cluster backend, which contradicts the requirement to minimize operational overhead.

References:

https://docs.aws.amazon.com/kms/latest/developerguide/multi-region-keys-overview.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/replication.html

https://docs.aws.amazon.com/athena/latest/ug/what-is.html

Domain

Design Resilient Architectures

Question 19Skipped

An enterprise SaaS provider is currently operating a legacy web application hosted on a single Amazon EC2 instance within a public subnet. The same instance also hosts a MySQL database. DNS records for the application are configured through Amazon Route 53. As part of a modernization initiative, the company wants to rearchitect this application for high availability and scalability. In addition, the company wants to improve read performance on the database layer to handle increasing user traffic.

Which combination of solutions will meet these requirements? (Select two)

Use an Auto Scaling group to deploy EC2 instances across multiple Availability Zones in two AWS Regions. Register the instances in a target group behind an Application Load Balancer to distribute web traffic evenly

Use Amazon CloudFront with Lambda@Edge to serve dynamic content from EC2 instances located in different Regions

Correct selection

Migrate the existing MySQL database to an Amazon Aurora MySQL cluster. Deploy the primary DB instance and one or more read replicas in different Availability Zones

Deploy an additional EC2 instance in a different AWS Region, and configure Amazon Route 53 with a failover routing policy to direct traffic to the secondary instance during primary Region outages

Correct selection

Use an Auto Scaling group to deploy EC2 instances across multiple Availability Zones within a single Region. Register the instances in a target group behind an Application Load Balancer to distribute web traffic evenly

Overall explanation

Correct options:

Use an Auto Scaling group to deploy EC2 instances across multiple Availability Zones within a single Region. Register the instances in a target group behind an Application Load Balancer to distribute web traffic evenly

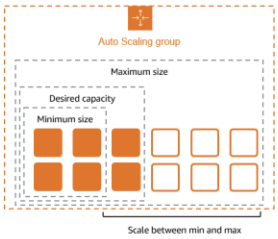
This option enhances scalability and high availability of the web application. Deploying EC2 instances across multiple Availability Zones ensures that the application remains available even if one Availability Zone experiences an outage. The Application Load Balancer (ALB) evenly distributes incoming web traffic across all healthy instances in the target group, improving performance and fault tolerance. The Auto Scaling group automatically adjusts the number of EC2 instances in response to changing traffic, reducing operational burden and costs while ensuring optimal performance.

What is Amazon EC2 Auto Scaling?

PDF RSS Focus mode

Amazon EC2 Auto Scaling helps you ensure that you have the correct number of Amazon EC2 instances available to handle the load for your application. You create collections of EC2 instances, called *Auto Scaling groups*. You can specify the minimum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes below this size. You can specify the maximum number of instances in each Auto Scaling group, and Amazon EC2 Auto Scaling ensures that your group never goes above this size. If you specify the desired capacity, either when you create the group or at any time thereafter, Amazon EC2 Auto Scaling ensures that your group has this many instances. If you specify scaling policies, then Amazon EC2 Auto Scaling can launch or terminate instances as demand on your application increases or decreases.

For example, the following Auto Scaling group has a minimum size of four instances, a desired capacity of six instances, and a maximum size of twelve instances. The scaling policies that you define adjust the number of instances, within your minimum and maximum number of instances, based on the criteria that you specify.



Features of Amazon EC2 Auto Scaling

With Amazon EC2 Auto Scaling, your EC2 instances are organized into Auto Scaling groups so that they can be treated as a logical unit for the purposes of scaling and management. Auto Scaling groups use launch templates (or launch configurations) as configuration templates for their EC2 instances.

The following are key features of Amazon EC2 Auto Scaling:

Monitoring the health of running instances
Amazon EC2 Auto Scaling automatically monitors the health and availability of your instances using EC2 health checks and replaces terminated or impaired instances to maintain your desired capacity.

Custom health checks
In addition to the built-in health checks, you can define custom health checks that are specific to your application to verify that it's responding as expected. If an instance fails your custom health check, it's automatically replaced to maintain your desired capacity.

Balancing capacity across Availability Zones
You can specify multiple Availability Zones for your Auto Scaling group, and Amazon EC2 Auto Scaling balances your instances evenly across the Availability Zones as the group scales. This provides high availability and resiliency by protecting your applications from failures in a single location.

Multiple instance types and purchase options
Within a single Auto Scaling group, you can launch multiple instance types and purchase options (Spot and On-Demand Instances), allowing you to optimize costs through Spot Instance usage. You can also take advantage of Reserved Instance and Savings Plan discounts by using them in conjunction with On-Demand Instances in the group.

Automated replacement of Spot Instances
If your group includes Spot Instances, Amazon EC2 Auto Scaling can automatically request replacement Spot capacity if your Spot Instances are interrupted. Through Capacity Rebalancing, Amazon EC2 Auto Scaling can also monitor and proactively replace your Spot Instances that are at an elevated risk of interruption.

Load balancing
You can use Elastic Load Balancing load balancing and health checks to ensure an even distribution of application traffic to your healthy instances. Whenever instances are launched or terminated, Amazon EC2 Auto Scaling automatically registers and deregisters the instances from the load balancer.

Scalability
Amazon EC2 Auto Scaling also provides several ways for you to scale your Auto Scaling groups. Using auto scaling allows you to maintain application availability and reduce costs by adding capacity to handle peak loads and removing capacity when demand is lower. You can also manually adjust the size of your Auto Scaling group as needed.

Instance refresh
The instance refresh feature provides a mechanism to update instances in a rolling fashion when you update your AMI or launch template. You can also use a phased approach, known as a canary deployment, to test a new AMI or launch template on a small set of instances before rolling it out to the whole group.

Lifecycle hooks
Lifecycle hooks are useful for defining custom actions that are invoked as new instances launch or before instances are terminated. This feature is particularly useful for building event-driven architectures, but it also helps you manage instances through their lifecycle.

Support for stateful workloads
Lifecycle hooks also offer a mechanism for persisting state on shut down. To ensure continuity for stateful applications, you can also use scale-in protection or custom termination policies to prevent instances with long-running processes from terminating early.

via - https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html

Migrate the existing MySQL database to an Amazon Aurora MySQL cluster. Deploy the primary DB instance and one or more read replicas in different Availability Zones

This option addresses the need to reduce read latency and improve database availability. Amazon Aurora is a fully managed, MySQL-compatible relational database engine designed for high performance and availability. By deploying a writer (primary) instance and at least one reader instance in different Availability Zones, the application can offload read-heavy workloads to the reader instance, improving throughput and reducing response times. Aurora's automatic failover capabilities also help ensure continued operation in the event of a failure of the primary instance. Aurora further supports features like replication, automatic backups, and storage autoscaling, reducing operational overhead.

Incorrect options:

Deploy an additional EC2 instance in a different AWS Region, and configure Amazon Route 53 with a failover routing policy to direct traffic to the secondary instance during primary Region outages - This setup lacks auto scaling capabilities. If traffic spikes in either Region, the architecture would require

manual intervention or additional setup to scale. No Application Load Balancer (ALB) or Auto Scaling group is used in this option — which are best practices for horizontally scaling stateless workloads.

Use an Auto Scaling group to deploy EC2 instances across multiple Availability Zones in two AWS Regions. Register the instances in a target group behind an Application Load Balancer to distribute web traffic evenly - An Application Load Balancer operates only within a single AWS Region and can route traffic to targets in multiple Availability Zones within that Region, but not across Regions. So, this option is incorrect.

Use Amazon CloudFront with Lambda@Edge to serve dynamic content from EC2 instances located in different Regions - CloudFront is ideal for caching static content, but it's not designed for dynamic database-driven applications hosted on EC2. Using Lambda@Edge for a database-backed application introduces architectural mismatches and increases complexity.

References:

https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html

https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/CHAP_AuroraOverview.html

https://docs.aws.amazon.com/AmazonRDS/latest/AuroraUserGuide/Aurora.Replication.html

Domain

Design Resilient Architectures

Question 20Skipped

A media company wants to get out of the business of owning and maintaining its own IT infrastructure. As part of this digital transformation, the media company wants to archive about 5 petabytes of data in its on-premises data center to durable long term storage.

As a solutions architect, what is your recommendation to migrate this data in the MOST cost-optimal way?

Transfer the on-premises data into multiple AWS Snowball Edge Storage Optimized devices. Copy the AWS Snowball Edge data into Amazon S3 Glacier

Setup AWS direct connect between the on-premises data center and AWS Cloud. Use this connection to transfer the data into Amazon S3 Glacier

Correct answer

Transfer the on-premises data into multiple AWS Snowball Edge Storage Optimized devices. Copy the AWS Snowball Edge data into Amazon S3 and create a lifecycle policy to transition the data into Amazon S3 Glacier

Setup AWS Site-to-Site VPN connection between the on-premises data center and AWS Cloud. Use this connection to transfer the data into Amazon S3 Glacier

Overall explanation

Correct option:

Transfer the on-premises data into multiple AWS Snowball Edge Storage Optimized devices. Copy the AWS Snowball Edge data into Amazon S3 and create a lifecycle policy to transition the data into Amazon S3 Glacier

AWS Snowball Edge Storage Optimized is the optimal choice if you need to securely and quickly transfer dozens of terabytes to petabytes of data to AWS. It provides up to 80 TB of usable HDD storage, 40 vCPUs, 1 TB of SATA SSD storage, and up to 40 Gb network connectivity to address large scale data transfer and pre-processing use cases. The data stored on AWS Snowball Edge device can be copied into Amazon S3 bucket and later transitioned into Amazon S3 Glacier via a lifecycle policy. You can't directly copy data from AWS Snowball Edge devices into Amazon S3 Glacier.

Incorrect options:

Transfer the on-premises data into multiple AWS Snowball Edge Storage Optimized devices. Copy the AWS Snowball Edge data into Amazon S3 Glacier - As mentioned earlier, you can't directly copy data from AWS Snowball Edge devices into Amazon S3 Glacier. Hence, this option is incorrect.

Setup AWS direct connect between the on-premises data center and AWS Cloud. Use this connection to transfer the data into Amazon S3 Glacier - AWS Direct Connect lets you establish a dedicated network connection between your network and one of the AWS Direct Connect locations. Using industry-standard 802.1q VLANs, this dedicated connection can be partitioned into multiple virtual interfaces. Direct Connect involves significant monetary investment and takes more than a month to set up, therefore it's not the correct fit for this use-case where just a one-time data transfer has to be done.

Setup AWS Site-to-Site VPN connection between the on-premises data center and AWS Cloud. Use this connection to transfer the data into Amazon S3 Glacier - AWS Site-to-Site VPN enables you to securely connect your on-premises network or branch office site to your Amazon Virtual Private Cloud (Amazon VPC). VPN Connections are a good solution if you have an immediate need, and have low to modest bandwidth requirements. Because of the high data volume for the given use-case, Site-to-Site VPN is not the correct choice.

Reference:

https://aws.amazon.com/snowball/

Domain

Design Cost-Optimized Architectures

Question 21Skipped

The data engineering team at an e-commerce company has set up a workflow to ingest the clickstream data into the raw zone of the Amazon S3 data lake. The team wants to run some SQL based data sanity checks on the raw zone of the data lake.

What AWS services would you recommend for this use-case such that the solution is cost-effective and easy to maintain?

Load the incremental raw zone data into an Amazon EMR based Spark Cluster on an hourly basis and use SparkSQL to run the SQL based sanity checks

Correct answer

Use Amazon Athena to run SQL based analytics against Amazon S3 data

Load the incremental raw zone data into Amazon Redshift on an hourly basis and run the SQL based sanity checks

Load the incremental raw zone data into Amazon RDS on an hourly basis and run the SQL based sanity checks

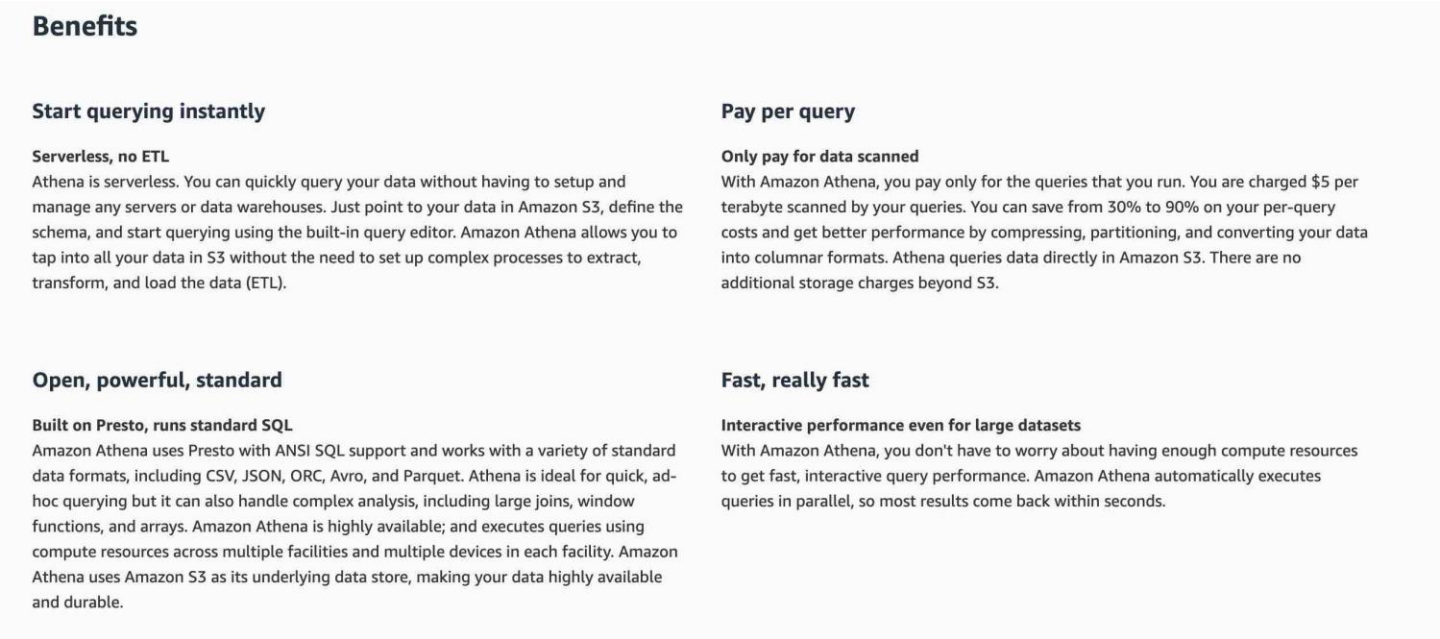
Overall explanation

Correct option:

Use Amazon Athena to run SQL based analytics against Amazon S3 data

Amazon Athena is an interactive query service that makes it easy to analyze data directly in Amazon S3 using standard SQL. Amazon Athena is serverless, so there is no infrastructure to set up or manage, and customers pay only for the queries they run. You can use Athena to process logs, perform ad-hoc analysis, and run interactive queries.

Amazon Athena Benefits:



via - https://aws.amazon.com/athena/

Incorrect options:

Load the incremental raw zone data into Amazon Redshift on an hourly basis and run the SQL based sanity checks - Amazon Redshift is a fully-managed petabyte-scale cloud-based data warehouse product designed for large scale data set storage and analysis. As the development team would have to

maintain and monitor the Amazon Redshift cluster size and would require significant development time to set up the processes to consume the data periodically, so this option is ruled out.

Load the incremental raw zone data into an Amazon EMR based Spark Cluster on an hourly basis and use SparkSQL to run the SQL based sanity checks - Amazon EMR is the industry-leading cloud big data platform for processing vast amounts of data using open source tools such as Apache Spark, Apache Hive, Apache HBase, Apache Flink, Apache Hudi, and Presto. Amazon EMR uses Hadoop, an open-source framework, to distribute your data and processing across a resizable cluster of Amazon EC2 instances. Using an Amazon EMR cluster would imply managing the underlying infrastructure so it's ruled out because the correct solution for the given use-case should require the least amount of development effort and ongoing maintenance.

Load the incremental raw zone data into Amazon RDS on an hourly basis and run the SQL based sanity checks - Loading the incremental data into Amazon RDS implies data migration jobs will have to be written via a AWS Lambda function or an Amazon EC2 based process. This goes against the requirement that the solution should involve the least amount of development effort and ongoing maintenance. Hence this option is not correct.

Reference:

https://aws.amazon.com/athena/

Domain

Design Cost-Optimized Architectures

Question 22Skipped

A fintech company recently conducted a security audit and discovered that some IAM roles and Amazon S3 buckets might be unintentionally shared with external accounts or publicly accessible. The security team wants to identify these overly permissive resources and ensure that only intended principals (within their AWS Organization or specific AWS accounts) have access. They need a solution that can analyze IAM policies and resource policies to detect unintended access paths to AWS resources such as S3 buckets, IAM roles, KMS keys, and SNS topics.

Which solution should the team use to meet this requirement?

Use IAM Access Advisor to get detailed access analysis of S3 bucket policies and determine which principals outside the organization have access

Use Amazon Inspector to detect over-permissive IAM policies and access paths across the environment

Use AWS Config to track configuration changes and infer resource-sharing behavior by analyzing compliance rules

Correct answer

Use AWS Identity and Access Management (IAM) Access Analyzer to evaluate resource-based and identity-based policies and identify resources shared outside the account or organization

Overall explanation

Correct option:

Use AWS Identity and Access Management (IAM) Access Analyzer to evaluate resource-based and identity-based policies and identify resources shared outside the account or organization

AWS IAM Access Analyzer helps identify resources shared with external entities by analyzing resource-based policies (e.g., S3 buckets, KMS keys, SNS topics, IAM roles) and detecting unintended access. It uses automated reasoning to evaluate the effects of policies and determines if a resource is accessible from outside the account or organization. Access Analyzer also supports policy validation, helping administrators craft secure and compliant access policies. It's purpose-built for detecting and auditing external access paths to resources.

Incorrect options:

Use IAM Access Advisor to get detailed access analysis of S3 bucket policies and determine which principals outside the organization have access - IAM Access Advisor shows the last accessed information for AWS services used by IAM users and roles, but it does not evaluate resource-based policies or determine if resources are shared externally. It is designed to help with least privilege enforcement by identifying unused permissions, not external access analysis. It cannot detect whether S3 buckets or IAM roles are publicly accessible or shared with specific accounts.

Use AWS Config to track configuration changes and infer resource-sharing behavior by analyzing compliance rules - AWS Config is used to record configuration changes and evaluate resource compliance against custom or managed rules. While it can track policy changes or detect public S3 access if configured properly, it does not natively reason over policy semantics to determine whether a policy grants external access. Config works best when used in conjunction with Access Analyzer for a comprehensive compliance and access strategy but does not replace Access Analyzer's policy evaluation capabilities.

Use Amazon Inspector to detect over-permissive IAM policies and access paths across the environment - Amazon Inspector is primarily a vulnerability assessment tool focused on EC2 instances, container images, and Lambda functions. It checks for software vulnerabilities and security best practices, but it does not evaluate IAM policies or analyze whether resources like S3 buckets or roles are accessible to external accounts. It is not designed for access path analysis or policy-level visibility.

References:

https://docs.aws.amazon.com/IAM/latest/UserGuide/what-is-access-analyzer.html

https://aws.amazon.com/blogs/security/set-permission-guardrails-using-iam-access-advisor-analyze-service-last-accessed-information-aws-organization/

Domain

Design Secure Architectures

Question 23Skipped

A company needs a massive PostgreSQL database and the engineering team would like to retain control over managing the patches, version upgrades for the database, and consistent performance with high IOPS. The team wants to install the database on an Amazon EC2 instance with the optimal storage type on the attached Amazon EBS volume.

As a solutions architect, which of the following configurations would you suggest to the engineering team?

Amazon EC2 with Amazon EBS volume of cold HDD (sc1) type

Amazon EC2 with Amazon EBS volume of Throughput Optimized HDD (st1) type

Correct answer

Amazon EC2 with Amazon EBS volume of Provisioned IOPS SSD (io1) type

Amazon EC2 with Amazon EBS volume of General Purpose SSD (gp2) type

Overall explanation

Correct option:

Amazon EC2 with Amazon EBS volume of Provisioned IOPS SSD (io1) type

Amazon EBS provides the following volume types, which differ in performance characteristics and price so that you can tailor your storage performance and cost to the needs of your applications.

The volumes types fall into two categories:

SSD-backed volumes optimized for transactional workloads involving frequent read/write operations with small I/O size, where the dominant performance attribute is IOPS

HDD-backed volumes optimized for large streaming workloads where throughput (measured in MiB/s) is a better performance measure than IOPS

Provision IOPS type supports critical business applications that require sustained IOPS performance, or more than 16,000 IOPS or 250 MiB/s of throughput per volume. Examples are large database workloads, such as: MongoDB Cassandra Microsoft SQL Server MySQL PostgreSQL Oracle

Therefore, Amazon EC2 with Amazon EBS volume of Provisioned IOPS SSD (io1) type is the right fit for the given use-case.

Please see this detailed overview of the volume types for Amazon EBS volumes.

Volume characteristics

The following table describes the use cases and performance characteristics for each volume type. The default volume type is General Purpose SSD (gp2).

	Solid-state drives (SSD)		Hard disk drives (HDD)	
Volume type	General Purpose SSD (gp2)	Provisioned IOPS SSD (io1)	Throughput Optimized HDD (st1)	Cold HDD (sc1)
Description	General purpose SSD volume that balances price and performance for a wide variety of workloads	Highest-performance SSD volume for mission-critical low-latency or high-throughput workloads	Low-cost HDD volume designed for frequently accessed, throughput-intensive workloads	Lowest cost HDD volume designed for less frequently accessed workloads

Use cases	• Recommended for most workloads
	• System boot volumes
	• Virtual desktops
	• Low-latency interactive apps
	• Development and test environments
		• Critical business applications that require sustained IOPS performance, or more than 16,000 IOPS or 250 MiB/s of throughput per volume
		• Large database workloads, such as:
		  ◦ MongoDB
		  ◦ Cassandra
		  ◦ Microsoft SQL Server
		  ◦ MySQL
		  ◦ PostgreSQL
		  ◦ Oracle
			• Streaming workloads requiring consistent, fast throughput at a low price
			• Big data
			• Data warehouses
			• Log processing
			• Cannot be a boot volume
				• Throughput-oriented storage for large volumes of data that is infrequently accessed
				• Scenarios where the lowest storage cost is important
				• Cannot be a boot volume

via - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html

Incorrect options:

Amazon EC2 with Amazon EBS volume of General Purpose SSD (gp2) type

Amazon EC2 with Amazon EBS volume of Throughput Optimized HDD (st1) type

Amazon EC2 with Amazon EBS volume of cold HDD (sc1) type

Per the explanation in the detailed overview provided above, these three options are incorrect.

Reference:

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html

Domain

Design High-Performing Architectures

Question 24Skipped

A company hires experienced specialists to analyze the customer service calls attended by its call center representatives. Now, the company wants to move to AWS Cloud and is looking at an automated solution to analyze customer service calls for sentiment analysis via ad-hoc SQL queries.

As a Solutions Architect, which of the following solutions would you recommend?

Use Amazon Kinesis Data Streams to read the audio files and machine learning (ML) algorithms to convert the audio files into text and run customer sentiment analysis

Use Amazon Kinesis Data Streams to read the audio files and Amazon Alexa to convert them into text. Amazon Kinesis Data Analytics can be used to analyze these files and Amazon Quicksight can be used to visualize and display the output

Correct answer

Use Amazon Transcribe to convert audio files to text and Amazon Athena to perform SQL based analysis to understand the underlying customer sentiments

Use Amazon Transcribe to convert audio files to text and Amazon Quicksight to perform SQL based analysis on these text files to understand the underlying patterns. Visualize and display them onto user Dashboards for reporting purposes

Overall explanation

Correct option:

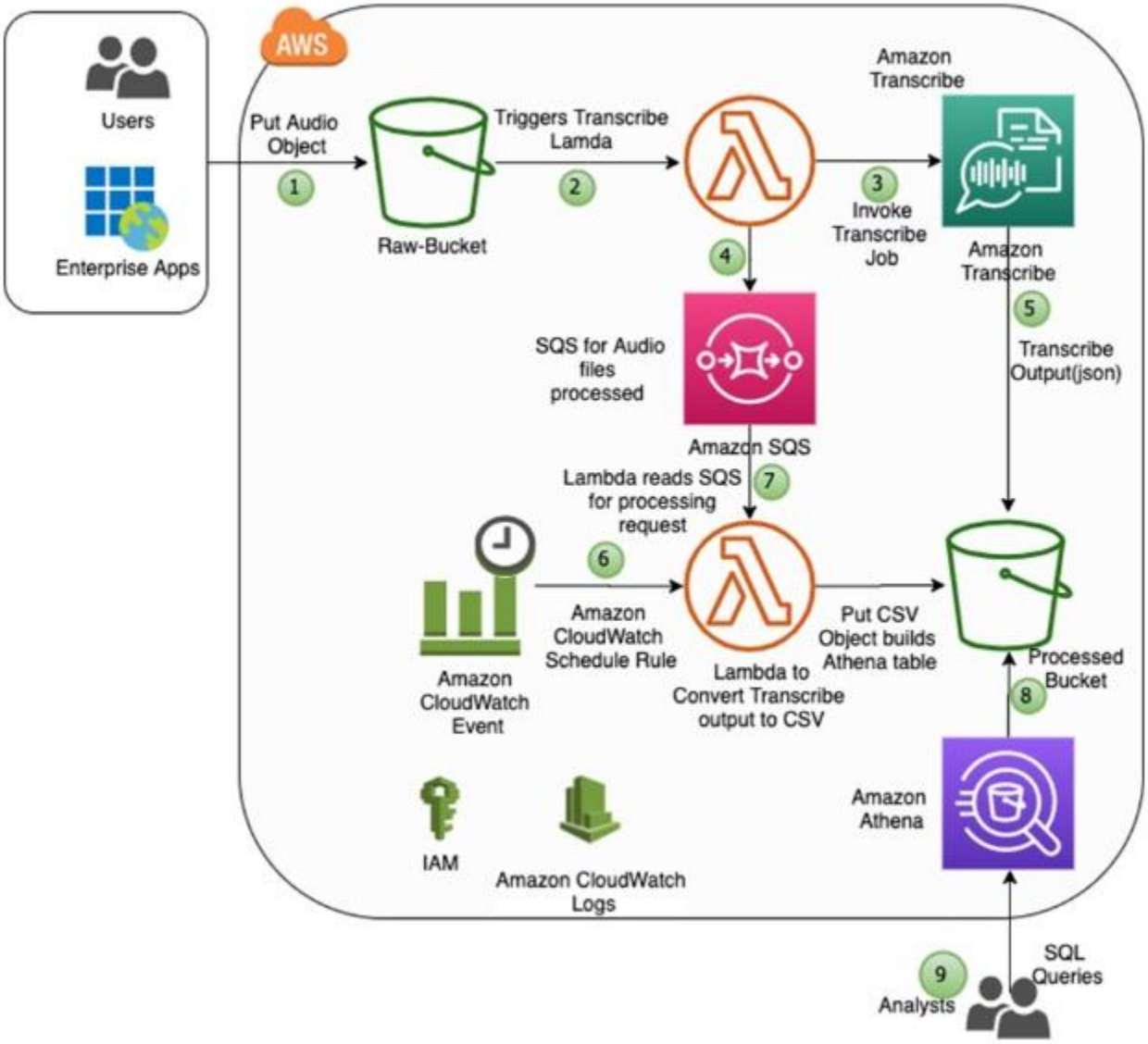
Use Amazon Transcribe to convert audio files to text and Amazon Athena to perform SQL based analysis to understand the underlying customer sentiments

Amazon Transcribe is an automatic speech recognition (ASR) service that makes it easy to convert audio to text. One key feature of the service is called speaker identification, which you can use to label each individual speaker when transcribing multi-speaker audio files. You can specify Amazon Transcribe to identify 2–10 speakers in the audio clip.

Amazon Athena is an interactive query service that makes it easy to analyze data in Amazon S3 using standard SQL. Athena is serverless, so there is no infrastructure to manage, and you pay only for the queries that you run. To leverage Athena, you can simply point to your data in Amazon S3, define the schema, and start querying using standard SQL. Most results are delivered within seconds.

Analyzing multi-speaker audio files using Amazon Transcribe and Amazon Athena:

Diagram: Analyze Multi-Speaker Audio Files Using Amazon Transcribe and Amazon Athena



via - https://aws.amazon.com/blogs/machine-learning/automating-the-analysis-of-multi-speaker-audio-files-using-amazon-transcribe-and-amazon-athena

Incorrect options:

Use Amazon Kinesis Data Streams to read the audio files and machine learning (ML) algorithms to convert the audio files into text and run customer sentiment analysis - Amazon Kinesis can be used to stream real-time data for further analysis and storage. Kinesis Data Streams cannot read audio files. You will still need to use AWS Transcribe for ASR services.

Use Amazon Kinesis Data Streams to read the audio files and Amazon Alexa to convert them into text. Amazon Kinesis Data Analytics can be used to analyze these files and Amazon Quicksight can be used to visualize and display the output - Amazon Kinesis Data Streams cannot read audio files. Amazon Alexa cannot be used as an Automatic Speech Recognition (ASR) service, though Alexa internally uses ASR for its working.

Use Amazon Transcribe to convert audio files to text and Amazon Quicksight to perform SQL based analysis on these text files to understand the underlying patterns. Visualize and display them onto user Dashboards for reporting purposes - Amazon Quicksight is used for the visual representation of data through dashboards. However, it is not an SQL query based analysis tool like Amazon Athena. So, this option is incorrect.

References:

https://aws.amazon.com/blogs/machine-learning/automating-the-analysis-of-multi-speaker-audio-files-using-amazon-transcribe-and-amazon-athena

https://aws.amazon.com/athena

Domain

Design High-Performing Architectures

Question 25Skipped

While troubleshooting, a cloud architect realized that the Amazon EC2 instance is unable to connect to the internet using the Internet Gateway.

Which conditions should be met for internet connectivity to be established? (Select two)

Correct selection

The network access control list (network ACL) associated with the subnet must have rules to allow inbound and outbound traffic

Correct selection

The route table in the instance's subnet should have a route to an Internet Gateway

The subnet has been configured to be public and has no access to the internet

The instance's subnet is not associated with any route table

The instance's subnet is associated with multiple route tables with conflicting configurations

Overall explanation

Correct options:

The network access control list (network ACL) associated with the subnet must have rules to allow inbound and outbound traffic

The network access control list (network ACL) that is associated with the subnet must have rules to allow inbound and outbound traffic on port 80 (for HTTP traffic) and port 443 (for HTTPs traffic). This is a necessary condition for Internet Gateway connectivity.

The route table in the instance's subnet should have a route to an Internet Gateway

A route table contains a set of rules, called routes, that are used to determine where network traffic from your subnet or gateway is directed. The route table in the instance's subnet should have a route defined to the Internet Gateway.

Incorrect options:

The instance's subnet is not associated with any route table - This is an incorrect statement. A subnet is implicitly associated with the main route table if it is not explicitly associated with a particular route table. So, a subnet is always associated with some route table.

The instance's subnet is associated with multiple route tables with conflicting configurations - This is an incorrect statement. A subnet can only be associated with one route table at a time.

The subnet has been configured to be public and has no access to the internet - This is an incorrect statement. Public subnets have access to the internet via Internet Gateway.

Reference:

https://docs.aws.amazon.com/vpc/latest/userguide/VPC_Route_Tables.html

Domain

Design Secure Architectures

Question 26Skipped

A company's cloud architect has set up a solution that uses Amazon Route 53 to configure the DNS records for the primary website with the domain pointing to the Application Load Balancer (ALB). The company wants a solution where users will be directed to a static error page, configured as a backup, in case of unavailability of the primary website.

Which configuration will meet the company's requirements, while keeping the changes to a bare minimum?

Use Amazon Route 53 Latency-based routing. Create a latency record to point to the Amazon S3 bucket that holds the error page to be displayed

Set up Amazon Route 53 active-active type of failover routing policy. If Amazon Route 53 health check determines the Application Load Balancer endpoint as unhealthy, the traffic will be diverted to a static error page, hosted on Amazon S3 bucket

Use Amazon Route 53 Weighted routing to give minimum weight to Amazon S3 bucket that holds the error page to be displayed. In case of primary failure, the requests get routed to the error page

Correct answer

Set up Amazon Route 53 active-passive type of failover routing policy. If Amazon Route 53 health check determines the Application Load Balancer endpoint as unhealthy, the traffic will be diverted to a static error page, hosted on Amazon S3 bucket

Overall explanation

Correct option:

Set up Amazon Route 53 active-passive type of failover routing policy. If Amazon Route 53 health check determines the Application Load Balancer endpoint as unhealthy, the traffic will be diverted to a static error page, hosted on Amazon S3 bucket

Use an active-passive failover configuration when you want a primary resource or group of resources to be available the majority of the time and you want a secondary resource or group of resources to be on standby in case all the primary resources become unavailable. When responding to queries, Amazon Route 53 includes only healthy primary resources. If all the primary resources are unhealthy, Route 53 begins to include only the healthy secondary resources in response to DNS queries.

Incorrect options:

Set up Amazon Route 53 active-active type of failover routing policy. If Amazon Route 53 health check determines the Application Load Balancer endpoint as unhealthy, the traffic will be diverted to a static error page, hosted on Amazon S3 bucket - This option has been added as a distractor as there is no such thing as an active-active failover routing policy in Amazon Route 53. You can configure active-active failover using any routing policy (or combination of routing policies) other than failover routing policy and you configure active-passive failover only using the failover routing policy. In active-active failover configuration, all the records that have the same name, the same type (such as A or AAAA), and the same routing policy (such as weighted or latency) are active unless Amazon Route 53 considers them unhealthy. Amazon Route 53 can respond to a DNS query using any healthy record.

Use Amazon Route 53 Latency-based routing. Create a latency record to point to the Amazon S3 bucket that holds the error page to be displayed - If your application is hosted in multiple AWS Regions, you can improve performance for your users by serving their requests from the AWS Region that provides the lowest latency - this is Latency-based routing and is not helpful for the current use case.

Use Amazon Route 53 Weighted routing to give minimum weight to Amazon S3 bucket that holds the error page to be displayed. In case of primary failure, the requests get routed to the error page - Weighted routing lets you associate multiple resources with a single domain name (example.com) or subdomain name (acme.example.com) and choose how much traffic is routed to each resource. This can be useful for a variety of purposes, including load balancing and testing new versions of the software. This is not useful for the current use case.

References:

https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/dns-failover-types.html#dns-failover-types-active-passive

https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html#routing-policy-latency

Domain

Design Resilient Architectures

Question 27Skipped

A health-care company manages its web application on Amazon EC2 instances running behind Auto Scaling group (ASG). The company provides ambulances for critical patients and needs the application to be reliable. The workload of the company can be managed on 2 Amazon EC2 instances and can peak up to 6 instances when traffic increases.

As a Solutions Architect, which of the following configurations would you select as the best fit for these requirements?

The Auto Scaling group should be configured with the minimum capacity set to 4, with 2 instances each in two different AWS Regions. The maximum capacity of the Auto Scaling group should be set to 6

Correct answer

The Auto Scaling group should be configured with the minimum capacity set to 4, with 2 instances each in two different Availability Zones. The maximum capacity of the Auto Scaling group should be set to 6

The Auto Scaling group should be configured with the minimum capacity set to 2 and the maximum capacity set to 6 in a single Availability Zone

The Auto Scaling group should be configured with the minimum capacity set to 2, with 1 instance each in two different Availability Zones. The maximum capacity of the Auto Scaling group should be set to 6

Overall explanation

Correct option:

The Auto Scaling group should be configured with the minimum capacity set to 4, with 2 instances each in two different Availability Zones. The maximum capacity of the Auto Scaling group should be set to 6

You configure the size of your Auto Scaling group by setting the minimum, maximum, and desired capacity. The minimum and maximum capacity are required to create an Auto Scaling group, while the desired capacity is optional. If you do not define your desired capacity upfront, it defaults to your minimum capacity.

Amazon EC2 Auto Scaling enables you to take advantage of the safety and reliability of geographic redundancy by spanning Auto Scaling groups across multiple Availability Zones within a Region. When one Availability Zone becomes unhealthy or unavailable, Auto Scaling launches new instances in an unaffected Availability Zone. When the unhealthy Availability Zone returns to a healthy state, Auto Scaling automatically redistributes the application instances evenly across all of the designated Availability Zones. Since the application is extremely critical and needs to have a reliable architecture to support it, the Amazon EC2 instances should be maintained in at least two Availability Zones (AZs) for uninterrupted service.

Amazon EC2 Auto Scaling attempts to distribute instances evenly between the Availability Zones that are enabled for your Auto Scaling group. This is why the minimum capacity should be 4 instances and not 2.

Auto Scaling group will launch 2 instances each in both the AZs and this redundancy is needed to keep the service available always.

Incorrect options:

The Auto Scaling group should be configured with the minimum capacity set to 2, with 1 instance each in two different Availability Zones. The maximum capacity of the Auto Scaling group should be set to 6

The Auto Scaling group should be configured with the minimum capacity set to 2 and the maximum capacity set to 6 in a single Availability Zone

The explanation above gives the correct rationale for minimum capacity as well as the instance distribution across AZs, so both these options are incorrect.

The Auto Scaling group should be configured with the minimum capacity set to 4, with 2 instances each in two different AWS Regions. The maximum capacity of the Auto Scaling group should be set to 6 - An Auto Scaling group can contain Amazon EC2 instances in one or more Availability Zones within the same region. However, Auto Scaling groups cannot span multiple Regions.

Reference:

https://docs.aws.amazon.com/autoscaling/ec2/userguide/auto-scaling-benefits.html

Domain

Design Resilient Architectures

Question 28Skipped

A financial auditing firm uses Amazon S3 to store sensitive client records that are subject to write-once-read-many (WORM) regulations to prevent alteration or deletion of records for a specific retention period. The firm wants to enforce immutable storage, such that even administrators cannot overwrite or delete the records during the lock duration. They also need audit-friendly enforcement to prevent accidental or malicious deletion.

Which configuration of S3 Object Lock will ensure that the retention policy is strictly enforced, and no user (including root or administrators) can override or delete protected objects during the lock period?

Use S3 Lifecycle Policies to transition data to Glacier Deep Archive and treat it as immutable during the archival period

Correct answer

Use S3 Object Lock in Compliance Mode, which enforces retention policies strictly and prevents all users from modifying or deleting data during the retention period

Enable S3 Versioning and set a bucket policy that denies s3:DeleteObject to all users during the retention period

Use S3 Object Lock in Governance Mode, which allows only IAM users with elevated permissions to override or remove retention settings

Overall explanation

Correct option:

Use S3 Object Lock in Compliance Mode, which enforces retention policies strictly and prevents all users from modifying or deleting data during the retention period

S3 Object Lock in Compliance Mode is designed to meet stringent regulatory requirements. In this mode, no user, not even the root user, can override or delete a locked object until the retention period expires. This ensures strict WORM behavior, making Compliance Mode ideal for regulated industries where immutability must be enforced regardless of user permissions. Attempting to modify or delete objects locked in compliance mode results in an AccessDenied error, even for administrators.

Retention modes

S3 Object Lock provides two retention modes that apply different levels of protection to your objects:

- Compliance mode
- Governance mode

In *compliance* mode, a protected object version can't be overwritten or deleted by any user, including the root user in your AWS account. When an object is locked in compliance mode, its retention mode can't be changed, and its retention period can't be shortened. Compliance mode helps ensure that an object version can't be overwritten or deleted for the duration of the retention period.

> **Note**
> The only way to delete an object under the compliance mode before its retention date expires is to delete the associated AWS account.

In *governance* mode, users can't overwrite or delete an object version or alter its lock settings unless they have special permissions. With governance mode, you protect objects against being deleted by most users, but you can still grant some users permission to alter the retention settings or delete the objects if necessary. You can also use governance mode to test retention-period settings before creating a compliance-mode retention period.

To override or remove governance-mode retention settings, you must have the `s3:BypassGovernanceRetention` permission and must explicitly include `x-amz-bypass-governance-retention:true` as a request header with any request that requires overriding governance mode.

> **Note**
> By default, the Amazon S3 console includes the `x-amz-bypass-governance-retention:true` header. If you try to delete objects protected by *governance* mode and have the `s3:BypassGovernanceRetention` permission, the operation will succeed.

Legal holds

With Object Lock, you can also place a *legal hold* on an object version. Like a retention period, a legal hold prevents an object version from being overwritten or deleted. However, a legal hold doesn't have an associated fixed amount of time and remains in effect until removed. Legal holds can be freely placed and removed by any user who has the `s3:PutObjectLegalHold` permission.

Legal holds are independent from retention periods. Placing a legal hold on an object version doesn't affect the retention mode or retention period for that object version.

For example, suppose that you place a legal hold on an object version and that object version is also protected by a retention period. If the retention period expires, the object doesn't lose its WORM protection. Rather, the legal hold continues to protect the object until an authorized user explicitly removes the legal hold. Similarly, if you remove a legal hold while an object version has a retention period in effect, the object version remains protected until the retention period expires.

Best practices for using S3 Object Lock

Consider using *Governance mode* if you want to protect objects from being deleted by most users during a pre-defined retention period, but at the same time want some users with special permissions to have the flexibility to alter the retention settings or delete the objects.

Consider using *Compliance mode* if you never want any user, including the root user in your AWS account, to be able to delete the objects during a pre-defined retention period. You can use this mode in case you have a requirement to store compliant data.

You can use *Legal Hold* when you are not sure for how long you want your objects to stay immutable. This could be because you have an upcoming external audit of your data and want to keep objects immutable till the audit is complete. Alternately, you may have an ongoing project utilizing a dataset that you want to keep immutable until the project is complete.

via - https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html#object-lock-retention-modes

Incorrect options:

Use S3 Object Lock in Governance Mode, which allows only IAM users with elevated permissions to override or remove retention settings - Governance Mode also provides WORM-like protections, but unlike Compliance Mode, it allows privileged users (with the s3:BypassGovernanceRetention permission) to override retention settings or delete objects before the retention period expires. This is useful in environments where operational flexibility is needed, but does not meet regulatory immutability requirements because deletions are still technically possible. Therefore, Governance Mode is not sufficient for legally enforced retention.

Enable S3 Versioning and set a bucket policy that denies s3:DeleteObject to all users during the retention period - While enabling versioning and denying s3:DeleteObject through a bucket policy can prevent casual deletion, this approach is not tamper-proof and does not meet regulatory compliance for WORM storage. A user with sufficient IAM permissions (e.g., policy modification rights) can update or remove the bucket policy, making this control circumventable. Object Lock in Compliance Mode provides enforced immutability, which cannot be achieved using only versioning and bucket policies.

Use S3 Lifecycle Policies to transition data to Glacier Deep Archive and treat it as immutable during the archival period - S3 Lifecycle Policies are used to transition objects between storage classes or delete them after a specified time. Transitioning data to Glacier Deep Archive does not make the objects immutable. Users can still delete archived objects unless Object Lock is also enabled. Lifecycle configurations are not designed for WORM enforcement and do not prevent deletion or modification during archival.

Reference:

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lock.html#object-lock-retention-modes

Domain

Design Secure Architectures

Question 29Skipped

A financial analytics firm runs performance-intensive modeling software on Amazon EC2 instances backed by Amazon EBS volumes. The production data resides on EBS volumes attached to EC2 instances in the same AWS Region where the testing environment is hosted. To maintain data integrity, any changes made during testing must not affect production data. The development team needs to frequently create clones of this production data for simulations. The modeling software requires high and consistent I/O performance, and the firm wants to minimize the time required to provision test data.

Which solution should a solutions architect recommend to meet these requirements?

Correct answer

Take snapshots of the production EBS volumes. Enable EBS fast snapshot restore on the snapshots. Create new EBS volumes from the snapshots and attach them to EC2 instances in the test environment

Use Amazon EBS io2 volumes with Multi-Attach enabled. Attach the same production EBS volumes to both the production and test EC2 instances simultaneously to avoid cloning delays and ensure high IOPS performance

Create new EBS volumes in the test environment and use AWS Backup to perform a backup job of the production volumes. Restore the backup directly to the test EBS volumes to begin simulations

Create Amazon EBS-backed Amazon Machine Images (AMIs) from the production EC2 instances. Launch new EC2 instances in the test environment from the AMIs. Use Amazon EC2 instance store volumes for temporary simulation data

Overall explanation

Correct option:

Take snapshots of the production EBS volumes. Enable EBS fast snapshot restore on the snapshots. Create new EBS volumes from the snapshots and attach them to EC2 instances in the test environment

This solution uses EBS fast snapshot restore (FSR), which allows immediate, full-performance access to new EBS volumes created from snapshots—eliminating the usual initialization delay. Creating volumes from snapshots ensures that test data is logically cloned without impacting the source, and the FSR feature guarantees the I/O performance required by the analytics workloads. This method is efficient, scalable, and preserves isolation between test and production environments.

Incorrect options:

Create Amazon EBS-backed Amazon Machine Images (AMIs) from the production EC2 instances. Launch new EC2 instances in the test environment from the AMIs. Use Amazon EC2 instance store volumes for temporary simulation data - While AMIs do provide a fast way to clone EC2 instances, EBS-backed AMIs are optimized for instance bootstrapping, not large-scale data cloning or simulation. Additionally, using instance store volumes for high I/O operations is risky, as they are ephemeral and can be lost on stop/terminate. This setup doesn't offer the persistent high-performance storage required for consistent simulation workloads and would not scale well for repeated cloning needs.

Create new EBS volumes in the test environment and use AWS Backup to perform a backup job of the production volumes. Restore the backup directly to the test EBS volumes to begin simulations - While AWS Backup supports backing up and restoring EBS volumes, it is designed for data protection and compliance use cases, not for rapid, low-latency cloning. The process of creating a backup job and restoring to new volumes adds significant delay and operational overhead compared to using native EBS snapshots with fast snapshot restore. Additionally, AWS Backup does not provide the same flexibility or speed when initiating volume restoration, especially when frequent or automated cloning is required. This approach does not meet the goal of minimizing the time to clone data for performance-critical testing.

Use Amazon EBS io2 volumes with Multi-Attach enabled. Attach the same production EBS volumes to both the production and test EC2 instances simultaneously to avoid cloning delays and ensure

high IOPS performance - While Amazon EBS Multi-Attach allows an io1 or io2 volume to be attached to multiple EC2 instances within the same Availability Zone, it is designed for use in clustered, coordinated applications (such as shared-disk file systems or clustered databases). Attaching the same production volume to both production and test environments introduces the risk of uncoordinated writes, data corruption, and unexpected state changes, especially since the requirement explicitly states that test modifications must not impact production data. Additionally, Multi-Attach does not eliminate the need for cloning, it merely provides concurrent access, which is unsuitable for isolated test simulations.

References:

https://docs.aws.amazon.com/ebs/latest/userguide/ebs-fast-snapshot-restore.html

https://docs.aws.amazon.com/ebs/latest/userguide/ebs-volumes-multi.html

Domain

Design High-Performing Architectures

Question 30Skipped

The engineering team at a retail company manages 3 Amazon EC2 instances that make read-heavy database requests to the Amazon RDS for the PostgreSQL database instance. As an AWS Certified Solutions Architect - Associate, you have been tasked to make the database instance resilient from a disaster recovery perspective.

Which of the following features will help you in disaster recovery of the database? (Select two)

Enable the automated backup feature of Amazon RDS in a multi-AZ deployment that creates backups in a single AWS Region

Correct selection

Use cross-Region Read Replicas

Correct selection

Enable the automated backup feature of Amazon RDS in a multi-AZ deployment that creates backups across multiple Regions

Use the database cloning feature of the Amazon RDS Database cluster

Use Amazon RDS Provisioned IOPS (SSD) Storage in place of General Purpose (SSD) Storage

Overall explanation

Correct options:

Use cross-Region Read Replicas

In addition to using Read Replicas to reduce the load on your source database instance, you can also use Read Replicas to implement a DR solution for your production DB environment. If the source DB instance fails, you can promote your Read Replica to a standalone source server. Read Replicas can also be

created in a different Region than the source database. Using a cross-Region Read Replica can help ensure that you get back up and running if you experience a regional availability issue.

Enable the automated backup feature of Amazon RDS in a multi-AZ deployment that creates backups across multiple Regions

Amazon RDS provides high availability and failover support for database instances using Multi-AZ deployments. Amazon RDS uses several different technologies to provide failover support. Multi-AZ deployments for MariaDB, MySQL, Oracle, and PostgreSQL DB instances use Amazon's failover technology.

The automated backup feature of Amazon RDS enables point-in-time recovery for your database instance. Amazon RDS will back up your database and transaction logs and store both for a user-specified retention period. If it's a Multi-AZ configuration, backups occur on standby to reduce the I/O impact on the primary. Amazon RDS supports Cross-Region Automated Backups. Manual snapshots and Read Replicas are also supported across multiple Regions.

Incorrect options:

Enable the automated backup feature of Amazon RDS in a multi-AZ deployment that creates backups in a single AWS Region - This is an incorrect statement. Automated backups can be created across AWS Regions.

Use Amazon RDS Provisioned IOPS (SSD) Storage in place of General Purpose (SSD) Storage - Amazon RDS Provisioned IOPS Storage is an SSD-backed storage option designed to deliver fast, predictable, and consistent I/O performance. This storage type enhances the performance of the RDS database, but this isn't a disaster recovery option.

Use the database cloning feature of the Amazon RDS Database cluster - This option has been added as a distractor. Database cloning is only available for Amazon Aurora and not for Amazon RDS.

References:

https://aws.amazon.com/rds/features/

https://aws.amazon.com/blogs/database/implementing-a-disaster-recovery-strategy-with-amazon-rds/

https://aws.amazon.com/about-aws/whats-new/2021/07/amazon-rds-cross-region-automated-backups-regional-expansion/

Domain

Design Resilient Architectures

Question 31Skipped

A logistics company runs a two-step job handling process on AWS. The first step quickly receives job submissions from clients, while the second step requires longer processing time to complete each job. Currently, both steps run on separate Amazon EC2 Auto Scaling groups. However, during high-demand hours, the job processing stage falls behind, and there is concern that jobs may be lost due to instance

termination during scaling events. A solutions architect needs to design a more scalable and reliable architecture that preserves job data and accommodates fluctuating demand in both stages.

Which solution will meet these requirements?

Set up two Amazon SQS queues to decouple the job intake and job processing stages respectively. Assign one SQS queue to collect incoming jobs, and another to queue them for processing. Configure the EC2 instances to poll the relevant queue. Scale the Auto Scaling groups based on notifications from each queue

Correct answer

Set up two Amazon SQS queues to decouple the job intake and job processing stages respectively. Assign one SQS queue to collect incoming jobs, and another to queue them for processing. Configure the EC2 instances to poll the relevant queue. Scale the Auto Scaling groups based on number of messages in each queue

Set up a single Amazon SQS queue for both the job intake and job processing stages. Assign the SQS queue to collect incoming jobs as well as processing jobs. Configure all EC2 instances to poll this queue. Scale the Auto Scaling groups based on number of messages in the queue

Configure each Auto Scaling group to maintain its maximum expected size during peak hours by setting a fixed minimum capacity. Monitor CPUUtilization through Amazon CloudWatch to ensure consistent scaling behavior

Overall explanation

Correct option:

Set up two Amazon SQS queues to decouple the job intake and job processing stages respectively. Assign one SQS queue to collect incoming jobs, and another to queue them for processing. Configure the EC2 instances to poll the relevant queue. Scale the Auto Scaling groups based on number of messages in each queue

This solution effectively addresses both the need for scalability and the requirement to avoid data loss. By introducing Amazon SQS queues between the two processing stages (order collection and order fulfillment), the architecture becomes decoupled and resilient. One queue buffers incoming orders, while the second queue holds orders awaiting fulfillment, ensuring no data is lost even when processing slows during peak traffic. Each group of EC2 instances polls its respective queue for work, and Auto Scaling policies can be configured based on the number of messages in each queue. This allows the system to dynamically scale based on real-time demand, ensuring responsiveness without over-provisioning resources. This approach minimizes latency for order intake while allowing asynchronous processing of order fulfillment, leading to a highly available and cost-efficient solution.

Incorrect options:

Set up a single Amazon SQS queue for both the job intake and job processing stages. Assign the SQS queue to collect incoming jobs as well as processing jobs. Configure all EC2 instances to poll

this queue. Scale the Auto Scaling groups based on number of messages in the queue - This option is incorrect because using a single Amazon SQS queue for both job intake and job processing introduces ambiguity and tight coupling between two logically distinct stages—order collection and order fulfillment. In this architecture, the order intake is fast, while order fulfillment is slower and more resource-intensive. By combining both types of messages into a single SQS queue, there's no way to prioritize, distinguish, or independently scale resources for each task. For example, the fulfillment EC2 instances might mistakenly process intake-type messages or vice versa unless complex custom filtering logic is implemented, which increases operational complexity and risk.

Configure each Auto Scaling group to maintain its maximum expected size during peak hours by setting a fixed minimum capacity. Monitor CPUUtilization through Amazon CloudWatch to ensure consistent scaling behavior - This approach pre-provisions the infrastructure for peak traffic, using CPUUtilization as a general metric. While this reduces the risk of lag due to insufficient instances, it lacks flexibility and may result in over-provisioning, leading to unnecessary costs. It also does not guarantee data persistence if instances terminate unexpectedly during processing, since there's no queue-based buffering.

Set up two Amazon SQS queues to decouple the job intake and job processing stages respectively. Assign one SQS queue to collect incoming jobs, and another to queue them for processing. Configure the EC2 instances to poll the relevant queue. Scale the Auto Scaling groups based on notifications from each queue - While this solution correctly suggests decoupling the order collection and order fulfillment processes using Amazon SQS queues, it proposes using notifications from the queues to trigger scaling. This acts as a distractor, since SQS queues do not support sending notifications. Notifications (e.g., using Amazon SNS) are not the most effective mechanism for dynamic and responsive scaling. The more reliable and recommended approach is to scale based on the number of pending messages in each queue.

References:

https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-using-sqs-queue.html

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/welcome.html

Domain

Design Resilient Architectures

Question 32Skipped

A digital media company runs its content rendering service on Amazon EC2 instances that are registered with an Application Load Balancer (ALB) using IP-based target groups. The company relies on AWS Systems Manager to manage and patch these instances regularly. According to new compliance requirements, EC2 instances must be safely removed from production traffic during patching to prevent user disruption and maintain application integrity. However, during the most recent patch cycle, the operations team noticed application failures and API timeouts, even though patching succeeded on the instances. You are asked to suggest a reliable and scalable way to ensure safe patching while preserving service availability.

Which solution will best meet the new compliance and operational requirements? (Select two)

Configure a custom Lambda function triggered by an Amazon EventBridge rule that disables the EC2 instance's network interface during the patching window and re-enables it after patching completes

Correct selection

Configure Systems Manager Maintenance Windows to coordinate patching and instance removal from the ALB during the defined window

Use Amazon CloudWatch Logs Insights to monitor patching success and then manually adjust ALB target group registrations before and after each patch window

Modify the load balancer configuration to attach EC2 instances using instance ID-based target groups instead of IP-based targets, allowing Systems Manager to directly communicate with instance metadata

Correct selection

Use AWS Systems Manager Automation with the AWSEC2-PatchLoadBalancerInstance document to manage patching

Overall explanation

Correct options:

Use AWS Systems Manager Automation with the AWSEC2-PatchLoadBalancerInstance document to manage patching

The AWSEC2-PatchLoadBalancerInstance Systems Manager Automation document is specifically designed for patching EC2 instances that are part of a load balancer. It automatically removes the instance from the ALB target group, waits for in-flight requests to complete, applies patches, performs reboots if needed, and then safely re-registers the instance. This workflow prevents application downtime or request failures during patching and ensures compliance with security policies.

Configure Systems Manager Maintenance Windows to coordinate patching and instance removal from the ALB during the defined window

Systems Manager Maintenance Windows can be configured to run Automation documents or Lambda functions at precise times. You can schedule a task to remove instances from the load balancer using pre-defined documents (such as AWSEC2-PatchLoadBalancerInstance) and then re-register them once patching is complete. This offers fine-grained scheduling and orchestration for controlled patching without disrupting traffic.

Incorrect options:

Configure a custom Lambda function triggered by an Amazon EventBridge rule that disables the EC2 instance's network interface during the patching window and re-enables it after patching completes - This is a completely impractical and risky approach. Disabling the instance's network interface during patching would sever communication with AWS Systems Manager, which relies on the SSM Agent and internet/network connectivity to manage the patching workflow. This would likely cause

patching failures or leave instances in an inconsistent state. It also introduces custom automation and complexity that is neither secure nor supported by AWS best practices.

Modify the load balancer configuration to attach EC2 instances using instance ID-based target groups instead of IP-based targets, allowing Systems Manager to directly communicate with instance metadata - Switching from IP-based to instance ID-based targets does not solve the problem related to application failures during patching. The issue lies in how traffic is handled during the patching process—not in the communication between Systems Manager and the instances. Furthermore, IP-based target groups are commonly used when EC2 instances are managed through services like ECS or are deployed in different subnets or VPCs. Changing the target type could introduce compatibility or architectural issues without addressing the root cause.

Use Amazon CloudWatch Logs Insights to monitor patching success and then manually adjust ALB target group registrations before and after each patch window - While monitoring with CloudWatch Logs Insights can provide visibility into patching status, it does not automate the removal or addition of instances to/from the ALB. Manually adjusting target group registrations is not scalable, prone to human error, and does not align with the goal of automation and compliance. This option lacks operational efficiency and doesn't prevent traffic from reaching instances while they are being patched.

References:

https://docs.aws.amazon.com/systems-manager/latest/userguide/what-is-systems-manager.html

https://docs.aws.amazon.com/systems-manager-automation-runbooks/latest/userguide/automation-awsec2-patch-load-balancer-instance.html

https://docs.aws.amazon.com/systems-manager-automation-runbooks/latest/userguide/automation-runbook-reference.html

https://docs.aws.amazon.com/elasticloadbalancing/latest/application/load-balancer-target-groups.html

Domain

Design Resilient Architectures

Question 33Skipped

The content division at a digital media agency has an application that generates a large number of files on Amazon S3, each approximately 10 megabytes in size. The agency mandates that the files be stored for 5 years before they can be deleted. The files are frequently accessed in the first 30 days of the object creation but are rarely accessed after the first 30 days. The files contain critical business data that is not easy to reproduce, therefore, immediate accessibility is always required.

Which solution is the MOST cost-effective for the given use case?

Set up an Amazon S3 bucket lifecycle policy to move files from Amazon S3 Standard to Amazon S3 Glacier Flexible Retrieval 30 days after object creation. Delete the files 5 years after object creation

Set up an Amazon S3 bucket lifecycle policy to move files from Amazon S3 Standard to Amazon S3 One Zone-IA 30 days after object creation. Delete the files 5 years after object creation

Correct answer

Set up an Amazon S3 bucket lifecycle policy to move files from Amazon S3 Standard to Amazon S3 Standard-IA 30 days after object creation. Delete the files 5 years after object creation

Set up an Amazon S3 bucket lifecycle policy to move files from Amazon S3 Standard to Amazon S3 Standard-IA 30 days after object creation. Archive the files to Amazon S3 Glacier Deep Archive 5 years after object creation

Overall explanation

Correct option:

Set up an Amazon S3 bucket lifecycle policy to move files from Amazon S3 Standard to Amazon S3 Standard-IA 30 days after object creation. Delete the files 5 years after object creation

Amazon S3 Standard-IA class is for data that is accessed less frequently but requires rapid access when needed. Amazon S3 Standard-IA offers the high durability, high throughput, and low latency of S3 Standard, with a low per gigabyte storage price and per GB retrieval charge.

Performance across the S3 Storage Classes

	S3 Standard	S3 Intelligent-Tiering*	S3 Standard-IA	S3 One Zone-IA†	S3 Glacier Instant Retrieval	S3 Glacier Flexible Retrieval	S3 Glacier Deep Archive
Designed for durability	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)	99.999999999% (11 9's)
Designed for availability	99.99%	99.9%	99.9%	99.5%	99.9%	99.99%	99.99%
Availability SLA	99.9%	99%	99%	99%	99%	99.%	99.9%
Availability Zones	≥3	≥3	≥3	1	≥3	≥3	≥3
Minimum capacity charge per object	N/A	N/A	128 KB	128 KB	128 KB	40 KB	40 KB
Minimum storage duration charge	N/A	N/A	30 days	30 days	90 days	90 days	180 days
Retrieval charge	N/A	N/A	per GB retrieved	per GB retrieved	per GB retrieved	per GB retrieved	per GB retrieved
First byte latency	milliseconds	milliseconds	milliseconds	milliseconds	milliseconds	minutes or hours	hours
Storage type	Object	Object	Object	Object	Object	Object	Object
Lifecycle transitions	Yes	Yes	Yes	Yes	Yes	Yes	Yes

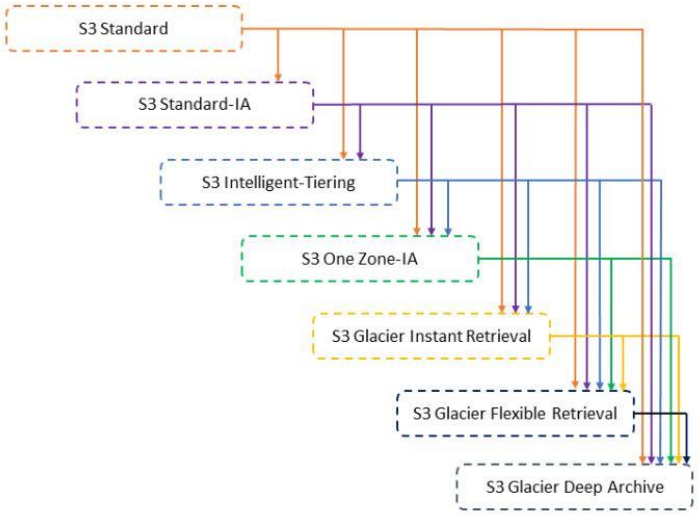
via - https://aws.amazon.com/s3/storage-classes/

For the given use case, you can set up an Amazon S3 lifecycle configuration and create a transition action to move objects from Amazon S3 Standard to Amazon S3 Standard-IA 30 days after object creation. You can set up an expiration action to delete the object 5 years after object creation.

Supported transitions and related constraints

In an S3 Lifecycle configuration, you can define rules to transition objects from one storage class to another to save on storage costs. When you don't know the access patterns of your objects, or if your access patterns are changing over time, you can transition the objects to the S3 Intelligent-Tiering storage class for automatic cost savings. For information about storage classes, see Using Amazon S3 storage classes.

Amazon S3 supports a waterfall model for transitioning between storage classes, as shown in the following diagram.



via - https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-transition-general-considerations.html

Managing your storage lifecycle

PDF | RSS

To manage your objects so that they are stored cost effectively throughout their lifecycle, configure their *Amazon S3 Lifecycle*. An *S3 Lifecycle configuration* is a set of rules that define actions that Amazon S3 applies to a group of objects. There are two types of actions:

- **Transition actions** – These actions define when objects transition to another storage class. For example, you might choose to transition objects to the S3 Standard-IA storage class 30 days after creating them, or archive objects to the S3 Glacier Flexible Retrieval storage class one year after creating them. For more information, see Using Amazon S3 storage classes.

  There are costs associated with lifecycle transition requests. For pricing information, see Amazon S3 pricing.
- **Expiration actions** – These actions define when objects expire. Amazon S3 deletes expired objects on your behalf.

  Lifecycle expiration costs depend on when you choose to expire objects. For more information, see Expiring objects.

If there is any delay between when an object becomes eligible for a lifecycle action and when Amazon S3 transfers or expires your object, billing changes are applied as soon as the object becomes eligible for the lifecycle action. For example, if an object is scheduled to expire and Amazon S3 does not immediately expire the object, you won't be charged for storage after the expiration time. The one exception to this behavior is if you have a lifecycle rule to transition to the S3 Intelligent-Tiering storage class. In that case, billing changes do not occur until the object has transitioned to S3 Intelligent-Tiering.

via - https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html

Incorrect options:

Set up an Amazon S3 bucket lifecycle policy to move files from Amazon S3 Standard to Amazon S3 Glacier Flexible Retrieval 30 days after object creation. Delete the files 5 years after object creation - Amazon S3 Glacier Flexible Retrieval storage class has the best case retrieval time of the order of minutes, so this option is incorrect for the given requirement.

Set up an Amazon S3 bucket lifecycle policy to move files from Amazon S3 Standard to Amazon S3 Standard-IA 30 days after object creation. Archive the files to Amazon S3 Glacier Deep Archive 5 years after object creation - The files can simply be deleted 5 years after object creation instead of archiving the files to Amazon S3 Glacier Deep Archive. There is no need to incur the cost of archival.

Set up an Amazon S3 bucket lifecycle policy to move files from Amazon S3 Standard to Amazon S3 One Zone-IA 30 days after object creation. Delete the files 5 years after object creation - Unlike other Amazon S3 Storage Classes which store data in a minimum of three Availability Zones (AZs), Amazon S3 One Zone-IA stores data in a single AZ and costs 20% less than Amazon S3 Standard-IA. Amazon S3 One Zone-IA is a good choice for storing secondary backup copies of on-premises data or easily re-creatable data. The given scenario clearly states that the business-critical data is not easy to reproduce, so this option is incorrect.

References:

https://aws.amazon.com/s3/storage-classes/

https://aws.amazon.com/s3/storage-classes/glacier/

https://docs.aws.amazon.com/AmazonS3/latest/userguide/lifecycle-transition-general-considerations.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/object-lifecycle-mgmt.html

Domain

Design Cost-Optimized Architectures

Question 34Skipped

A biomedical research firm operates a file exchange system for external research partners to upload and download experimental data. Currently, the system runs on two Amazon EC2 Linux instances, each configured with Elastic IP addresses to allow access from trusted IPs. File transfers use the SFTP protocol, and Linux user accounts are manually provisioned to enforce file-level access control. Data is stored on a shared file system mounted to both EC2 instances. The firm wants to modernize the solution to a fully managed, serverless model with high IOPS, fine-grained user permission control, and strict IP-based access restrictions. They also want to reduce operational overhead without sacrificing performance or security.

Which solution best meets these requirements?

Use Amazon S3 with server-side encryption enabled. Create an AWS Transfer Family SFTP endpoint with a VPC endpoint in a private subnet. Restrict access to known IPs using security group rules. Manage user-level permissions using IAM role-based access mappings

Correct answer

Use Amazon EFS with encryption enabled. Create an AWS Transfer Family SFTP endpoint in a VPC with Elastic IP addresses. Restrict access using a security group that allows traffic only from known IPs. Manage user access using POSIX identity mappings and IAM policies

Use Amazon FSx for Lustre as the backend storage. Create an AWS Transfer Family SFTP service with a public endpoint. Configure IAM policies to manage user access and attach a security group that restricts access to trusted IP addresses

Use AWS Storage Gateway in file gateway mode to expose an NFS file share. Deploy AWS Transfer Family with a public endpoint and map user identities using IAM roles. Configure IP allow lists using AWS WAF

Overall explanation

Correct option:

Use Amazon EFS with encryption enabled. Create an AWS Transfer Family SFTP endpoint in a VPC with Elastic IP addresses. Restrict access using a security group that allows traffic only from known IPs. Manage user access using POSIX identity mappings and IAM policies

This solution uses Amazon EFS, which is natively supported by AWS Transfer Family and provides shared, low-latency, and scalable file storage, ideal for high IOPS use cases. Creating the SFTP endpoint within a VPC with Elastic IP addresses allows secure, internet-facing access. Using IAM policies and POSIX user ID mappings, administrators can enforce per-user access control, similar to Linux user permissions. The security group restricts SFTP access to trusted IP addresses. This configuration satisfies all the company's performance, security, and manageability goals in a serverless architecture.

Incorrect options:

Use Amazon FSx for Lustre as the backend storage. Create an AWS Transfer Family SFTP service with a public endpoint. Configure IAM policies to manage user access and attach a security group that restricts access to trusted IP addresses - While Amazon FSx for Lustre offers high-performance, POSIX-compliant storage, it is not natively integrated with AWS Transfer Family as a backend for SFTP services. Transfer Family only supports Amazon S3 and Amazon EFS as data stores. This option would require complex, unsupported workarounds to integrate FSx, making it operationally risky and unsupported.

Use AWS Storage Gateway in file gateway mode to expose an NFS file share. Deploy AWS Transfer Family with a public endpoint and map user identities using IAM roles. Configure IP allow lists using AWS WAF - AWS Storage Gateway (file gateway mode) allows access to S3 via NFS or SMB but is not natively integrated with AWS Transfer Family. Although it supports on-premises caching and hybrid workloads, using it in this context introduces unnecessary complexity and cannot meet the requirement for native SFTP integration with a serverless backend. AWS WAF is also not applicable for SFTP traffic, as it operates on Layer 7 HTTP(S) protocols.

Use Amazon S3 with server-side encryption enabled. Create an AWS Transfer Family SFTP endpoint with a VPC endpoint in a private subnet. Restrict access to known IPs using security group rules. Manage user-level permissions using IAM role-based access mappings - While Amazon S3 is a

supported backend for AWS Transfer Family and offers excellent scalability and durability, it does not offer file system semantics or the high IOPS required by workloads involving frequent reads/writes like SFTP-based workflows. S3 performance is sufficient for bulk transfer or archiving, but EFS is a better choice for interactive file exchange with low latency and high throughput. This solution satisfies access control requirements but falls short on performance.

References:

https://docs.aws.amazon.com/transfer/latest/userguide/what-is-aws-transfer-family.html

https://docs.aws.amazon.com/filegateway/latest/files3/what-is-file-s3.html

https://docs.aws.amazon.com/fsx/latest/LustreGuide/what-is.html

Domain

Design Secure Architectures

Question 35Skipped

An e-commerce company uses Amazon Simple Queue Service (Amazon SQS) queues to decouple their application architecture. The engineering team has observed message processing failures for some customer orders.

As a solutions architect, which of the following solutions would you recommend for handling such message failures?

Use a temporary queue to handle message processing failures

Use short polling to handle message processing failures

Use long polling to handle message processing failures

Correct answer

Use a dead-letter queue to handle message processing failures

Overall explanation

Correct option:

Use a dead-letter queue to handle message processing failures

Dead-letter queues can be used by other queues (source queues) as a target for messages that can't be processed (consumed) successfully. Dead-letter queues are useful for debugging your application or messaging system because they let you isolate problematic messages to determine why their processing doesn't succeed. Sometimes, messages can’t be processed because of a variety of possible issues, such as when a user comments on a story but it remains unprocessed because the original story itself is deleted by the author while the comments were being posted. In such a case, the dead-letter queue can be used to handle message processing failures.

How do dead-letter queues work?:

How do dead-letter queues work?

Sometimes, messages can't be processed because of a variety of possible issues, such as erroneous conditions within the producer or consumer application or an unexpected state change that causes an issue with your application code. For example, if a user places a web order with a particular product ID, but the product ID is deleted, the web store's code fails and displays an error, and the message with the order request is sent to a dead-letter queue.

Occasionally, producers and consumers might fail to interpret aspects of the protocol that they use to communicate, causing message corruption or loss. Also, the consumer's hardware errors might corrupt message payload.

The *redrive policy* specifies the *source queue*, the *dead-letter queue*, and the conditions under which Amazon SQS moves messages from the former to the latter if the consumer of the source queue fails to process a message a specified number of times. When the `ReceiveCount` for a message exceeds the `maxReceiveCount` for a queue, Amazon SQS moves the message to a dead-letter queue (with its original message ID). For example, if the source queue has a redrive policy with `maxReceiveCount` set to 5, and the consumer of the source queue receives a message 6 times without ever deleting it, Amazon SQS moves the message to the dead-letter queue.

via - https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html

Incorrect options:

Use a temporary queue to handle message processing failures - The most common use case for temporary queues is the request-response messaging pattern (for example, processing a login request), where a requester creates a temporary queue for receiving each response message. To avoid creating an Amazon SQS queue for each response message, the Temporary Queue Client lets you create and delete multiple temporary queues without making any Amazon SQS API calls. Temporary queues cannot be used to handle message processing failures.

Use short polling to handle message processing failures

Use long polling to handle message processing failures

Amazon SQS provides short polling and long polling to receive messages from a queue. By default, queues use short polling. With short polling, Amazon SQS sends the response right away, even if the query found no messages. With long polling, Amazon SQS sends a response after it collects at least one available message, up to the maximum number of messages specified in the request. Amazon SQS sends an empty response only if the polling wait time expires. Neither short polling nor long polling can be used to handle message processing failures.

Reference:

https://docs.aws.amazon.com/AWSSimpleQueueService/latest/SQSDeveloperGuide/sqs-dead-letter-queues.html

Domain

Design Resilient Architectures

Question 36Skipped

A medium-sized business has a taxi dispatch application deployed on an Amazon EC2 instance. Because of an unknown bug, the application causes the instance to freeze regularly. Then, the instance has to be manually restarted via the AWS management console.

Which of the following is the MOST cost-optimal and resource-efficient way to implement an automated solution until a permanent fix is delivered by the development team?

Correct answer

Setup an Amazon CloudWatch alarm to monitor the health status of the instance. In case of an Instance Health Check failure, an EC2 Reboot CloudWatch Alarm Action can be used to reboot the instance

Use Amazon EventBridge events to trigger an AWS Lambda function to reboot the instance status every 5 minutes

Setup an Amazon CloudWatch alarm to monitor the health status of the instance. In case of an Instance Health Check failure, Amazon CloudWatch Alarm can publish to an Amazon Simple Notification Service (Amazon SNS) event which can then trigger an AWS lambda function. The AWS lambda function can use Amazon EC2 API to reboot the instance

Use Amazon EventBridge events to trigger an AWS Lambda function to check the instance status every 5 minutes. In the case of Instance Health Check failure, the AWS lambda function can use Amazon EC2 API to reboot the instance

Overall explanation

Correct option:

Setup an Amazon CloudWatch alarm to monitor the health status of the instance. In case of an Instance Health Check failure, an EC2 Reboot CloudWatch Alarm Action can be used to reboot the instance

Using Amazon CloudWatch alarm actions, you can create alarms that automatically stop, terminate, reboot, or recover your Amazon EC2 instances. You can use the stop or terminate actions to help you save money when you no longer need an instance to be running. You can use the reboot and recover actions to automatically reboot those instances or recover them onto new hardware if a system impairment occurs.

You can create an Amazon CloudWatch alarm that monitors an Amazon EC2 instance and automatically reboots the instance. The reboot alarm action is recommended for Instance Health Check failures (as opposed to the recover alarm action, which is suited for System Health Check failures).

Incorrect options:

Setup an Amazon CloudWatch alarm to monitor the health status of the instance. In case of an Instance Health Check failure, Amazon CloudWatch Alarm can publish to an Amazon Simple Notification Service (Amazon SNS) event which can then trigger an AWS lambda function. The AWS lambda function can use Amazon EC2 API to reboot the instance

Use Amazon EventBridge events to trigger an AWS Lambda function to check the instance status every 5 minutes. In the case of Instance Health Check failure, the AWS lambda function can use Amazon EC2 API to reboot the instance

Use Amazon EventBridge events to trigger an AWS Lambda function to reboot the instance status every 5 minutes

Using Amazon EventBridge event or Amazon CloudWatch alarm to trigger an AWS lambda function, directly or indirectly, is wasteful of resources. You should just use the EC2 Reboot CloudWatch Alarm Action to reboot the instance. So all the options that trigger the AWS lambda function are incorrect.

Reference:

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/UsingAlarmActions.html

Domain

Design Cost-Optimized Architectures

Question 37Skipped

An application with global users across AWS Regions had suffered an issue when the Elastic Load Balancing (ELB) in a Region malfunctioned thereby taking down the traffic with it. The manual intervention cost the company significant time and resulted in major revenue loss.

What should a solutions architect recommend to reduce internet latency and add automatic failover across AWS Regions?

Correct answer

Set up AWS Global Accelerator and add endpoints to cater to users in different geographic locations

Create Amazon S3 buckets in different AWS Regions and configure Amazon CloudFront to pick the nearest edge location to the user

Set up an Amazon Route 53 geoproximity routing policy to route traffic

Set up AWS Direct Connect as the backbone for each of the AWS Regions where the application is deployed

Overall explanation

Correct option:

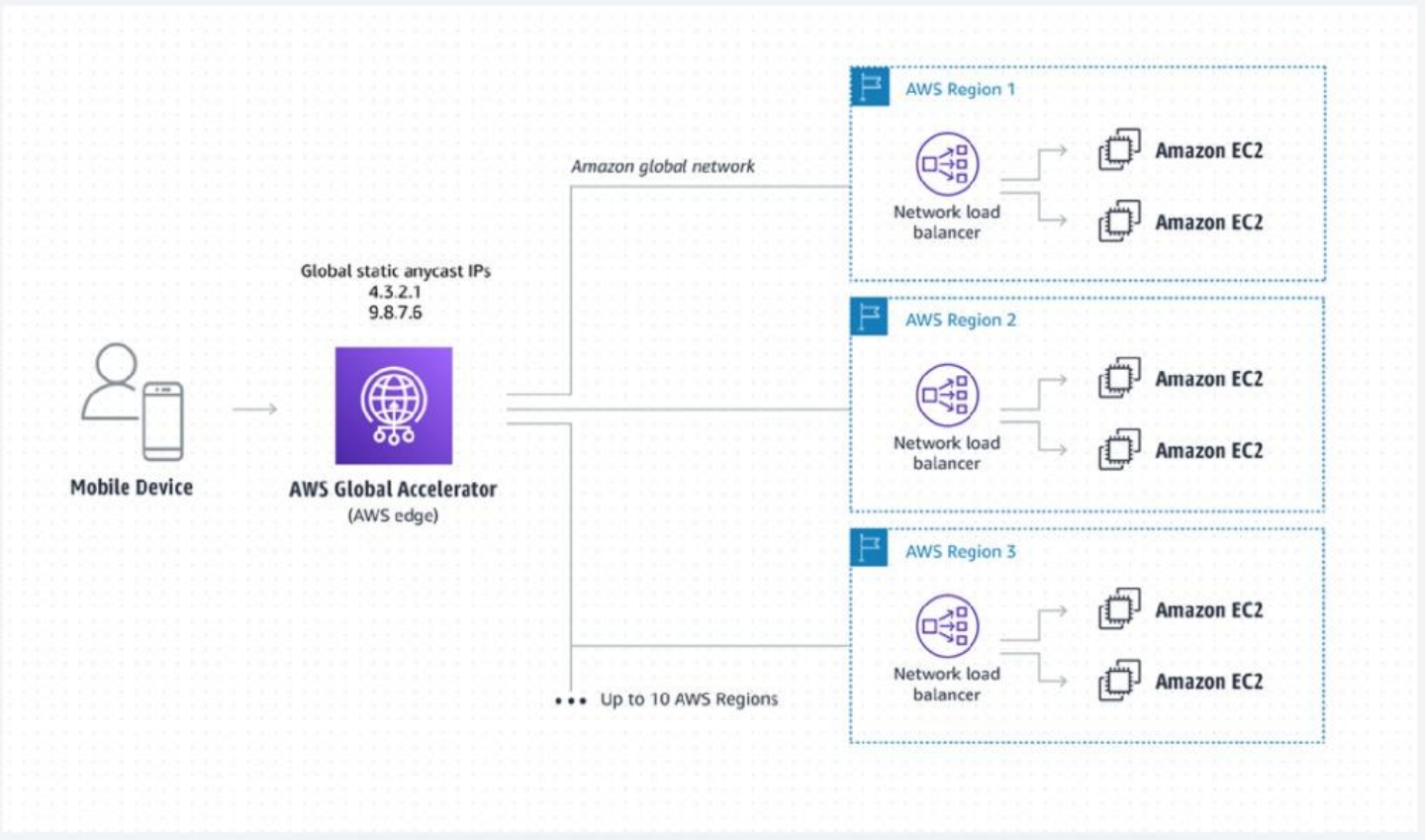
Set up AWS Global Accelerator and add endpoints to cater to users in different geographic locations

As your application architecture grows, so does the complexity, with longer user-facing IP lists and more nuanced traffic routing logic. AWS Global Accelerator solves this by providing you with two static IPs that are anycast from our globally distributed edge locations, giving you a single entry point to your application, regardless of how many AWS Regions it's deployed in. This allows you to add or remove origins, Availability Zones or Regions without reducing your application availability. Your traffic routing is managed manually, or in console with endpoint traffic dials and weights. If your application endpoint has a failure or availability issue, AWS Global Accelerator will automatically redirect your new connections to a healthy endpoint within seconds.

By using AWS Global Accelerator, you can:

1. Associate the static IP addresses provided by AWS Global Accelerator to regional AWS resources or endpoints, such as Network Load Balancers, Application Load Balancers, EC2 Instances, and Elastic IP addresses. The IP addresses are anycast from AWS edge locations so they provide onboarding to the AWS global network close to your users.
2. Easily move endpoints between Availability Zones or AWS Regions without needing to update your DNS configuration or change client-facing applications.
3. Dial traffic up or down for a specific AWS Region by configuring a traffic dial percentage for your endpoint groups. This is especially useful for testing performance and releasing updates.
4. Control the proportion of traffic directed to each endpoint within an endpoint group by assigning weights across the endpoints.

AWS Global Accelerator for Multi-Region applications:



via - https://aws.amazon.com/global-accelerator/

Incorrect options:

Set up AWS Direct Connect as the backbone for each of the AWS Regions where the application is deployed - AWS Direct Connect can reduce latency to great extent. Direct Connect is used to connect on-premises systems to AWS Cloud for extremely low latency use cases. It cannot be used to serve users directly.

Create Amazon S3 buckets in different AWS Regions and configure Amazon CloudFront to pick the nearest edge location to the user - If most of the content is static, we can configure Amazon CloudFront to improve performance. In the current scenario, the architecture has ELBs, Amazon EC2 instances too that need to be covered in the automatic failover plan.

Set up an Amazon Route 53 geoproximity routing policy to route traffic - Geoproximity routing lets Amazon Route 53 route traffic to your resources based on the geographic location of your users and your resources. Unlike AWS Global Accelerator, managing and routing to different instances, ELBs and other AWS resources will become an operational overhead as the resource count reaches into the hundreds. With inbuilt features like Static anycast IP addresses, fault tolerance using network zones, Global performance-based routing, TCP Termination at the Edge - AWS Global Accelerator is the right choice for multi-region, low latency use cases.

References:

https://aws.amazon.com/global-accelerator/features/

https://docs.aws.amazon.com/Route53/latest/DeveloperGuide/routing-policy.html#routing-policy-geoproximity

Domain

Design High-Performing Architectures

Question 38Skipped

A streaming solutions company is building a video streaming product by using an Application Load Balancer (ALB) that routes the requests to the underlying Amazon EC2 instances. The engineering team has noticed a peculiar pattern. The Application Load Balancer removes an instance from its pool of healthy instances whenever it is detected as unhealthy but the Auto Scaling group fails to kick-in and provision the replacement instance.

What could explain this anomaly?

Correct answer

The Auto Scaling group is using Amazon EC2 based health check and the Application Load Balancer is using ALB based health check

Both the Auto Scaling group and Application Load Balancer are using Amazon EC2 based health check

The Auto Scaling group is using ALB based health check and the Application Load Balancer is using Amazon EC2 based health check

Both the Auto Scaling group and Application Load Balancer are using ALB based health check

Overall explanation

Correct option:

The Auto Scaling group is using Amazon EC2 based health check and the Application Load Balancer is using ALB based health check

An Auto Scaling group contains a collection of Amazon EC2 instances that are treated as a logical grouping for automatic scaling and management.

Auto Scaling Group Overview:

via - https://docs.aws.amazon.com/autoscaling/ec2/userguide/what-is-amazon-ec2-auto-scaling.html

Application Load Balancer automatically distributes incoming application traffic across multiple targets, such as Amazon EC2 instances, containers, and AWS Lambda functions. It can handle the varying load of your application traffic in a single Availability Zone or across multiple Availability Zones.

If the Auto Scaling group (ASG) is using EC2 as the health check type and the Application Load Balancer (ALB) is using its in-built health check, there may be a situation where the ALB health check fails because the health check pings fail to receive a response from the instance. At the same time, ASG health check

can come back as successful because it is based on EC2 based health check. Therefore, in this scenario, the ALB will remove the instance from its inventory, however, the Auto Scaling Group will fail to provide the replacement instance. This can lead to the scaling issues mentioned in the problem statement.

Incorrect options:

The Auto Scaling group is using ALB based health check and the Application Load Balancer is using Amazon EC2 based health check - Application Load Balancer cannot use EC2 based health checks, so this option is incorrect.

Both the Auto Scaling group and Application Load Balancer are using ALB based health check - It is recommended to use ALB based health checks for both Auto Scaling group and Application Load Balancer. If both the Auto Scaling group and Application Load Balancer use ALB based health checks, then you will be able to avoid the scenario mentioned in the question.

Both the Auto Scaling group and Application Load Balancer are using Amazon EC2 based health check - Application Load Balancer cannot use EC2 based health checks, so this option is incorrect.

References:

https://docs.aws.amazon.com/autoscaling/ec2/userguide/ec2-auto-scaling-health-checks.html

https://docs.aws.amazon.com/autoscaling/ec2/userguide/health-checks-overview.html

https://docs.aws.amazon.com/autoscaling/ec2/userguide/as-add-elb-healthcheck.html

Domain

Design Resilient Architectures

Question 39Skipped

A pharmaceutical company is considering moving to AWS Cloud to accelerate the research and development process. Most of the daily workflows would be centered around running batch jobs on Amazon EC2 instances with storage on Amazon Elastic Block Store (Amazon EBS) volumes. The CTO is concerned about meeting HIPAA compliance norms for sensitive data stored on Amazon EBS.

Which of the following options outline the correct capabilities of an encrypted Amazon EBS volume? (Select three)

Correct selection

Any snapshot created from the volume is encrypted

Any snapshot created from the volume is NOT encrypted

Correct selection

Data at rest inside the volume is encrypted

Data moving between the volume and the instance is NOT encrypted

Data at rest inside the volume is NOT encrypted

Correct selection

Data moving between the volume and the instance is encrypted

Overall explanation

Correct options:

Data at rest inside the volume is encrypted

Any snapshot created from the volume is encrypted

Data moving between the volume and the instance is encrypted

Amazon Elastic Block Store (Amazon EBS) provides block-level storage volumes for use with Amazon EC2 instances. When you create an encrypted Amazon EBS volume and attach it to a supported instance type, data stored at rest on the volume, data moving between the volume and the instance, snapshots created from the volume and volumes created from those snapshots are all encrypted. It uses AWS Key Management Service (AWS KMS) customer master keys (CMK) when creating encrypted volumes and snapshots. Encryption operations occur on the servers that host Amazon EC2 instances, ensuring the security of both data-at-rest and data-in-transit between an instance and its attached Amazon EBS storage.

Therefore, the incorrect options are:

Data moving between the volume and the instance is NOT encrypted

Any snapshot created from the volume is NOT encrypted

Data at rest inside the volume is NOT encrypted

Reference:

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/EBSEncryption.html

Domain

Design Secure Architectures

Question 40Skipped

A silicon valley based healthcare startup uses AWS Cloud for its IT infrastructure. The startup stores patient health records on Amazon Simple Storage Service (Amazon S3). The engineering team needs to implement an archival solution based on Amazon S3 Glacier to enforce regulatory and compliance controls on data access.

As a solutions architect, which of the following solutions would you recommend?

Use Amazon S3 Glacier to store the sensitive archived data and then use an Amazon S3 Access Control List to enforce compliance controls

Use Amazon S3 Glacier to store the sensitive archived data and then use an Amazon S3 lifecycle policy to enforce compliance controls

Use Amazon S3 Glacier vault to store the sensitive archived data and then use an Amazon S3 Access Control List to enforce compliance controls

Correct answer

Use Amazon S3 Glacier vault to store the sensitive archived data and then use a vault lock policy to enforce compliance controls

Overall explanation

Correct option:

Use Amazon S3 Glacier vault to store the sensitive archived data and then use a vault lock policy to enforce compliance controls

Amazon S3 Glacier is a secure, durable, and extremely low-cost Amazon S3 cloud storage class for data archiving and long-term backup. It is designed to deliver 99.999999999% durability, and provide comprehensive security and compliance capabilities that can help meet even the most stringent regulatory requirements.

An Amazon S3 Glacier vault is a container for storing archives. When you create a vault, you specify a vault name and the AWS Region in which you want to create the vault. Amazon S3 Glacier Vault Lock allows you to easily deploy and enforce compliance controls for individual Amazon S3 Glacier vaults with a vault lock policy. You can specify controls such as “write once read many” (WORM) in a vault lock policy and lock the policy from future edits. Therefore, this is the correct option.

Incorrect options:

Use Amazon S3 Glacier to store the sensitive archived data and then use an Amazon S3 lifecycle policy to enforce compliance controls - You can use lifecycle policy to define actions you want Amazon S3 to take during an object's lifetime. For example, use a lifecycle policy to transition objects to another storage class, archive them, or delete them after a specified period. It cannot be used to enforce compliance controls. Therefore, this option is incorrect.

Use Amazon S3 Glacier vault to store the sensitive archived data and then use an Amazon S3 Access Control List to enforce compliance controls- Amazon S3 access control lists (ACLs) enable you to manage access to buckets and objects. It cannot be used to enforce compliance controls. Therefore, this option is incorrect.

Use Amazon S3 Glacier to store the sensitive archived data and then use an Amazon S3 Access Control List to enforce compliance controls - Amazon S3 access control lists (ACLs) enable you to manage access to buckets and objects. It cannot be used to enforce compliance controls. Therefore, this option is incorrect.

References:

https://docs.aws.amazon.com/amazonglacier/latest/dev/working-with-vaults.html

https://docs.aws.amazon.com/amazonglacier/latest/dev/vault-lock.html

https://docs.aws.amazon.com/AmazonS3/latest/user-guide/create-lifecycle.html

Domain

Design Secure Architectures

Question 41Skipped

An enterprise runs a critical Oracle database workload in its on-premises environment. The company now plans to replicate both existing records and continuous transactional changes to a managed Oracle environment in AWS. The target database will run on Amazon RDS for Oracle. Data transfer volume is expected to fluctuate throughout the day, and the team wants the solution to provision compute resources automatically based on actual workload requirements.

Which solution will meet these requirements?

Deploy the AWS DMS replication instance on Amazon EC2. Configure the instance with custom scripts that monitor CPU usage and resize the instance using EC2 Auto Scaling policies.

Use AWS Lambda to capture change data from the on-premises Oracle database. Trigger Lambda functions to write the updates to Amazon RDS for Oracle in real time.

Correct answer

Configure an AWS DMS Serverless replication task to synchronize historical and ongoing changes between the on-premises Oracle database and Amazon RDS for Oracle

Use AWS Glue to extract data from the on-premises Oracle database and write the output to Amazon RDS for Oracle. Configure Glue to run on demand when changes are detected

Overall explanation

Correct option:

Configure an AWS DMS Serverless replication task to synchronize historical and ongoing changes between the on-premises Oracle database and Amazon RDS for Oracle

AWS DMS Serverless is designed specifically for scenarios like this — where data replication workloads vary and automation is required. AWS DMS Serverless automatically provisions and scales compute resources based on workload demand. It supports ongoing replication, including change data capture (CDC), and is fully managed, reducing operational complexity. This solution avoids the need to guess capacity requirements or manually adjust instance sizes.

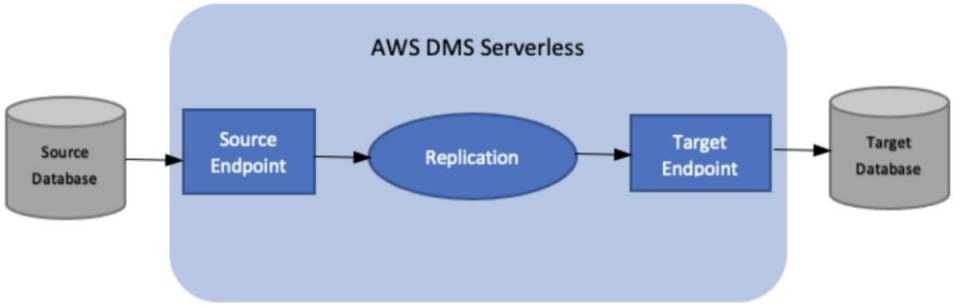
Working with AWS DMS Serverless

PDF RSS Focus mode

AWS DMS Serverless is a feature that provides automatic provisioning, scaling, built-in high availability, and a pay-for-use billing model, to increase operations agility and optimize your costs. The Serverless feature eliminates replication instance management tasks like capacity estimation, provisioning, cost optimization, and managing replication engine versions and patching.

With AWS DMS Serverless, similar to the current functionality of AWS DMS (referred to in this document as AWS DMS Standard), you create source and target connections using endpoints. After you create your source and target endpoints, you create a replication configuration, which includes configuration settings for the given replication. You can manage the replications by starting, stopping, modifying, or deleting them. Each replication has settings that you can configure according to the requirements of your database migration. You specify these settings using either a JSON file or the AWS DMS section of the AWS Management Console. For more information about replication settings, see Working with AWS DMS endpoints. After starting the replication, AWS DMS serverless connects to the source database and collects the database metadata to analyze the replication workload. Using this metadata, AWS DMS computes and provisions the required capacity and starts the data replication.

The following diagram shows the AWS DMS Serverless replication process.



via - https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Serverless.html

Incorrect options:

Use AWS Glue to extract data from the on-premises Oracle database and write the output to Amazon RDS for Oracle. Configure Glue to run on demand when changes are detected - AWS Glue is an ETL service, not designed for real-time or change data capture (CDC) replication from transactional databases. It works well for batch processing and data transformation pipelines, but not for continuously syncing ongoing changes from an Oracle source. Using it in this context would lead to inconsistent data and increased latency.

Deploy the AWS DMS replication instance on Amazon EC2. Configure the instance with custom scripts that monitor CPU usage and resize the instance using EC2 Auto Scaling policies. - Deploying DMS on EC2 and building custom Auto Scaling logic introduces unnecessary operational burden. EC2 Auto Scaling is not natively integrated with DMS replication tasks and cannot scale based on replication task load. This approach defeats the goal of minimizing management and may introduce performance instability due to lag in scaling triggers.

Use AWS Lambda to capture change data from the on-premises Oracle database. Trigger Lambda functions to write the updates to Amazon RDS for Oracle in real time. - AWS Lambda is not designed to process high-volume, stateful, transactional replication workloads from an Oracle database. Oracle does not natively trigger Lambda functions, and capturing CDC via Lambda would require complex, non-scalable workarounds. This option also lacks support for maintaining transactional consistency.

Reference:

https://docs.aws.amazon.com/dms/latest/userguide/CHAP_Serverless.html

Domain

Design High-Performing Architectures

Question 42Skipped

A leading media company wants to do an accelerated online migration of hundreds of terabytes of files from their on-premises data center to Amazon S3 and then establish a mechanism to access the migrated data for ongoing updates from the on-premises applications.

As a solutions architect, which of the following would you select as the MOST performant solution for the given use-case?

Correct answer

Use AWS DataSync to migrate existing data to Amazon S3 and then use File Gateway to retain access to the migrated data for ongoing updates from the on-premises applications

Use AWS DataSync to migrate existing data to Amazon S3 as well as access the Amazon S3 data for ongoing updates

Use File Gateway configuration of AWS Storage Gateway to migrate data to Amazon S3 and then use Amazon S3 Transfer Acceleration (Amazon S3TA) for ongoing updates from the on-premises applications

Use Amazon S3 Transfer Acceleration (Amazon S3TA) to migrate existing data to Amazon S3 and then use AWS DataSync for ongoing updates from the on-premises applications

Overall explanation

Correct option:

Use AWS DataSync to migrate existing data to Amazon S3 and then use File Gateway to retain access to the migrated data for ongoing updates from the on-premises applications

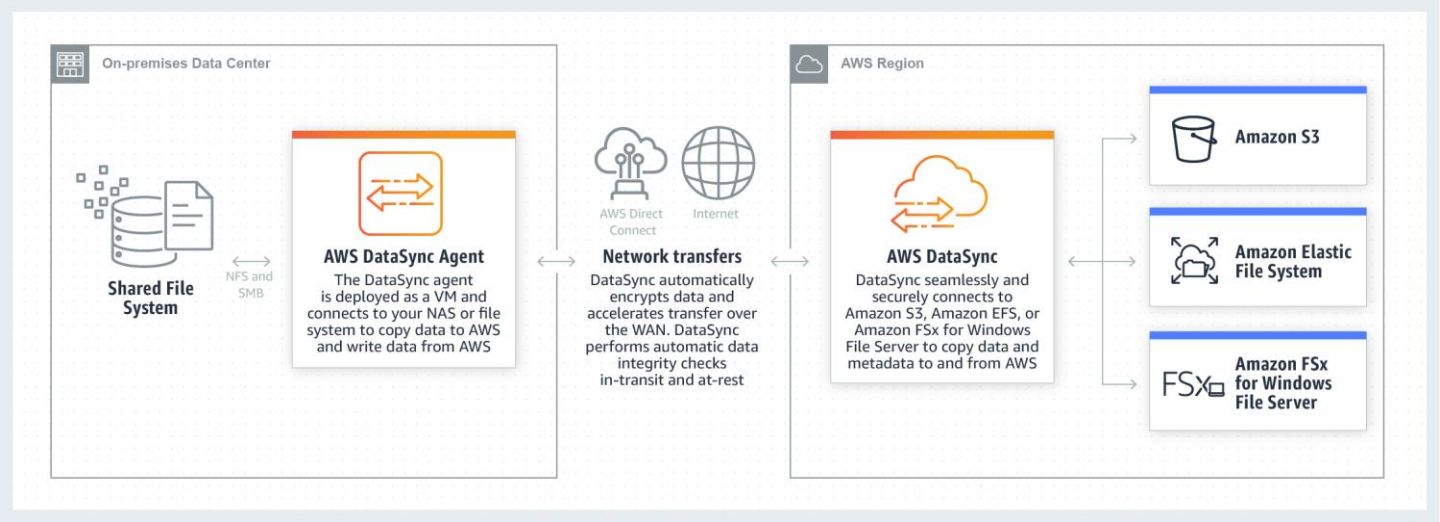
AWS DataSync is an online data transfer service that simplifies, automates, and accelerates copying large amounts of data to and from AWS storage services over the internet or AWS Direct Connect.

AWS DataSync fully automates and accelerates moving large active datasets to AWS, up to 10 times faster than command-line tools. It is natively integrated with Amazon S3, Amazon EFS, Amazon FSx for Windows File Server, Amazon CloudWatch, and AWS CloudTrail, which provides seamless and secure access to your storage services, as well as detailed monitoring of the transfer. DataSync uses a purpose-

built network protocol and scale-out architecture to transfer data. A single AWS DataSync agent is capable of saturating a 10 Gbps network link.

AWS DataSync fully automates the data transfer. It comes with retry and network resiliency mechanisms, network optimizations, built-in task scheduling, monitoring via the AWS DataSync API and Console, and Amazon CloudWatch metrics, events, and logs that provide granular visibility into the transfer process. AWS DataSync performs data integrity verification both during the transfer and at the end of the transfer.

How AWS DataSync Works:



via - https://aws.amazon.com/datasync/

AWS Storage Gateway is a hybrid cloud storage service that gives you on-premises access to virtually unlimited cloud storage. The service provides three different types of gateways – Tape Gateway, File Gateway, and Volume Gateway – that seamlessly connect on-premises applications to cloud storage, caching data locally for low-latency access. File gateway offers SMB or NFS-based access to data in Amazon S3 with local caching.

The combination of AWS DataSync and File Gateway is the correct solution. AWS DataSync enables you to automate and accelerate online data transfers to AWS storage services. File Gateway then provides your on-premises applications with low latency access to the migrated data.

Incorrect options:

Use AWS DataSync to migrate existing data to Amazon S3 as well as access the Amazon S3 data for ongoing updates - AWS DataSync is used to easily transfer data to and from AWS with up to 10x faster speeds. It is used to transfer data and cannot be used to facilitate ongoing updates to the migrated files from the on-premises applications.

Use File Gateway configuration of AWS Storage Gateway to migrate data to Amazon S3 and then use Amazon S3 Transfer Acceleration (Amazon S3TA) for ongoing updates from the on-premises applications - File Gateway can be used to move on-premises data to AWS Cloud, but it not an optimal solution for high volumes. Migration services such as AWS DataSync are best suited for this purpose.

Amazon S3 Transfer Acceleration cannot facilitate ongoing updates to the migrated files from the on-premises applications.

Use Amazon S3 Transfer Acceleration (Amazon S3TA) to migrate existing data to Amazon S3 and then use AWS DataSync for ongoing updates from the on-premises applications - If your application is already integrated with the Amazon S3 API, and you want higher throughput for transferring large files to Amazon S3, Amazon S3 Transfer Acceleration can be used. However AWS DataSync cannot be used to facilitate ongoing updates to the migrated files from the on-premises applications.

Reference:

https://aws.amazon.com/datasync/features/

Domain

Design High-Performing Architectures

Question 43Skipped

A retail company wants to establish encrypted network connectivity between its on-premises data center and AWS Cloud. The company wants to get the solution up and running in the fastest possible time and it should also support encryption in transit.

As a solutions architect, which of the following solutions would you suggest to the company?

Use AWS Direct Connect to establish encrypted network connectivity between the on-premises data center and AWS Cloud

Correct answer

Use AWS Site-to-Site VPN to establish encrypted network connectivity between the on-premises data center and AWS Cloud

Use AWS Secrets Manager to establish encrypted network connectivity between the on-premises data center and AWS Cloud

Use AWS Data Sync to establish encrypted network connectivity between the on-premises data center and AWS Cloud

Overall explanation

Correct option:

Use AWS Site-to-Site VPN to establish encrypted network connectivity between the on-premises data center and AWS Cloud

AWS Site-to-Site VPN enables you to securely connect your on-premises network or branch office site to your Amazon Virtual Private Cloud (Amazon VPC). You can securely extend your data center or branch office network to the cloud with an AWS Site-to-Site VPN connection. A VPC VPN Connection utilizes IPSec to establish encrypted network connectivity between your on-premises network and Amazon VPC

over the Internet. IPsec is a protocol suite for securing IP communications by authenticating and encrypting each IP packet in a data stream.

Incorrect options:

Use AWS Direct Connect to establish encrypted network connectivity between the on-premises data center and AWS Cloud - AWS Direct Connect lets you establish a dedicated network connection between your network and one of the AWS Direct Connect locations. Using industry-standard 802.1q VLANs, this dedicated connection can be partitioned into multiple virtual interfaces. AWS Direct Connect does not encrypt your traffic that is in transit. To encrypt the data in transit that traverses AWS Direct Connect, you must use the transit encryption options for that service. As AWS Direct Connect does not support encrypted network connectivity between an on-premises data center and AWS Cloud, therefore this option is incorrect.

Use AWS Data Sync to establish encrypted network connectivity between the on-premises data center and AWS Cloud - AWS DataSync makes it simple and fast to move large amounts of data online between on-premises storage and AWS. AWS DataSync eliminates or automatically handles many of these tasks, including scripting copy jobs, scheduling, and monitoring transfers, validating data, and optimizing network utilization. As AWS Data Sync cannot be used to establish network connectivity between an on-premises data center and AWS Cloud, therefore this option is incorrect.

Use AWS Secrets Manager to establish encrypted network connectivity between the on-premises data center and AWS Cloud - AWS Secrets Manager helps you protect secrets needed to access your applications, services, and IT resources. The service enables you to easily rotate, manage, and retrieve database credentials, API keys, and other secrets throughout their lifecycle. As AWS Secrets Manager cannot be used to establish network connectivity between an on-premises data center and AWS Cloud, therefore this option is incorrect.

References:

https://docs.aws.amazon.com/vpn/latest/s2svpn/internetwork-traffic-privacy.html

https://docs.aws.amazon.com/directconnect/latest/UserGuide/encryption-in-transit.html

Domain

Design Secure Architectures

Question 44Skipped

A digital media startup allows users to submit images through its web portal. These images are uploaded directly into an Amazon S3 bucket. On average, around 200 images are uploaded daily. The company wants to automatically generate a smaller preview version (thumbnail) of each new image and store the resulting thumbnails in a separate Amazon S3 bucket. The team prefers a design that is low-cost, requires minimal infrastructure management, and automatically reacts to new uploads.

Which solution will meet these requirements MOST cost-effectively?

Enable Amazon S3 Access Analyzer and configure it to call an AWS Lambda function whenever a new image is added. Use the Lambda function to generate and store the thumbnail

Correct answer

Configure the S3 bucket to send an event notification to an AWS Lambda function each time a new image is uploaded. Use the Lambda function to process the image, create a thumbnail, and store the thumbnail in the second S3 bucket

Set up a step-based processing workflow using AWS Glue jobs triggered on a regular interval. Use the jobs to scan the primary S3 bucket for new files and generate thumbnails for any that lack them. Write the thumbnails to a second S3 bucket

Deploy a containerized application on AWS Fargate that polls the S3 bucket every minute to detect new uploads. Configure the container to generate thumbnails and save them in the second bucket

Overall explanation

Correct option:

Configure the S3 bucket to send an event notification to an AWS Lambda function each time a new image is uploaded. Use the Lambda function to process the image, create a thumbnail, and store the thumbnail in the second S3 bucket

This approach follows a serverless, event-driven architecture that is highly cost-effective, scalable, and operationally efficient. The solution involves configuring Amazon S3 to send an event notification to an AWS Lambda function each time a new photo is uploaded to the primary S3 bucket. When triggered, the Lambda function processes the uploaded image, generates a thumbnail and stores it in a separate S3 bucket. This event-driven architecture ensures near real-time processing with minimal operational overhead. Because AWS Lambda is a fully managed, serverless compute service, it runs only when invoked and scales automatically, making it highly cost-effective for handling approximately 200 uploads per day. Additionally, S3 event notifications are natively supported and integrate seamlessly with Lambda, eliminating the need for persistent infrastructure or scheduled jobs. This solution provides scalability, low latency, and cost efficiency while ensuring clean separation of original images and their thumbnails.

Incorrect options:

Set up a step-based processing workflow using AWS Glue jobs triggered on a regular interval. Use the jobs to scan the primary S3 bucket for new files and generate thumbnails for any that lack them. Write the thumbnails to a second S3 bucket - AWS Glue is designed for ETL (Extract, Transform, Load) operations on structured and semi-structured data, not real-time image processing. Running Glue jobs on a schedule for this use case introduces unnecessary complexity and cost.

Deploy a containerized application on AWS Fargate that polls the S3 bucket every minute to detect new uploads. Configure the container to generate thumbnails and save them in the second bucket - While Fargate is serverless, continuously polling the S3 bucket is inefficient and incurs costs even when no new images are present. It also adds operational complexity compared to event-driven processing.

Enable Amazon S3 Access Analyzer and configure it to call an AWS Lambda function whenever a new image is added. Use the Lambda function to generate and store the thumbnail - S3 Access Analyzer is used for auditing and security analysis, not event-driven processing or triggering Lambda functions. It cannot be used to detect object uploads or initiate thumbnail generation.

References:

https://docs.aws.amazon.com/AmazonS3/latest/userguide/EventNotifications.html

https://docs.aws.amazon.com/lambda/latest/dg/with-s3.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/access-analyzer.html

Domain

Design High-Performing Architectures

Question 45Skipped

An online gaming company wants to block access to its application from specific countries; however, the company wants to allow its remote development team (from one of the blocked countries) to have access to the application. The application is deployed on Amazon EC2 instances running under an Application Load Balancer with AWS Web Application Firewall (AWS WAF).

As a solutions architect, which of the following solutions can be combined to address the given use-case? (Select two)

Create a deny rule for the blocked countries in the network access control list (network ACL) associated with each of the Amazon EC2 instances

Use Application Load Balancer geo match statement listing the countries that you want to block

Correct selection

Use AWS WAF geo match statement listing the countries that you want to block

Use Application Load Balancer IP set statement that specifies the IP addresses that you want to allow through

Correct selection

Use AWS WAF IP set statement that specifies the IP addresses that you want to allow through

Overall explanation

Correct options:

Use AWS WAF geo match statement listing the countries that you want to block

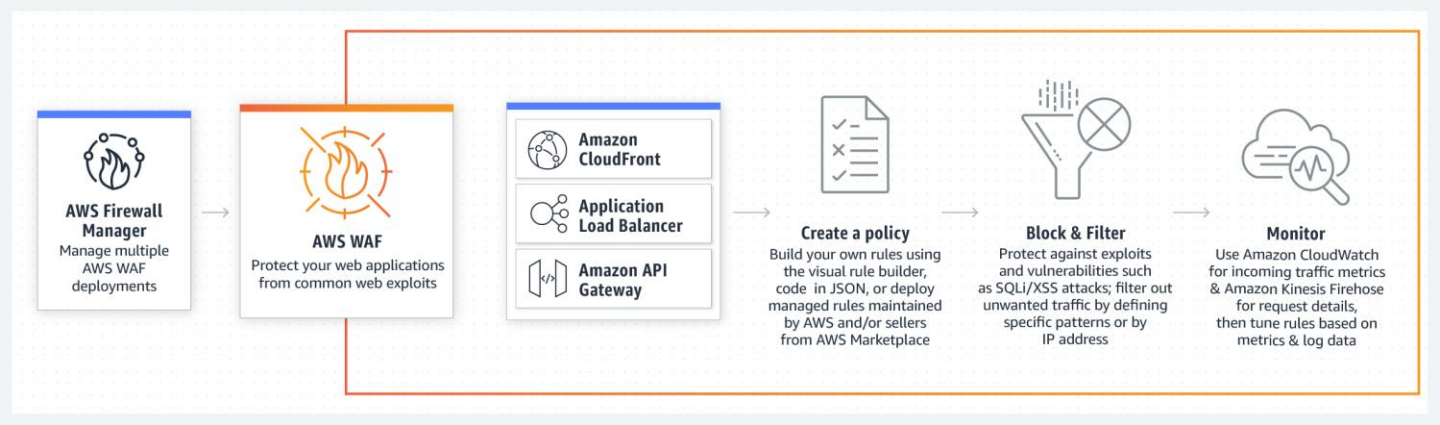
Use AWS WAF IP set statement that specifies the IP addresses that you want to allow through

AWS WAF is a web application firewall that helps protect your web applications or APIs against common web exploits that may affect availability, compromise security, or consume excessive resources. AWS

WAF gives you control over how traffic reaches your applications by enabling you to create security rules that block common attack patterns and rules that filter out specific traffic patterns you define.

You can deploy AWS WAF on Amazon CloudFront as part of your CDN solution, the Application Load Balancer that fronts your web servers or origin servers running on Amazon EC2, or Amazon API Gateway for your APIs.

AWS WAF - How it Works?:



via - https://aws.amazon.com/waf/

To block specific countries, you can create a AWS WAF geo match statement listing the countries that you want to block, and to allow traffic from IPs of the remote development team, you can create a WAF IP set statement that specifies the IP addresses that you want to allow through. You can combine the two rules as shown below:

Incorrect options:

Create a deny rule for the blocked countries in the network access control list (network ACL) associated with each of the Amazon EC2 instances - A network access control list (network ACL) is an optional layer of security for your VPC that acts as a firewall for controlling traffic in and out of one or more subnets. A network access control list (network ACL) does not have the capability to block traffic based on geographic match conditions.

Use Application Load Balancer geo match statement listing the countries that you want to block

Use Application Load Balancer IP set statement that specifies the IP addresses that you want to allow through

An Application Load Balancer operates at the request level (layer 7), routing traffic to targets – Amazon EC2 instances, containers, IP addresses, and AWS Lambda functions based on the content of the request. Ideal for advanced load balancing of HTTP and HTTPS traffic, Application Load Balancer provides advanced request routing targeted at delivery of modern application architectures, including microservices and container-based applications.

An Application Load Balancer cannot block or allow traffic based on geographic match conditions or IP based conditions. Both these options have been added as distractors.

References:

https://docs.aws.amazon.com/waf/latest/developerguide/waf-rule-statement-type-geo-match.html

https://aws.amazon.com/blogs/security/how-to-use-aws-waf-to-filter-incoming-traffic-from-embargoed-countries/

Domain

Design Secure Architectures

Question 46Skipped

A healthcare startup runs a lightweight reporting application on a single Amazon EC2 On-Demand instance. The application is designed to be stateless, fault-tolerant, and optimized for fast rendering of analytics dashboards. During major health events or news cycles, the team observes latency issues and occasional 5xx errors due to traffic spikes. To meet growing demand without over-provisioning resources during off-peak hours, the company wants to implement a cost-effective, scalable solution that ensures consistent performance even under unpredictable load.

Which approach best meets the requirements while minimizing costs?

Configure an Amazon EventBridge rule to monitor system-level metrics from the EC2 instance. Trigger a Lambda function to re-deploy the application in a different Availability Zone when CPU utilization exceeds 70%

Correct answer

Build an Amazon Machine Image (AMI) from the existing EC2 instance and configure a launch template. Create an Auto Scaling group using the launch template with Spot Instance pricing enabled. Attach an Application Load Balancer to distribute traffic across dynamically launched instances

Containerize the application using Amazon ECS with Fargate launch type. Deploy the container to a single Fargate task and set a CloudWatch alarm to increase memory and CPU allocation dynamically based on load

Clone the EC2 instance using an AMI and launch a second On-Demand instance. Register both instances with an Application Load Balancer to distribute incoming traffic evenly

Overall explanation

Correct option:

Build an Amazon Machine Image (AMI) from the existing EC2 instance and configure a launch template. Create an Auto Scaling group using the launch template with Spot Instance pricing enabled. Attach an Application Load Balancer to distribute traffic across dynamically launched instances

This approach leverages EC2 Auto Scaling to automatically scale the number of instances based on demand while using Spot Instances to minimize compute costs. The application's stateless design is ideal for this setup. The Application Load Balancer distributes traffic across healthy instances. Using a

launch template with an AMI ensures consistency across instances. This solution balances scalability with cost efficiency and aligns with AWS best practices for elastic, stateless workloads.

Use the following table to determine which fleet method to use.

Fleet method	When to use?	Use case
Amazon EC2 Auto Scaling	• You need multiple instances with either a single configuration or a mixed configuration. • You want to automate the lifecycle management of your instances.	Create an Auto Scaling group that manages the lifecycle of your instances while maintaining the desired number of instances. Supports horizontal scaling (adding more instances) between specified minimum and maximum limits.
EC2 Fleet	• You need multiple instances with either a single configuration or a mixed configuration. • You want to self-manage your instance lifecycle. • If you don't need auto scaling, we recommend that you use an `instant` type EC2 Fleet.	Create an `instant` fleet of both On-Demand Instances and Spot Instances in a single operation, with multiple launch specifications that vary by instance type, AMI, Availability Zone, or subnet. The Spot Instance allocation strategy defaults to `lowest-price` per unit, but we recommend changing it to `price-capacity-optimized`.
Spot Fleet	• We strongly discourage using Spot Fleet because it is based on a legacy API with no planned investment. • If you want to manage your instance lifecycle, rather use EC2 Fleet. • If you don't want to manage your instance lifecycle, rather use an Auto Scaling group.	Use Spot Fleet only if you need console support for a use case for when you would use EC2 Fleet.

via - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/which-fleet-method-to-use.html

Incorrect options:

Clone the EC2 instance using an AMI and launch a second On-Demand instance. Register both instances with an Application Load Balancer to distribute incoming traffic evenly - While this setup provides basic load distribution and supports horizontal scaling, using multiple On-Demand instances significantly increases operational costs, especially if the application only experiences brief traffic bursts. This design lacks elasticity and automation—instances run continuously even during idle periods. It's functional but not cost-effective compared to using Auto Scaling and Spot Instances.

Configure an Amazon EventBridge rule to monitor system-level metrics from the EC2 instance. Trigger a Lambda function to re-deploy the application in a different Availability Zone when CPU utilization exceeds 70% - Although EventBridge can be used to respond to metrics, re-deploying the

application to another Availability Zone in response to high CPU usage does not address scalability. This approach is reactive and inefficient, and it assumes a single-instance architecture. It also doesn't eliminate the source of the 5xx errors, which stem from overload—not instance location. Furthermore, there's no load balancing or auto-scaling involved.

Containerize the application using Amazon ECS with Fargate launch type. Deploy the container to a single Fargate task and set a CloudWatch alarm to increase memory and CPU allocation dynamically based on load - While Amazon ECS with Fargate offers a serverless way to run containers without managing infrastructure, this approach is not inherently scalable when only one task is deployed. Even with increased CPU and memory allocation triggered by a CloudWatch alarm, a single Fargate task will still remain a bottleneck under heavy load. ECS services are designed to scale horizontally by adding more tasks, not by resizing a single task. Additionally, there is no load balancing mechanism included in this setup, and it doesn't address traffic distribution or redundancy. This configuration underutilizes the scalable nature of ECS and fails to deliver the needed elasticity.

References:

https://docs.aws.amazon.com/autoscaling/ec2/userguide/launch-template-spot-instances.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/which-fleet-method-to-use.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/launch-instances-from-launch-template.html#launch-templates-spot-fleet

https://docs.aws.amazon.com/AmazonECS/latest/developerguide/AWS_Fargate.html

Domain

Design Resilient Architectures

Question 47Skipped

The infrastructure team at a company maintains 5 different VPCs (let's call these VPCs A, B, C, D, E) for resource isolation. Due to the changed organizational structure, the team wants to interconnect all VPCs together. To facilitate this, the team has set up VPC peering connection between VPC A and all other VPCs in a hub and spoke model with VPC A at the center. However, the team has still failed to establish connectivity between all VPCs.

As a solutions architect, which of the following would you recommend as the MOST resource-efficient and scalable solution?

Establish VPC peering connections between all VPCs

Correct answer

Use AWS transit gateway to interconnect the VPCs

Use a VPC endpoint to interconnect the VPCs

Use an internet gateway to interconnect the VPCs

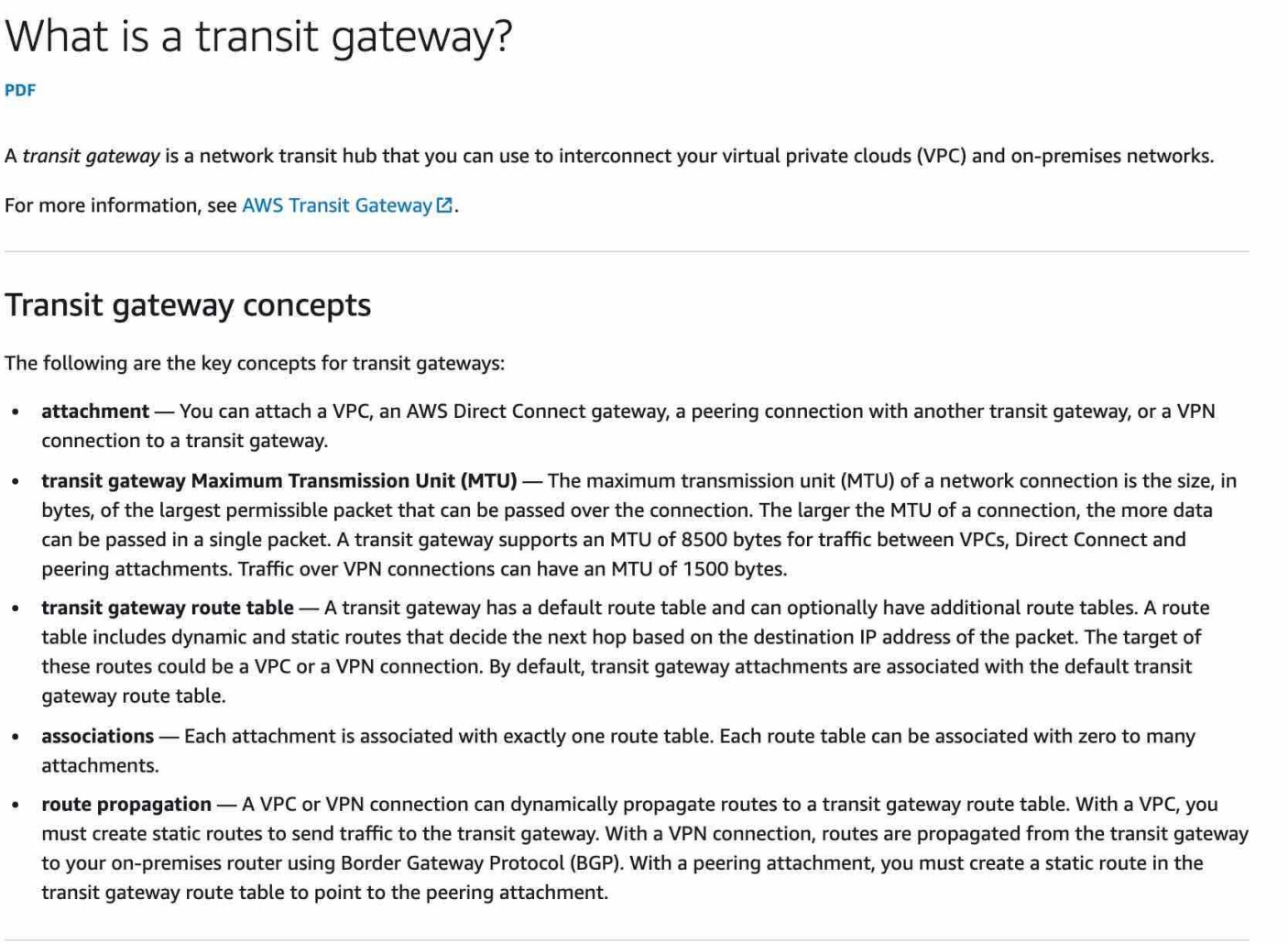
Overall explanation

Correct option:

Use AWS transit gateway to interconnect the VPCs

An AWS transit gateway is a network transit hub that you can use to interconnect your virtual private clouds (VPC) and on-premises networks.

AWS Transit Gateway Overview:



What is a transit gateway?

PDF

A *transit gateway* is a network transit hub that you can use to interconnect your virtual private clouds (VPC) and on-premises networks.

For more information, see AWS Transit Gateway.

Transit gateway concepts

The following are the key concepts for transit gateways:

- **attachment** — You can attach a VPC, an AWS Direct Connect gateway, a peering connection with another transit gateway, or a VPN connection to a transit gateway.
- **transit gateway Maximum Transmission Unit (MTU)** — The maximum transmission unit (MTU) of a network connection is the size, in bytes, of the largest permissible packet that can be passed over the connection. The larger the MTU of a connection, the more data can be passed in a single packet. A transit gateway supports an MTU of 8500 bytes for traffic between VPCs, Direct Connect and peering attachments. Traffic over VPN connections can have an MTU of 1500 bytes.
- **transit gateway route table** — A transit gateway has a default route table and can optionally have additional route tables. A route table includes dynamic and static routes that decide the next hop based on the destination IP address of the packet. The target of these routes could be a VPC or a VPN connection. By default, transit gateway attachments are associated with the default transit gateway route table.
- **associations** — Each attachment is associated with exactly one route table. Each route table can be associated with zero to many attachments.
- **route propagation** — A VPC or VPN connection can dynamically propagate routes to a transit gateway route table. With a VPC, you must create static routes to send traffic to the transit gateway. With a VPN connection, routes are propagated from the transit gateway to your on-premises router using Border Gateway Protocol (BGP). With a peering attachment, you must create a static route in the transit gateway route table to point to the peering attachment.

via - https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html

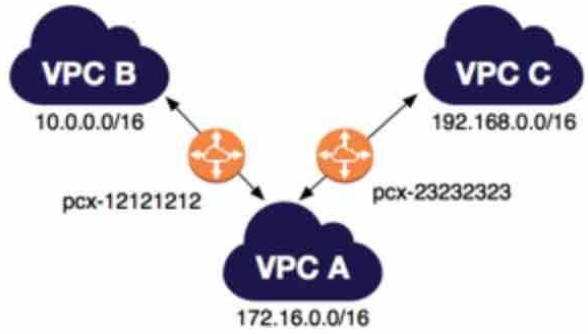
A VPC peering connection is a networking connection between two VPCs that enables you to route traffic between them using private IPv4 addresses or IPv6 addresses. Transitive Peering does not work for VPC peering connections. So, if you have a VPC peering connection between VPC A and VPC B (pcx-aaaabbbb), and between VPC A and VPC C (pcx-aaaacccc). Then, there is no VPC peering connection between VPC B and VPC C. Instead of using VPC peering, you can use an AWS Transit Gateway that acts as a network transit hub, to interconnect your VPCs or connect your VPCs with on-premises networks. Therefore this is the correct option.

VPC Peering Connections Overview:

Multiple VPC peering connections

A VPC peering connection is a one to one relationship between two VPCs. You can create multiple VPC peering connections for each VPC that you own, but transitive peering relationships are not supported. You do not have any peering relationship with VPCs that your VPC is not directly peered with.

The following diagram is an example of one VPC peered to two different VPCs. There are two VPC peering connections: VPC A is peered with both VPC B and VPC C. VPC B and VPC C are not peered, and you cannot use VPC A as a transit point for peering between VPC B and VPC C. If you want to enable routing of traffic between VPC B and VPC C, you must create a unique VPC peering connection between them.



via - https://docs.aws.amazon.com/vpc/latest/peering/vpc-peering-basics.html

Incorrect options:

Use an internet gateway to interconnect the VPCs - An internet gateway is a horizontally scaled, redundant, and highly available VPC component that allows communication between instances in your VPC and the internet. It, therefore, imposes no availability risks or bandwidth constraints on your network traffic. You cannot use an internet gateway to interconnect your VPCs and on-premises networks, hence this option is incorrect.

Use a VPC endpoint to interconnect the VPCs - A VPC endpoint enables you to privately connect your VPC to supported AWS services and VPC endpoint services powered by AWS PrivateLink without requiring an internet gateway, NAT device, VPN connection, or AWS Direct Connect connection. You cannot use a VPC endpoint to interconnect your VPCs and on-premises networks, hence this option is incorrect.

Establish VPC peering connections between all VPCs - Establishing VPC peering between all VPCs is an inelegant and clumsy way to establish connectivity between all VPCs. Instead, you should use a Transit Gateway that acts as a network transit hub to interconnect your VPCs and on-premises networks.

References:

https://docs.aws.amazon.com/vpc/latest/peering/what-is-vpc-peering.html

https://docs.aws.amazon.com/vpc/latest/tgw/what-is-transit-gateway.html

Domain

Design Secure Architectures

Question 48Skipped

A streaming service provider collects user experience feedback through embedded feedback forms in their mobile and web apps. Feedback submissions frequently spike to thousands per hour during content launches or service outages. Currently, the feedback is sent via email to the operations team for manual review. The company now wants to automate feedback collection and sentiment analysis so that insights can be generated quickly and stored for a full year for trend analysis.

Which solution provides the most scalable and automated approach to meet these requirements?

Build a web service on Amazon EC2 that receives feedback data and stores each record in a DynamoDB table. Use the EC2 application to invoke Amazon Comprehend for sentiment detection and write results to a second table. Apply a TTL of 365 days to each table

Use Amazon EventBridge to capture feedback events and forward them to an AWS Step Functions workflow. The workflow invokes Lambda functions for validation, calls Amazon Transcribe to convert the text to audio for archival, and stores the results in an Amazon RDS database. Configure a lifecycle policy to remove records after 12 months

Correct answer

Design a RESTful API with Amazon API Gateway that forwards incoming feedback data to an Amazon SQS queue. Set up an AWS Lambda function to process the queue messages, analyze sentiment using Amazon Comprehend, and store results in a DynamoDB table with a 365-day TTL configured on each item

Route all feedback submissions through Amazon Kinesis Data Streams. Use an AWS Lambda consumer to batch process incoming records, invoke Amazon Translate to detect language and convert input to English, and save the processed content in an Amazon OpenSearch Service index. Configure OpenSearch Index State Management (ISM) policies to delete documents after 12 months

Overall explanation

Correct option:

Design a RESTful API with Amazon API Gateway that forwards incoming feedback data to an Amazon SQS queue. Set up an AWS Lambda function to process the queue messages, analyze sentiment using Amazon Comprehend, and store results in a DynamoDB table with a 365-day TTL configured on each item

This architecture decouples feedback ingestion from processing by using Amazon SQS as a buffer for potentially high volumes of incoming data. API Gateway handles secure and scalable API access, while

Lambda automatically scales to process feedback messages asynchronously. Amazon Comprehend is specifically designed for sentiment analysis on text input. Storing results in DynamoDB offers scalable, low-latency access to structured insights, and Time to Live (TTL) ensures automatic data expiry after one year. This is a serverless, fully managed solution that scales efficiently with bursts and long-term retention.

Incorrect options:

Route all feedback submissions through Amazon Kinesis Data Streams. Use an AWS Lambda consumer to batch process incoming records, invoke Amazon Translate to detect language and convert input to English, and save the processed content in an Amazon OpenSearch Service index. Configure OpenSearch Index State Management (ISM) policies to delete documents after 12 months - This option introduces Kinesis Data Streams to handle high-throughput ingestion, which is appropriate for large-scale, real-time data capture. However, the use of Amazon Translate is misaligned with the core requirement, which is sentiment analysis, not language translation. Additionally, storing feedback data in Amazon OpenSearch Service can enable full-text search and visualization, but it adds significant operational overhead, especially for a solution that doesn't require search capabilities. Furthermore, OpenSearch is less cost-effective and scalable for basic structured storage when compared to DynamoDB for this use case.

Use Amazon EventBridge to capture feedback events and forward them to an AWS Step Functions workflow. The workflow invokes Lambda functions for validation, calls Amazon Transcribe to convert the text to audio for archival, and stores the results in an Amazon RDS database. Configure a lifecycle policy to remove records after 12 months - This option introduces unnecessary complexity and the wrong tools for the job. Amazon Transcribe is used to convert speech to text, not the other way around, and has no benefit in archiving text-based user feedback. Storing the output in Amazon RDS increases management burden and doesn’t scale as well as DynamoDB for high-velocity, write-heavy workloads. Although EventBridge and Step Functions are powerful for orchestrating workflows, this particular implementation misuses services and introduces overhead without delivering meaningful improvements.

Build a web service on Amazon EC2 that receives feedback data and stores each record in a DynamoDB table. Use the EC2 application to invoke Amazon Comprehend for sentiment detection and write results to a second table. Apply a TTL of 365 days to each table - Although technically functional, this solution uses Amazon EC2 for feedback ingestion and sentiment analysis, which introduces operational overhead, limits scalability, and requires provisioning, patching, and scaling of EC2 instances. This is not a serverless or event-driven architecture, and the use of a second DynamoDB table adds unnecessary complexity. While Amazon Comprehend is correctly applied, the solution lacks elasticity for sudden spikes and isn't the most cost-effective or scalable approach.

References:

https://docs.aws.amazon.com/comprehend/latest/dg/what-is.html

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html

https://docs.aws.amazon.com/opensearch-service/latest/developerguide/ism.html

https://docs.aws.amazon.com/transcribe/latest/dg/what-is.html

https://docs.aws.amazon.com/translate/latest/dg/what-is.html

Domain

Design High-Performing Architectures

Question 49Skipped

A retail company maintains an AWS Direct Connect connection to AWS and has recently migrated its data warehouse to AWS. The data analysts at the company query the data warehouse using a visualization tool. The average size of a query returned by the data warehouse is 60 megabytes and the query responses returned by the data warehouse are not cached in the visualization tool. Each webpage returned by the visualization tool is approximately 600 kilobytes.

Which of the following options offers the LOWEST data transfer egress cost for the company?

Deploy the visualization tool in the same AWS region as the data warehouse. Access the visualization tool over the internet at a location in the same region

Deploy the visualization tool on-premises. Query the data warehouse directly over an AWS Direct Connect connection at a location in the same AWS region

Correct answer

Deploy the visualization tool in the same AWS region as the data warehouse. Access the visualization tool over a Direct Connect connection at a location in the same region

Deploy the visualization tool on-premises. Query the data warehouse over the internet at a location in the same AWS region

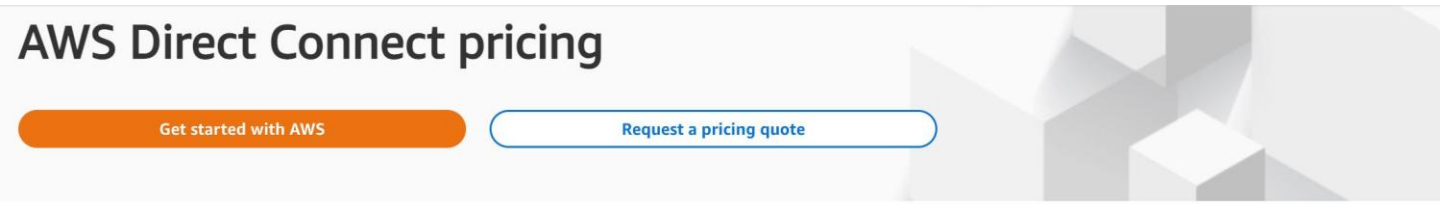
Overall explanation

Correct option:

Deploy the visualization tool in the same AWS region as the data warehouse. Access the visualization tool over a Direct Connect connection at a location in the same region

AWS Direct Connect is a networking service that provides an alternative to using the internet to connect to AWS. Using AWS Direct Connect, data that would have previously been transported over the internet is delivered through a private network connection between your on-premises data center and AWS.

For the given use case, the main pricing parameter while using the AWS Direct Connect connection is the Data Transfer Out (DTO) from AWS to the on-premises data center. DTO refers to the cumulative network traffic that is sent through AWS Direct Connect to destinations outside of AWS. This is charged per gigabyte (GB), and unlike capacity measurements, DTO refers to the amount of data transferred, not the speed.



AWS Direct Connect is a cloud service that links your network directly to AWS to deliver consistent, low-latency performance. With AWS Direct Connect, you pay only for what you use and there is no minimum fee. There are no setup charges, and you may cancel at any time. However, services provided by your AWS Direct Connect Delivery Partners or other local service provider may have other terms that apply.

Once you have linked your locations to AWS Direct Connect, you can send data between them using SiteLink. When using SiteLink, data travels over the shortest path between locations. The SiteLink feature is off by default and can be turned on or off at any time.

Pricing components

When connecting to resources running in any AWS Region (such as an Amazon Virtual Private Cloud or AWS Transit Gateway), there are three factors that determine pricing: capacity, port hours, and data transfer out (DTO).

Capacity is the maximum rate that data can be transferred through a network connection. The capacity of AWS Direct Connect connections are measured in megabit per second (Mbps) or gigabit per second (Gbps). One gigabit per second, or 1 Gbps, is equal to 1,000 megabits per second (1,000 Mbps).

Port hours measure the time that a port is provisioned for your use with AWS, or an AWS Direct Connect Delivery Partner's, networking equipment inside an AWS Direct Connect location. Even when no data is passing through the port, you are charged for port hours. Port hour pricing is determined by the connection type: dedicated or hosted.

> **Dedicated connections** are physical connections between your network port and an AWS network port inside an AWS Direct Connect location. Dedicated port hours are billed as long as that port is provisioned for your use. You request a dedicated connection through the AWS Direct Connect section of the AWS Management Console.
>
> **Hosted connections** are logical connections that an AWS Direct Connect Delivery Partner provisions on your behalf. When using hosted connections, you connect to the AWS network using one of the partner's ports. You request a hosted connection by contacting an AWS Direct Connect Delivery Partner directly.

Data transfer out (DTO) refers to the cumulative network traffic that is sent through AWS Direct Connect to destinations outside of AWS. This is charged per gigabyte (GB), and unlike capacity measurements, DTO refers to the amount of data transferred, not the speed. When calculating DTO, exact pricing depends on the AWS Region or AWS Local Zone, and the AWS Direct Connect location, you are using (see tables below).

Data transfer in refers to network traffic that is sent into AWS from outside, over AWS Direct Connect. AWS Direct Connect data transfer in is charged at 0.00 USD per GB in all locations.

Except as otherwise noted, our prices are exclusive of applicable taxes and duties, including VAT and applicable sales tax. For customers with a Japanese billing address, use of the Asia Pacific (Tokyo) Region is subject to Japanese Consumption Tax. Learn more.

via - https://aws.amazon.com/directconnect/pricing/

Each query response is 60 megabytes in size and each webpage for the visualization tool is 600 kilobytes in size. If you deploy the visualization tool in the same AWS region as the data warehouse, then you only need to pay for the 600 kilobytes of DTO charges for the webpage. Therefore this option is correct.

However, if you deploy the visualization tool on-premises, then you need to pay for the 60 MB of DTO charges for the query response from the data warehouse to the visualization tool.

Incorrect options:

Deploy the visualization tool in the same AWS region as the data warehouse. Access the visualization tool over the internet at a location in the same region

Deploy the visualization tool on-premises. Query the data warehouse over the internet at a location in the same AWS region

Data transfer pricing over AWS Direct Connect is lower than data transfer pricing over the internet, so both of these options are incorrect.

Deploy the visualization tool on-premises. Query the data warehouse directly over an AWS Direct Connect connection at a location in the same AWS region - As mentioned in the explanation above, if you deploy the visualization tool on-premises, then you need to pay for the 60 megabytes of DTO charges for the query response from the data warehouse to the visualization tool. So this option is incorrect.

References:

https://aws.amazon.com/directconnect/pricing/

https://aws.amazon.com/getting-started/hands-on/connect-data-center-to-aws/services-costs/

https://aws.amazon.com/directconnect/faqs/

Domain

Design Cost-Optimized Architectures

Question 50Skipped

A Customer relationship management (CRM) application is facing user experience issues with users reporting frequent sign-in requests from the application. The application is currently hosted on multiple Amazon EC2 instances behind an Application Load Balancer. The engineering team has identified the root cause as unhealthy servers causing session data to be lost. The team would like to implement a distributed in-memory cache-based session management solution.

As a solutions architect, which of the following solutions would you recommend?

Use Application Load Balancer sticky sessions

Correct answer

Use Amazon Elasticache for distributed in-memory cache based session management

Use Amazon DynamoDB for distributed in-memory cache based session management

Use Amazon RDS for distributed in-memory cache based session management

Overall explanation

Correct option:

Use Amazon Elasticache for distributed in-memory cache based session management

Amazon ElastiCache can be used as a distributed in-memory cache for session management. Amazon ElastiCache allows you to seamlessly set up, run, and scale popular open-Source compatible in-memory data stores in the cloud. Session stores can be set up using both Memcached or Redis for ElastiCache.

Amazon ElastiCache for Redis is a great choice for real-time transactional and analytical processing use cases such as caching, chat/messaging, gaming leaderboards, geospatial, machine learning, media streaming, queues, real-time analytics, and session store.

Amazon ElastiCache for Memcached is a Memcached-compatible in-memory key-value store service that can be used as a cache or a data store. Session stores are easy to create with Amazon ElastiCache for Memcached.

How Amazon ElastiCache Works:

via - https://aws.amazon.com/elasticache/

Incorrect options:

Use Amazon RDS for distributed in-memory cache based session management - Amazon Relational Database Service (Amazon RDS) makes it easy to set up, operate, and scale a relational database in the cloud. It cannot be used as a distributed in-memory cache for session management, hence this option is incorrect.

Use Amazon DynamoDB for distributed in-memory cache based session management - Amazon DynamoDB is a key-value and document database that delivers single-digit millisecond performance at any scale. Amazon DynamoDB is a NoSQL database and is not the right fit for a distributed in-memory cache-based session management solution.

Use Application Load Balancer sticky sessions - Although sticky sessions enable each user to interact with one server and one server only, however, in case of an unhealthy server, all the session data is gone as well. Therefore Amazon Elasticache powered distributed in-memory cache-based session management is a better solution.

References:

https://aws.amazon.com/getting-started/hands-on/building-fast-session-caching-with-amazon-elasticache-for-redis/

https://aws.amazon.com/elasticache/

Domain

Design High-Performing Architectures

Question 51Skipped

A global e-commerce platform currently operates its order processing system in a single on-premises data center located in Europe. As the company grows its customer base across Asia and North America, it plans to deploy the application across multiple AWS Regions to improve availability and reduce latency. The company requires that updates to the central order database be completed in under one second with global consistency. The application layer will be deployed separately in each Region, but the order management data must remain centrally managed and globally synchronized.

Which solution should a solutions architect recommend to meet these requirements?

Use Amazon Aurora database with MySQL engine, and configure read-only nodes in other Regions to handle local traffic while routing all write operations to the central Region

Use Amazon Neptune to store tracking updates as graph data. Deploy clusters in each Region and replicate changes using custom-built Lambda functions and Amazon SQS.

Correct answer

Migrate the order data to Amazon DynamoDB and create a global table. Deploy the application in each Region and connect to the local DynamoDB replica for low-latency access

Use Amazon RDS for MySQL with a cross-Region read replica. Route all writes to the primary Region and use read replicas for local access in other Regions

Overall explanation

Correct option:

Migrate the order data to Amazon DynamoDB and create a global table. Deploy the application in each Region and connect to the local DynamoDB replica for low-latency access

Amazon DynamoDB global tables are a fully managed, multi-Region, multi-writer database solution designed to deliver low-latency performance and high availability for globally distributed applications. In this architecture, a DynamoDB table is created in one Region and replicated automatically to other specified AWS Regions. Each replica behaves as a fully functional table, capable of handling both reads and writes locally. This setup allows regional deployments of the application to interact with the local replica of the database, minimizing cross-Region latency and avoiding the need to route write requests to a single centralized Region. Updates made to any Region's table are asynchronously propagated to all other Regions with typical replication latency under 1 second, satisfying the application's performance requirement.

This solution also eliminates the need to manage complex replication logic, ensures high resilience against Regional failures, and provides seamless multi-active availability—allowing each Region to continue operating independently even if another Region experiences disruption.

Incorrect options:

Use Amazon RDS for MySQL with a cross-Region read replica. Route all writes to the primary Region and use read replicas for local access in other Regions - Using RDS MySQL with read replicas in different Regions allows users to perform read queries from geographically closer endpoints. However, all

write operations must be handled by the single primary instance located in the original Region, which violates the latency requirement for updates. Furthermore, replication across Regions is asynchronous, which introduces delay and potential data consistency issues. This approach does not fulfill the global consistency and low-latency write requirement.

Use Amazon Neptune to store tracking updates as graph data. Deploy clusters in each Region and replicate changes using custom-built Lambda functions and Amazon SQS. - Amazon Neptune is a managed graph database, suitable for highly connected datasets such as social graphs or fraud detection. It does not support multi-Region deployment out of the box, and using custom replication via Lambda and SQS introduces complexity, increased latency, and potential consistency issues. This solution would not meet the company's needs for a low-latency, replicated write path across Regions.

Use Amazon Aurora database with MySQL engine, and configure read-only nodes in other Regions to handle local traffic while routing all write operations to the central Region - This architecture enables low-latency reads from secondary Regions by using Aurora Global Database read-only replicas. However, it restricts write operations to the primary Region only, meaning any update to the order database must still be processed in the central Region. This introduces latency for global users and fails to meet the requirement for sub-second update performance from all Regions. Additionally, the system doesn't support multi-Region active-active writes or built-in conflict resolution.

References:

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html

https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/USER_ReadRepl.XRgn.html

Domain

Design High-Performing Architectures

Question 52Skipped

Computer vision researchers at a university are trying to optimize the I/O bound processes for a proprietary algorithm running on Amazon EC2 instances. The ideal storage would facilitate high-performance IOPS when doing file processing in a temporary storage space before uploading the results back into Amazon S3.

As a solutions architect, which of the following AWS storage options would you recommend as the MOST performant as well as cost-optimal?

Use Amazon EC2 instances with Amazon EBS General Purpose SSD (gp2) as the storage option

Correct answer

Use Amazon EC2 instances with Instance Store as the storage option

Use Amazon EC2 instances with Amazon EBS Throughput Optimized HDD (st1) as the storage option

Use Amazon EC2 instances with Amazon EBS Provisioned IOPS SSD (io1) as the storage option

Overall explanation

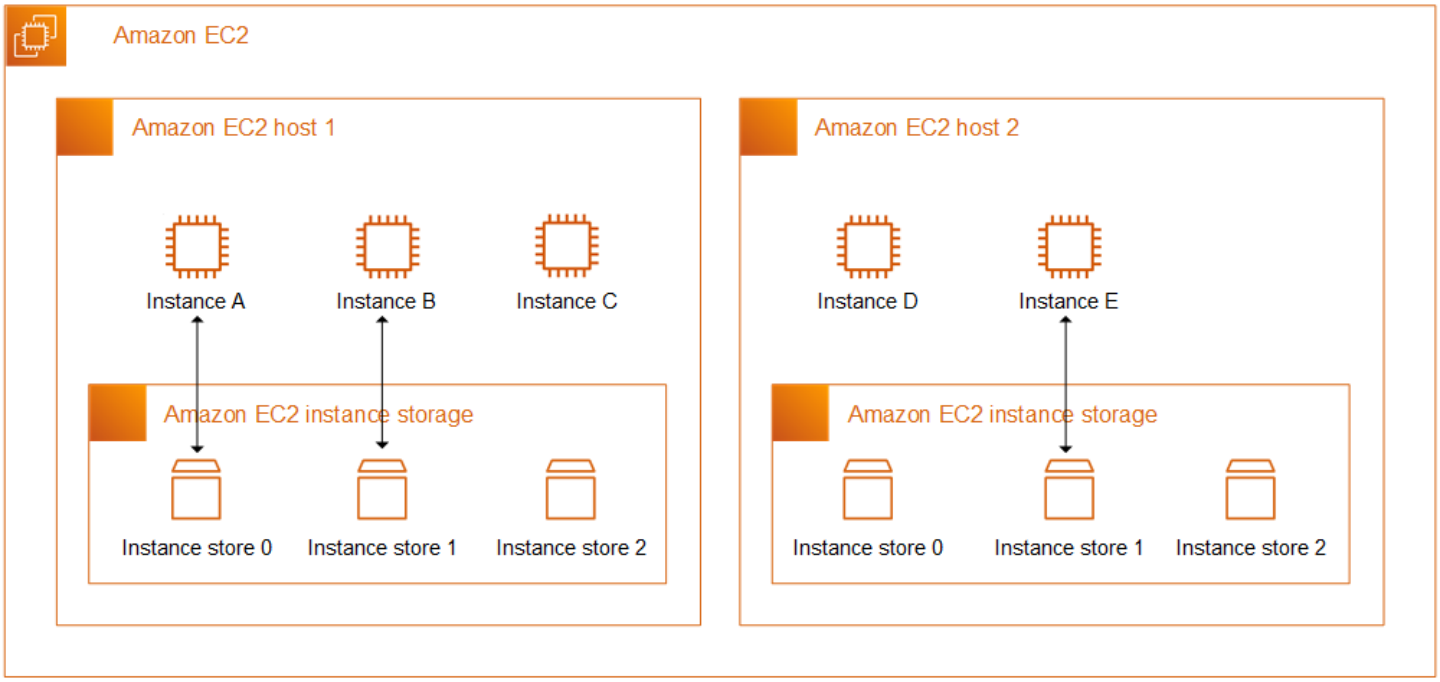
Correct option:

Use Amazon EC2 instances with Instance Store as the storage option

An instance store provides temporary block-level storage for your instance. This storage is located on disks that are physically attached to the host computer. Instance store is ideal for the temporary storage of information that changes frequently, such as buffers, caches, scratch data, and other temporary content, or for data that is replicated across a fleet of instances, such as a load-balanced pool of web servers. Some instance types use NVMe or SATA-based solid-state drives (SSD) to deliver high random I/O performance. This is a good option when you need storage with very low latency, but you don't need the data to persist when the instance terminates or you can take advantage of fault-tolerant architectures.

As Instance Store delivers high random I/O performance, it can act as a temporary storage space, and these volumes are included as part of the instance's usage cost, therefore this is the correct option.

Amazon EC2 Instance Store:



via - https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/InstanceStorage.html

Incorrect options:

Use Amazon EC2 instances with Amazon EBS General Purpose SSD (gp2) as the storage option - General Purpose SSD (gp2) volumes offer cost-effective storage that is ideal for a broad range of workloads. These volumes deliver single-digit millisecond latencies and the ability to burst to 3,000 IOPS for extended periods. Between a minimum of 100 IOPS (at 33.33 GiB and below) and a maximum of 16,000 IOPS (at 5,334 GiB and above), baseline performance scales linearly at 3 IOPS per GiB of volume size. AWS designs gp2 volumes to deliver its provisioned performance 99% of the time. A gp2 volume can range in size from 1 GiB to 16 TiB. Amazon EBS gp2 is persistent storage and costlier than Instance Stores

(the cost of the storage volume is in addition to that of the Amazon EC2 instance), therefore this option is not correct.

Use Amazon EC2 instances with Amazon EBS Provisioned IOPS SSD (io1) as the storage option - Provisioned IOPS SSD (io1) volumes are designed to meet the needs of I/O-intensive workloads, particularly database workloads, that are sensitive to storage performance and consistency. Unlike gp2, which uses a bucket and credit model to calculate performance, an io1 volume allows you to specify a consistent IOPS rate when you create the volume, and Amazon EBS delivers the provisioned performance 99.9 percent of the time. Amazon EBS io1 is persistent storage and costlier than Instance Stores (the cost of the storage volume is in addition to that of the Amazon EC2 instance), therefore this option is not correct.

Use Amazon EC2 instances with Amazon EBS Throughput Optimized HDD (st1) as the storage option - Throughput Optimized HDD (st1) are low-cost HDD volumes designed for frequently accessed, throughput-intensive workloads such as Big data and Data warehouses. Amazon EBS st1 is persistent storage and costlier than Instance Stores (the cost of the storage volume is in addition to that of the Amazon EC2 instance), therefore this option is not correct.

References:

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/AmazonEBS.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/InstanceStorage.html

https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html

Domain

Design Cost-Optimized Architectures

Question 53Skipped

A retail enterprise is expanding its hybrid IT infrastructure and plans to securely connect its on-premises corporate network to its AWS environment. The company wants to ensure that all data exchanged between on-premises systems and AWS is encrypted at both the network and session layers. Additionally, the solution must incorporate granular security controls that restrict unnecessary or unauthorized access between the cloud and on-premises environments. A solutions architect must recommend a scalable and secure approach that supports these goals.

Which solution best meets these requirements?

Use AWS Client VPN to allow corporate users to connect to the VPC individually. Manage access controls with security groups and IAM policies

Correct answer

Set up AWS Site-to-Site VPN to connect the on-premises network to the AWS VPC. Use route tables to manage traffic flow and configure security groups and network ACLs to allow only authorized communication between systems

Establish a dedicated AWS Direct Connect connection between the corporate network and AWS. Configure VPC route tables to control traffic flow and use security groups and network ACLs to restrict access as needed

Set up a bastion host in a public subnet of the VPC to provide SSH-based access to AWS resources from the corporate network. Use security groups to control access

Overall explanation

Correct options:

Set up AWS Site-to-Site VPN to connect the on-premises network to the AWS VPC. Use route tables to manage traffic flow and configure security groups and network ACLs to allow only authorized communication between systems

AWS Site-to-Site VPN uses IPsec tunnels, which provide encryption at the network layer, ensuring that traffic is securely transmitted between the on-premises network and the VPC. By applying security groups and network ACLs, fine-grained traffic control is possible. Additionally, session-layer encryption (e.g., HTTPS, TLS) can be layered on top, satisfying both encryption requirements with minimal overhead. This is the most cost-effective and secure solution for the given requirements.

Incorrect Options:

Establish a dedicated AWS Direct Connect connection between the corporate network and AWS. Configure VPC route tables to control traffic flow and use security groups and network ACLs to restrict access as needed - While AWS Direct Connect provides a dedicated private connection with high bandwidth and low latency, it does not encrypt traffic at the network layer by default. Additional technologies (e.g., MACsec or an overlay VPN) are needed to secure the connection. Furthermore, session-layer encryption is not inherently addressed by Direct Connect. Without additional configuration, this approach does not meet the encryption requirements.

Use AWS Client VPN to allow corporate users to connect to the VPC individually. Manage access controls with security groups and IAM policies - While AWS Client VPN does provide encrypted connectivity over the internet and can enable individual client devices to access AWS resources securely, it is not designed for network-to-network connectivity. This solution fails to meet the requirement of establishing a site-to-site connection between the corporate network and the VPC for seamless hybrid access. It also introduces operational complexity and does not scale well for scenarios where full internal network routing and access controls are required between entire corporate networks and AWS.

Set up a bastion host in a public subnet of the VPC to provide SSH-based access to AWS resources from the corporate network. Use security groups to control access - A bastion host provides access at the individual server level, typically over SSH or RDP, and is useful for administrative tasks. However, it does not establish a full network-layer or session-layer encrypted connection between the corporate network and AWS resources. It also does not scale for use cases requiring broad application or service access between on-premises and AWS. This solution lacks centralized routing and traffic control, making it insufficient for secure, scalable hybrid connectivity.

References:

https://docs.aws.amazon.com/vpn/latest/s2svpn/VPC_VPN.html

https://docs.aws.amazon.com/vpn/latest/clientvpn-admin/what-is.html

https://docs.aws.amazon.com/directconnect/latest/UserGuide/encryption-in-transit.html

Domain

Design Secure Architectures

Question 54Skipped

A media streaming company expects a major increase in user activity during the launch of a highly anticipated live event. The streaming platform is deployed on AWS and uses Amazon EC2 instances for the application layer and Amazon RDS for persistent storage. The operations team needs to proactively monitor system performance to ensure a smooth user experience during the event. Their monitoring setup must provide data visibility with intervals of no more than 2 minutes, and the team prefers a solution that is quick to implement and low-maintenance.

Which solution should the team implement?

Install the CloudWatch agent on all EC2 instances. Configure the agent to collect high-resolution custom metrics and stream them to CloudWatch Logs for analysis via Amazon Athena

Correct answer

Enable detailed monitoring on all EC2 instances and use Amazon CloudWatch metrics to track performance

Stream EC2 system logs to an Amazon OpenSearch Service domain for real-time indexing and visualization. Use OpenSearch Dashboards to monitor CPU and memory metrics

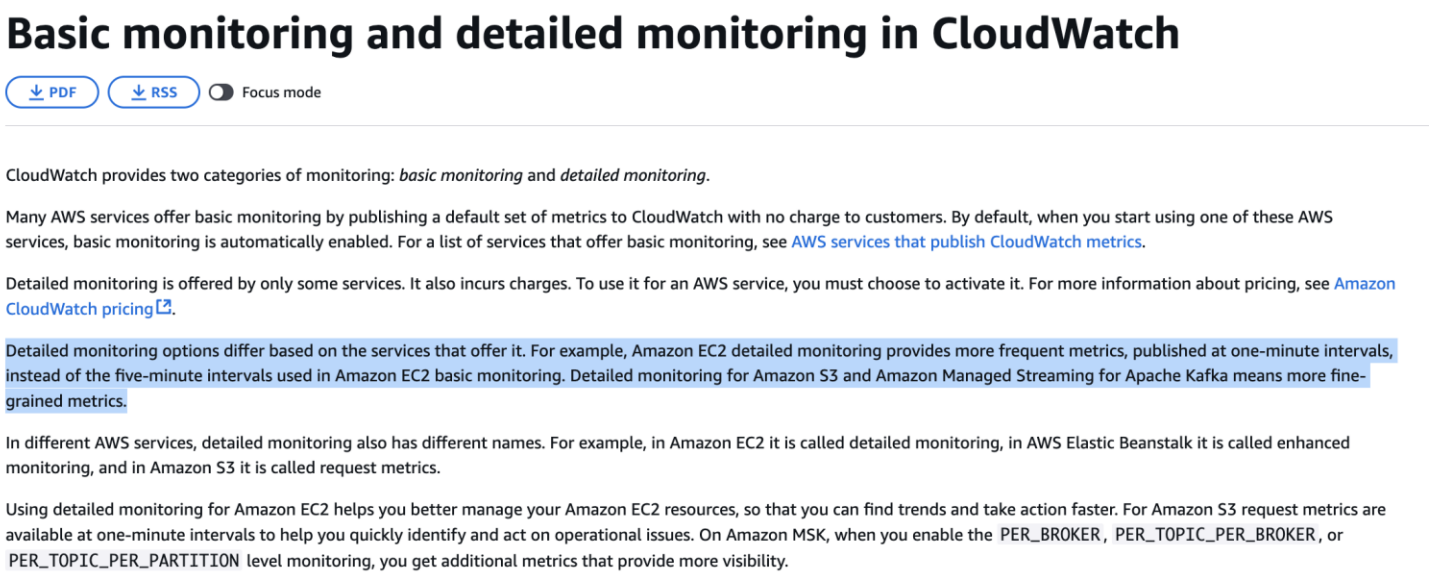
Use Amazon EventBridge to collect EC2 state changes and publish them to Amazon SNS. Subscribe a monitoring dashboard to the SNS topic to visualize metrics

Overall explanation

Correct option:

Enable detailed monitoring on all EC2 instances and use Amazon CloudWatch metrics to track performance

By default, EC2 instances publish CloudWatch metrics at 5-minute intervals. Enabling detailed monitoring changes this to 1-minute intervals, allowing the team to observe system performance with the necessary granularity. CloudWatch metrics provide native integration with EC2 and are the most efficient way to analyze CPU, network, and disk utilization without complex setups.



Basic monitoring and detailed monitoring in CloudWatch

PDF RSS Focus mode

CloudWatch provides two categories of monitoring: *basic monitoring* and *detailed monitoring*.

Many AWS services offer basic monitoring by publishing a default set of metrics to CloudWatch with no charge to customers. By default, when you start using one of these AWS services, basic monitoring is automatically enabled. For a list of services that offer basic monitoring, see AWS services that publish CloudWatch metrics.

Detailed monitoring is offered by only some services. It also incurs charges. To use it for an AWS service, you must choose to activate it. For more information about pricing, see Amazon CloudWatch pricing.

Detailed monitoring options differ based on the services that offer it. For example, Amazon EC2 detailed monitoring provides more frequent metrics, published at one-minute intervals, instead of the five-minute intervals used in Amazon EC2 basic monitoring. Detailed monitoring for Amazon S3 and Amazon Managed Streaming for Apache Kafka means more fine-grained metrics.

In different AWS services, detailed monitoring also has different names. For example, in Amazon EC2 it is called detailed monitoring, in AWS Elastic Beanstalk it is called enhanced monitoring, and in Amazon S3 it is called request metrics.

Using detailed monitoring for Amazon EC2 helps you better manage your Amazon EC2 resources, so that you can find trends and take action faster. For Amazon S3 request metrics are available at one-minute intervals to help you quickly identify and act on operational issues. On Amazon MSK, when you enable the `PER_BROKER`, `PER_TOPIC_PER_BROKER`, or `PER_TOPIC_PER_PARTITION` level monitoring, you get additional metrics that provide more visibility.

via - https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch-metrics-basic-detailed.html

Incorrect options:

Use Amazon EventBridge to collect EC2 state changes and publish them to Amazon SNS. Subscribe a monitoring dashboard to the SNS topic to visualize metrics - EventBridge and SNS are designed for event-driven architectures and alerting—not for performance monitoring or detailed metric analysis. This approach would not provide the continuous, granular data the company needs to evaluate real-time performance trends. Moreover, SNS is not built to directly support monitoring dashboards.

Stream EC2 system logs to an Amazon OpenSearch Service domain for real-time indexing and visualization. Use OpenSearch Dashboards to monitor CPU and memory metrics - While Amazon OpenSearch Service is excellent for log indexing and search, it does not natively collect EC2 performance metrics like CPU utilization or disk I/O. To make this work, the company would need to implement custom log collection and parsing, which adds unnecessary complexity compared to using CloudWatch.

Install the CloudWatch agent on all EC2 instances. Configure the agent to collect high-resolution custom metrics and stream them to CloudWatch Logs for analysis via Amazon Athena - Although this setup could support fine-grained metric collection, it introduces significant operational overhead. It requires manual agent installation, configuration, and maintenance across all instances. Analyzing logs via Athena also introduces query latency, which is not ideal for real-time performance tracking.

References:

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/cloudwatch-metrics-basic-detailed.html

https://docs.aws.amazon.com/AmazonCloudWatch/latest/monitoring/Install-CloudWatch-Agent.html

https://docs.aws.amazon.com/opensearch-service/latest/developerguide/what-is.html

Domain

Design Resilient Architectures

Question 55Skipped

A multinational logistics company operates its shipment tracking platform from Amazon EC2 instances deployed in the AWS us-west-2 Region. The platform exposes a set of APIs over HTTPS, which are used by logistics partners and customers around the world to retrieve real-time tracking data. The company has observed that users from Europe and Asia experience latency issues and inconsistent API response times when accessing the service. As a cloud architect, you have been tasked to propose the most cost-effective solution to improve performance for these international users without migrating the application.

Which solution should you recommend?

Use Amazon Route 53 latency-based routing to direct user requests to a copy of the EC2 API deployed in each major geographic Region

Deploy an Amazon CloudFront distribution in front of the API endpoint and apply the CachingOptimized managed policy to enhance caching behavior and improve content delivery efficiency

Deploy Amazon API Gateway in multiple AWS Regions and synchronize the API definitions. Use AWS Lambda as a proxy to forward requests to the EC2-hosted API in us-west-2

Correct answer

Configure AWS Global Accelerator in front of the existing HTTPS API, create one endpoint group in us-west-2 for the current application endpoint, and use the accelerator's global edge network to improve performance for users connecting from Europe and Asia

Overall explanation

Correct option:

Configure AWS Global Accelerator in front of the existing HTTPS API, create one endpoint group in us-west-2 for the current application endpoint, and use the accelerator's global edge network to improve performance for users connecting from Europe and Asia

This option is correct because AWS Global Accelerator improves the performance of globally accessed HTTPS applications without requiring the company to migrate or redeploy the existing API in multiple Regions. The shipment tracking platform already runs on Amazon EC2 in us-west-2, so Global Accelerator can be placed in front of that endpoint to provide static anycast IP addresses and route users from Europe and Asia to the nearest AWS edge location, where traffic enters the AWS global network early and then travels over the AWS backbone to us-west-2 with lower latency variation than the public internet. The option is also technically accurate because Global Accelerator endpoint groups are Regional resources, which means the correct design is to create a single endpoint group in us-west-2 for the existing application endpoint rather than separate endpoint groups for user geographies. This makes the solution more suitable than alternatives that depend on cacheable responses or require multi-Region application copies, because it improves global access performance for a real-time API while keeping the architecture simple and cost-effective.

Incorrect options:

Deploy an Amazon CloudFront distribution in front of the API endpoint and apply the CachingOptimized managed policy to enhance caching behavior and improve content delivery efficiency - This option is incorrect because Amazon CloudFront is primarily optimized for delivering static or cacheable content, and is not well-suited for APIs that serve highly dynamic or personalized responses, such as those in a shipment tracking system. These APIs typically return user-specific or real-time data that cannot be effectively cached, even with the use of the CachingOptimized managed cache policy. As a result, most requests would result in cache misses, causing CloudFront to forward the traffic to the origin in the us-west-2 Region. This adds an extra network hop and can actually increase latency for international users rather than reduce it. Furthermore, using CloudFront in this scenario adds unnecessary complexity and incurs additional costs without delivering significant performance improvements, making it a suboptimal and non-cost-effective solution for reducing API response times.

Use Amazon Route 53 latency-based routing to direct user requests to a copy of the EC2 API deployed in each major geographic Region - Latency-based routing works well when application stacks are deployed in multiple Regions. However, this approach involves replicating the EC2-based API infrastructure across multiple Regions, significantly increasing compute, data transfer, and management overhead. This is not the most cost-effective option as requested in the scenario.

Deploy Amazon API Gateway in multiple AWS Regions and synchronize the API definitions. Use AWS Lambda as a proxy to forward requests to the EC2-hosted API in us-west-2 - While deploying API Gateway in multiple Regions might help reduce DNS resolution time and provide regional endpoints, using Lambda to proxy requests back to a central Region introduces additional latency and unnecessary costs. This design offloads little from the original EC2 endpoints and does not leverage edge optimization services like CloudFront or Global Accelerator. Additionally, synchronizing API definitions and handling cross-region Lambda invocation can be complex. Reference:

References:

https://docs.aws.amazon.com/global-accelerator/latest/dg/what-is-global-accelerator.html

https://docs.aws.amazon.com/global-accelerator/latest/dg/introduction-how-it-works.html

https://aws.amazon.com/global-accelerator/features/#Use_cases

https://docs.aws.amazon.com/apigateway/latest/developerguide/api-gateway-api-endpoint-types.html

Domain

Design Cost-Optimized Architectures

Question 56Skipped

The engineering team at a weather tracking company wants to enhance the performance of its relational database and is looking for a caching solution that supports geospatial data.

As a solutions architect, which of the following solutions will you suggest?

Correct answer

Use Amazon ElastiCache for Redis

Use Amazon DynamoDB Accelerator (DAX)

Use AWS Global Accelerator

Use Amazon ElastiCache for Memcached

Overall explanation

Correct option:

Use Amazon ElastiCache for Redis

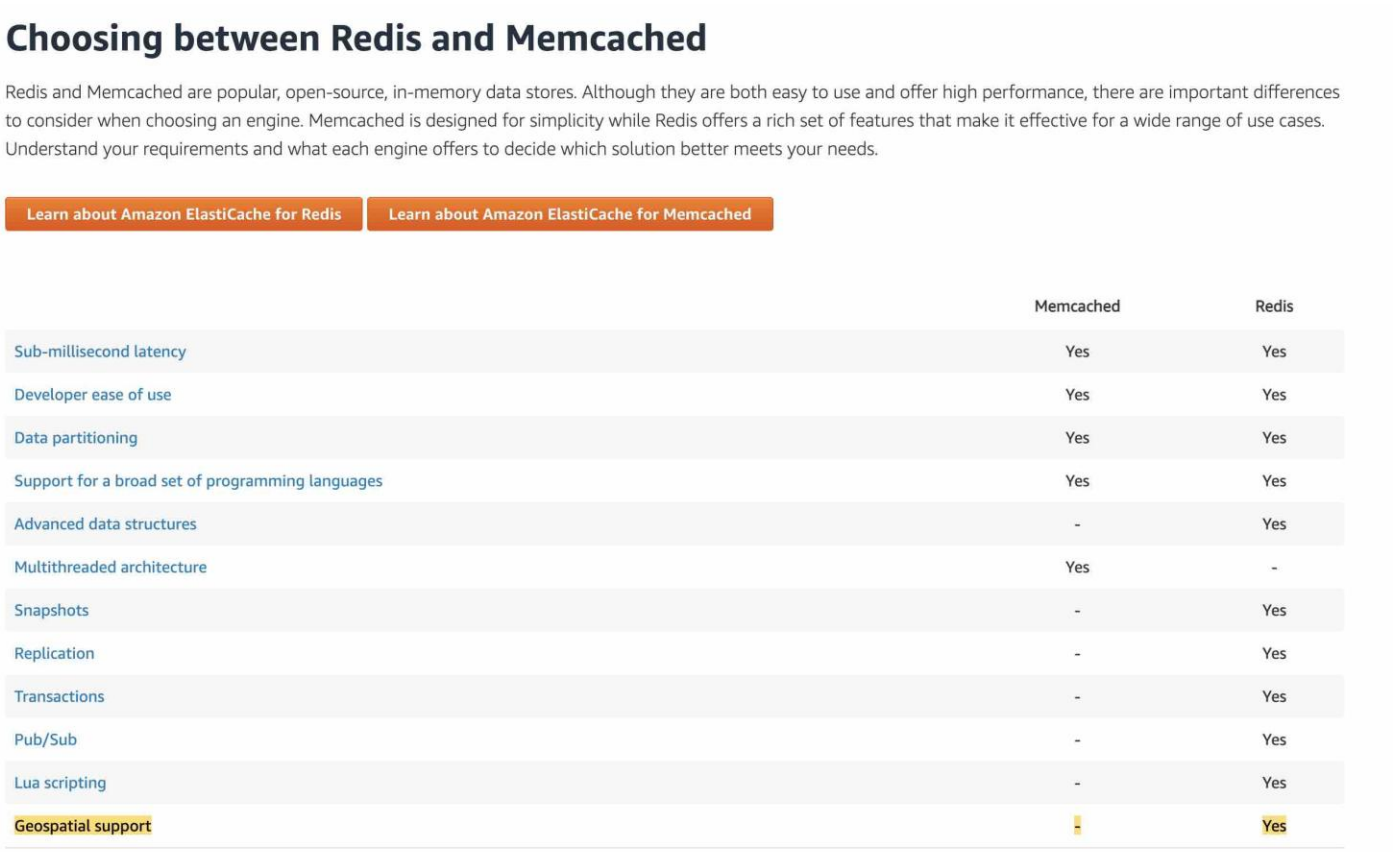
Amazon ElastiCache is a web service that makes it easy to set up, manage, and scale a distributed in-memory data store or cache environment in the cloud. Redis, which stands for Remote Dictionary Server, is a fast, open-source, in-memory key-value data store for use as a database, cache, message broker, and queue. Redis now delivers sub-millisecond response times enabling millions of requests per second for real-time applications in Gaming, Ad-Tech, Financial Services, Healthcare, and IoT. Redis is a popular choice for caching, session management, gaming, leaderboards, real-time analytics, geospatial, ride-hailing, chat/messaging, media streaming, and pub/sub apps.

All Redis data resides in the server's main memory, in contrast to databases such as PostgreSQL, Cassandra, MongoDB and others that store most data on disk or on SSDs. In comparison to traditional disk based databases where most operations require a roundtrip to disk, in-memory data stores such as Redis don't suffer the same penalty. They can therefore support an order of magnitude more operations and faster response times. The result is – blazing fast performance with average read or write operations taking less than a millisecond and support for millions of operations per second.

Redis has purpose-built commands for working with real-time geospatial data at scale. You can perform operations like finding the distance between two elements (for example people or places) and finding all elements within a given distance of a point.

Incorrect options:

Use Amazon ElastiCache for Memcached - Both Redis and MemCached are in-memory, open-source data stores. Memcached, a high-performance distributed memory cache service, is designed for simplicity while Redis offers a rich set of features that make it effective for a wide range of use cases. Memcached does not offer support for geospatial data.



Choosing between Redis and Memcached

Redis and Memcached are popular, open-source, in-memory data stores. Although they are both easy to use and offer high performance, there are important differences to consider when choosing an engine. Memcached is designed for simplicity while Redis offers a rich set of features that make it effective for a wide range of use cases. Understand your requirements and what each engine offers to decide which solution better meets your needs.

Learn about Amazon ElastiCache for Redis | Learn about Amazon ElastiCache for Memcached

	Memcached	Redis
Sub-millisecond latency	Yes	Yes
Developer ease of use	Yes	Yes
Data partitioning	Yes	Yes
Support for a broad set of programming languages	Yes	Yes
Advanced data structures	-	Yes
Multithreaded architecture	Yes	-
Snapshots	-	Yes
Replication	-	Yes
Transactions	-	Yes
Pub/Sub	-	Yes
Lua scripting	-	Yes
Geospatial support	-	Yes

via - https://aws.amazon.com/elasticache/redis-vs-memcached/

Use Amazon DynamoDB Accelerator (DAX) - Amazon DynamoDB Accelerator (DAX) is a fully managed, highly available, in-memory cache for Amazon DynamoDB. DAX does not support relational databases.

Use AWS Global Accelerator - AWS Global Accelerator is a networking service that helps you improve the availability and performance of the applications that you offer to your global users. This option has been added as a distractor, it has nothing to do with database caching.

Reference:

https://aws.amazon.com/elasticache/redis-vs-memcached/

Domain

Design High-Performing Architectures

Question 57Skipped

A company hosts a Microsoft SQL Server database on Amazon EC2 instances with attached Amazon EBS volumes. The operations team takes daily snapshots of these EBS volumes as backups. However, a recent incident occurred in which an automated script designed to clean up expired snapshots accidentally deleted all available snapshots, leading to potential data loss. The company wants to improve the backup strategy to avoid permanent data loss while still ensuring that old snapshots are eventually removed to optimize cost. A solutions architect needs to implement a mechanism that prevents immediate and irreversible deletion of snapshots.

Which solution will best meet these requirements with the least development effort?

Set up the IAM policy of the user to deny EBS snapshot deletion

Correct answer

Set up a 7-day EBS snapshot retention rule in Recycle Bin and apply the rule for all snapshots

Enable AWS Backup Vault Lock on the backup vault and store EBS snapshots in that vault to enforce deletion protection

Implement a Lambda-based backup automation workflow that archives snapshot metadata in DynamoDB and stores backups in Amazon S3 Glacier Deep Archive for long-term recovery

Overall explanation

Correct option:

Set up a 7-day EBS snapshot retention rule in Recycle Bin and apply the rule for all snapshots

The Amazon EBS Snapshot Recycle Bin allows organizations to retain deleted snapshots for a fixed duration before they are permanently deleted. By configuring a 7-day retention rule, snapshots that are deleted by mistake (e.g., through a faulty script) can still be recovered within the defined period. This provides protection from accidental deletion and satisfies the requirement to eventually remove old snapshots—thus balancing safety and cost. It requires minimal setup and no significant development effort.

Incorrect Options:

Implement a Lambda-based backup automation workflow that archives snapshot metadata in DynamoDB and stores backups in Amazon S3 Glacier Deep Archive for long-term recovery - This approach adds significant development effort and does not natively prevent snapshot deletion. So, it is incorrect.

Enable AWS Backup Vault Lock on the backup vault and store EBS snapshots in that vault to enforce deletion protection - This option appears secure by referencing Vault Lock, which can enforce write-once, read-many (WORM) policies for backups. However, EBS snapshots are not directly stored in AWS Backup vaults unless they are created through AWS Backup, and the original setup is using manual or script-driven EBS snapshot creation. Additionally, Vault Lock applies only to backups managed by AWS Backup, not to independently created snapshots through Amazon EC2 or custom scripts. Therefore, while it introduces a secure mechanism, it does not address the current architecture or snapshot process, making it an unsuitable solution without significant re-architecture.

Set up the IAM policy of the user to deny EBS snapshot deletion - While denying deletion can prevent accidental loss, it also completely blocks all snapshot deletions, including those that are genuinely expired or no longer needed. This violates the requirement to avoid indefinite snapshot retention, leading to storage bloat and increased cost.

References:

https://docs.aws.amazon.com/ebs/latest/userguide/recycle-bin.html

https://docs.aws.amazon.com/aws-backup/latest/devguide/vault-lock.html

Domain

Design Resilient Architectures

Question 58Skipped

A digital publishing platform stores large volumes of media assets (such as images and documents) in an Amazon S3 bucket. These assets are accessed frequently during business hours by internal editors and content delivery tools. The company has strict encryption policies and currently uses AWS KMS to handle server-side encryption. The cloud operations team notices that AWS KMS request costs are increasing significantly due to the high frequency of object uploads and accesses. The team is now looking for a way to maintain the same encryption method but reduce the cost of KMS usage, especially for frequent access patterns.

Which solution meets the company's encryption and cost optimization goals?

Correct answer

Enable S3 Bucket Keys for server-side encryption with AWS KMS (SSE-KMS) so that new objects use a bucket-level key rather than requesting individual KMS data keys for every object

Use client-side encryption by generating a local symmetric key and uploading it to Amazon S3 along with each object’s metadata for decryption

Switch to server-side encryption using Amazon S3 managed keys (SSE-S3) to eliminate all AWS KMS-related encryption charges while maintaining the same level of encryption control

Configure a VPC endpoint for S3 and restrict access to the bucket to traffic originating from the endpoint to avoid additional KMS charges

Overall explanation

Correct option:

Enable S3 Bucket Keys for server-side encryption with AWS KMS (SSE-KMS) so that new objects use a bucket-level key rather than requesting individual KMS data keys for every object

Amazon S3 Bucket Keys reduce the cost of Amazon S3 server-side encryption with AWS Key Management Service (AWS KMS) keys (SSE-KMS). Using a bucket-level key for SSE-KMS can reduce AWS KMS request costs by up to 99 percent by decreasing the request traffic from Amazon S3 to AWS KMS. Enabling S3 Bucket Keys with SSE-KMS allows S3 to generate a unique data key for each object locally using a bucket-level KMS key, which dramatically reduces the number of direct AWS KMS API calls. This approach maintains the same security and compliance properties as SSE-KMS, while reducing request-related costs—especially beneficial for workloads with frequent access or write operations.

S3 Bucket Keys for SSE-KMS

Workloads that access millions or billions of objects encrypted with SSE-KMS can generate large volumes of requests to AWS KMS. When you use SSE-KMS to protect your data without an S3 Bucket Key, Amazon S3 uses an individual AWS KMS data key for every object. In this case, Amazon S3 makes a call to AWS KMS every time a request is made against a KMS-encrypted object. For information about how SSE-KMS works, see Using server-side encryption with AWS KMS keys (SSE-KMS).

When you configure your bucket to use an S3 Bucket Key for SSE-KMS, AWS generates a short-lived bucket-level key from AWS KMS, then temporarily keeps it in S3. This bucket-level key will create data keys for new objects during its lifecycle. S3 Bucket Keys are used for a limited time period within Amazon S3, reducing the need for S3 to make requests to AWS KMS to complete encryption operations. This reduces traffic from S3 to AWS KMS, allowing you to access AWS KMS-encrypted objects in Amazon S3 at a fraction of the previous cost.

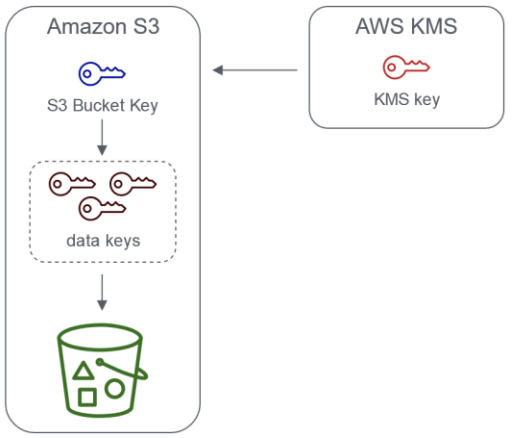
Unique bucket-level keys are fetched at least once per requester to ensure that the requester's access to the key is captured in an AWS KMS CloudTrail event. Amazon S3 treats callers as different requesters when they use different roles or accounts, or the same role with different scoping policies. AWS KMS request savings reflect the number of requesters, request patterns, and relative age of the objects requested. For example, a fewer number of requesters, requesting multiple objects in a limited time window, and encrypted with the same bucket-level key, results in greater savings.

Note

Using S3 Bucket Keys allows you to save on AWS KMS request costs by decreasing your requests to AWS KMS for `Encrypt`, `GenerateDataKey`, and `Decrypt` operations through the use of a bucket-level key. By design, subsequent requests that take advantage of this bucket-level key do not result in AWS KMS API requests or validate access against the AWS KMS key policy.

When you configure an S3 Bucket Key, objects that are already in the bucket do not use the S3 Bucket Key. To configure an S3 Bucket Key for existing objects, you can use a `CopyObject` operation. For more information, see Configuring an S3 Bucket Key at the object level.

Amazon S3 will only share an S3 Bucket Key for objects encrypted by the same AWS KMS key. S3 Bucket Keys are compatible with KMS keys created by AWS KMS, imported key material, and key material backed by custom key stores.



Server-side encryption with AWS Key Management service using an S3 Bucket Key

via - https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-key.html

Incorrect options:

Configure a VPC endpoint for S3 and restrict access to the bucket to traffic originating from the endpoint to avoid additional KMS charges - Setting up a VPC endpoint for S3 helps with network cost optimization and security controls, but it has no effect on AWS KMS pricing. KMS requests are charged separately from network data transfer, and restricting access through a VPC endpoint does not eliminate or reduce encryption-related API calls. This option improves access control but does not meet the goal of reducing encryption cost.

Use client-side encryption by generating a local symmetric key and uploading it to Amazon S3 along with each object's metadata for decryption - Client-side encryption offloads encryption from AWS to the client, but this method adds significant management complexity, such as key rotation, secure key storage, and decryption logic on the client side. Uploading the key with the metadata is also a security risk and not compliant with best practices. Additionally, this approach does not reduce costs related to KMS unless KMS is removed entirely from the encryption pipeline, which may violate encryption policy requirements.

Switch to server-side encryption using Amazon S3 managed keys (SSE-S3) to eliminate all AWS KMS-related encryption charges while maintaining the same level of encryption control - While SSE-S3 does eliminate AWS KMS usage and its associated costs, it does not meet the requirement to maintain the company's strict encryption policies that rely specifically on AWS KMS-managed keys. SSE-S3 uses S3-managed keys, which are not integrated with AWS KMS and lack the granular key management, audit logging, and fine-grained access control provided by KMS. Therefore, although this option reduces costs,

it fails to comply with the company's requirement to continue using KMS for encryption, making it an invalid solution in this scenario.

References:

https://docs.aws.amazon.com/AmazonS3/latest/userguide/bucket-key.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingClientSideEncryption.html

https://docs.aws.amazon.com/AmazonS3/latest/userguide/UsingEncryption.html

Domain

Design Secure Architectures

Question 59Skipped

A global media company uses a fleet of Amazon EC2 instances (behind an Application Load Balancer) to power its video streaming application. To improve the performance of the application, the engineering team has also created an Amazon CloudFront distribution with the Application Load Balancer as the custom origin. The security team at the company has noticed a spike in the number and types of SQL injection and cross-site scripting attack vectors on the application.

As a solutions architect, which of the following solutions would you recommend as the MOST effective in countering these malicious attacks?

Correct answer

Use AWS Web Application Firewall (AWS WAF) with Amazon CloudFront distribution

Use Amazon Route 53 with Amazon CloudFront distribution

Use AWS Firewall Manager with CloudFront distribution

Use AWS Security Hub with Amazon CloudFront distribution

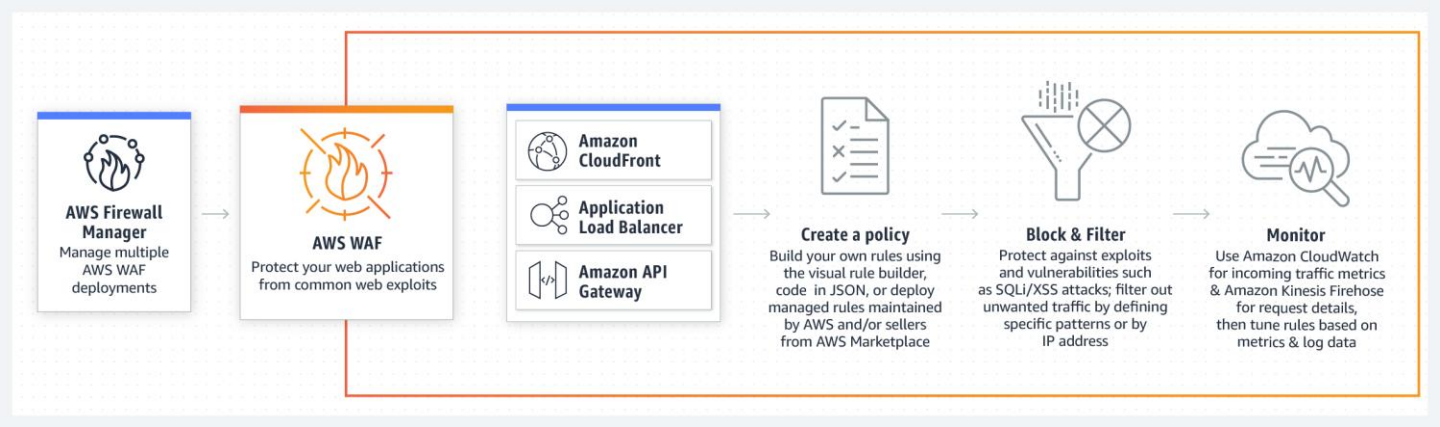
Overall explanation

Correct option:

Use AWS Web Application Firewall (AWS WAF) with Amazon CloudFront distribution

AWS WAF is a web application firewall that helps protect your web applications or APIs against common web exploits that may affect availability, compromise security, or consume excessive resources. AWS WAF gives you control over how traffic reaches your applications by enabling you to create security rules that block common attack patterns, such as SQL injection or cross-site scripting, and rules that filter out specific traffic patterns you define.

How AWS WAF Works:



via - https://aws.amazon.com/waf/

A web access control list (web ACL) gives you fine-grained control over the web requests that your Amazon CloudFront distribution, Amazon API Gateway API, or Application Load Balancer responds to.

When you create a web ACL, you can specify one or more Amazon CloudFront distributions that you want AWS WAF to inspect. AWS WAF starts to allow, block, or count web requests for those distributions based on the conditions that you identify in the web ACL. Therefore, combining AWS WAF with Amazon CloudFront can prevent SQL injection and cross-site scripting attacks. So this is the correct option.

Incorrect options:

Use Amazon Route 53 with Amazon CloudFront distribution - Amazon Route 53 is a highly available and scalable cloud Domain Name System (DNS) web service. You cannot use Route 53 to prevent SQL injection and cross-site scripting attacks. So this option is incorrect.

Use AWS Security Hub with Amazon CloudFront distribution - AWS Security Hub gives you a comprehensive view of your high-priority security alerts and security posture across your AWS accounts. With Security Hub, you have a single place that aggregates, organizes, and prioritizes your security alerts, or findings, from multiple AWS services, such as Amazon GuardDuty, Amazon Inspector, Amazon Macie, AWS Identity and Access Management (IAM) Access Analyzer, and AWS Firewall Manager, as well as from AWS Partner solutions. You cannot use Security Hub to prevent SQL injection and cross-site scripting attacks. So this option is incorrect.

Use AWS Firewall Manager with CloudFront distribution - AWS Firewall Manager is a security management service that allows you to centrally configure and manage firewall rules across your accounts and applications in AWS Organization. You cannot use AWS Firewall Manager to prevent SQL injection and cross-site scripting attacks. So this option is incorrect.

References:

https://aws.amazon.com/waf/features/

https://docs.aws.amazon.com/waf/latest/developerguide/web-acl.html

https://docs.aws.amazon.com/waf/latest/developerguide/cloudfront-features.html

Domain

Design Secure Architectures

Question 60Skipped

A media company is evaluating the possibility of moving its IT infrastructure to the AWS Cloud. The company needs at least 10 terabytes of storage with the maximum possible I/O performance for processing certain files which are mostly large videos. The company also needs close to 450 terabytes of very durable storage for storing media content and almost double of it, i.e. 900 terabytes for archival of legacy data.

As a Solutions Architect, which set of services will you recommend to meet these requirements?

Correct answer

Amazon EC2 instance store for maximum performance, Amazon S3 for durable data storage, and Amazon S3 Glacier for archival storage

Amazon EBS for maximum performance, Amazon S3 for durable data storage, and Amazon S3 Glacier for archival storage

Amazon S3 standard storage for maximum performance, Amazon S3 Intelligent-Tiering for intelligent, durable storage, and Amazon S3 Glacier Deep Archive for archival storage

Amazon EC2 instance store for maximum performance, AWS Storage Gateway for on-premises durable data access and Amazon S3 Glacier Deep Archive for archival storage

Overall explanation

Correct option:

Amazon EC2 instance store for maximum performance, Amazon S3 for durable data storage, and Amazon S3 Glacier for archival storage

An instance store provides temporary block-level storage for your instance. This storage is located on disks that are physically attached to the host computer. Instance store is ideal for the temporary storage of information that changes frequently, such as buffers, caches, scratch data, and other temporary content, or for data that is replicated across a fleet of instances, such as a load-balanced pool of web servers.

You can specify instance store volumes for an instance only when you launch it. You can't detach an instance store volume from one instance and attach it to a different instance.

Some instance types use NVMe or SATA-based solid-state drives (SSD) to deliver high random I/O performance. This is a good option when you need storage with very low latency, but you don't need the data to persist when the instance terminates or you can take advantage of fault-tolerant architectures.

Amazon S3 Standard offers high durability, availability, and performance object storage for frequently accessed data. Because it delivers low latency and high throughput, Amazon S3 Standard is appropriate for a wide variety of use cases, including cloud applications, dynamic websites, content distribution, mobile and gaming applications, and big data analytics.

Amazon S3 Glacier is a secure, durable, and low-cost storage class for data archiving. You can reliably store any amount of data at costs that are competitive with or cheaper than on-premises solutions. To keep costs low yet suitable for varying needs, Amazon S3 Glacier provides three retrieval options that range from a few minutes to hours. You can upload objects directly to Amazon S3 Glacier, or use S3 Lifecycle policies to transfer data between any of the Amazon S3 Storage Classes for active data (S3 Standard, S3 Intelligent-Tiering, S3 Standard-IA, and S3 One Zone-IA) and S3 Glacier.

Incorrect options:

Amazon S3 standard storage for maximum performance, Amazon S3 Intelligent-Tiering for intelligent, durable storage, and Amazon S3 Glacier Deep Archive for archival storage - Amazon EC2 instance store volumes provide the best I/O performance for low latency requirement, as in the current use case. The Amazon S3 Intelligent-Tiering storage class is designed to optimize costs by automatically moving data to the most cost-effective access tier, without performance impact or operational overhead.

Amazon S3 Glacier Deep Archive is Amazon S3's lowest-cost storage class and supports long-term retention and digital preservation for data that may be accessed once or twice a year. It is designed for customers — particularly those in highly-regulated industries, such as the Financial Services, Healthcare, and Public Sectors — that retain data sets for 7-10 years or longer to meet regulatory compliance requirements.

Amazon EBS for maximum performance, Amazon S3 for durable data storage, and Amazon S3 Glacier for archival storage - Amazon Elastic Block Store (Amazon EBS) provides block-level storage volumes for use with EC2 instances. Amazon EBS volumes are particularly well-suited for use as the primary storage for file systems, databases, or for any applications that require fine granular updates and access to raw, unformatted, block-level storage. For high I/O performance, instance store volumes are a better option.

Amazon EC2 instance store for maximum performance, AWS Storage Gateway for on-premises durable data access and Amazon S3 Glacier Deep Archive for archival storage - AWS Storage Gateway is a hybrid cloud storage service that gives you on-premises access to virtually unlimited cloud storage. AWS Storage Gateway will be the right answer if the customer wanted to retain the on-premises data storage and just move the applications to AWS Cloud. In the absence of such requirements, instance store is a better option for high performance and Amazon S3 for durable storage.

Reference:

https://aws.amazon.com/s3/storage-classes/

Domain

Design High-Performing Architectures

Question 61Skipped

A media company operates a web application that enables users to upload photos. These uploads are stored in an Amazon S3 bucket located in the eu-west-2 Region. To enhance performance and provide secure access under a custom domain name, the company wants to integrate Amazon CloudFront for

uploads to the S3 bucket. The architecture must support secure HTTPS connections using a custom domain, and the upload process must ensure optimal speed and security.

Which combination of actions will fulfill these requirements? (Select two)

Correct selection

Request a public certificate from AWS Certificate Manager (ACM) in the us-east-1 Region and associate it with the CloudFront distribution

Set up a custom origin request policy in CloudFront that includes all viewer headers and query strings. Enable S3 Object Ownership to allow CloudFront to assume control of uploaded files via a signed URL

Request a public certificate from AWS Certificate Manager (ACM) in the eu-west-2 Region and associate it with the CloudFront distribution

Correct selection

Set up Amazon S3 to accept uploads from CloudFront by enabling origin access control (OAC)

Create a CloudFront distribution with an S3 static website endpoint as the origin and enable upload operations

Overall explanation

Correct options:

Request a public certificate from AWS Certificate Manager (ACM) in the us-east-1 Region and associate it with the CloudFront distribution

CloudFront only supports ACM certificates that are created in the us-east-1 (N. Virginia) Region. Even if the content resides in a different Region (like eu-west-2), a public certificate for HTTPS on a custom domain name must originate from us-east-1. This certificate is used to secure content access through CloudFront over HTTPS.

Set up Amazon S3 to accept uploads from CloudFront by enabling origin access control (OAC)

Origin Access Control (OAC) is the recommended method for granting CloudFront permission to upload to (write into) an S3 bucket securely. OAC supersedes the older Origin Access Identity (OAI) approach and supports both read and write operations. This allows you to restrict direct access to the S3 bucket and ensure that only CloudFront can act as a secure intermediary for uploads.

Grant CloudFront permission to access the S3 bucket

Before you create an origin access control (OAC) or set it up in a CloudFront distribution, make sure that CloudFront has permission to access the S3 bucket origin. Do this after creating a CloudFront distribution, but before adding the OAC to the S3 origin in the distribution configuration.

Use an S3 bucket policy to allow the CloudFront service principal (`cloudfront.amazonaws.com`) to access the bucket. Use a `Condition` element in the policy to allow CloudFront to access the bucket only when the request is on behalf of the CloudFront distribution that contains the S3 origin. This is the distribution with the S3 origin that you want to add OAC to.

For information about adding or modifying a bucket policy, see Adding a bucket policy using the Amazon S3 console in the *Amazon S3 User Guide*.

The following are examples of S3 bucket policies that allow a CloudFront distribution with OAC enabled access to an S3 origin.

Example S3 bucket policy that allows read-only access for a CloudFront distribution with OAC enabled

```
{
    "Version": "2012-10-17",
    "Statement": {
        "Sid": "AllowCloudFrontServicePrincipalReadOnly",
        "Effect": "Allow",
        "Principal": {
            "Service": "cloudfront.amazonaws.com"
        },
        "Action": "s3:GetObject",
        "Resource": "arn:aws:s3:::<S3 bucket name>/*",
        "Condition": {
            "StringEquals": {
                "AWS:SourceArn": "arn:aws:cloudfront::111122223333:distribution/<CloudFront distribution ID>"
            }
        }
    }
}
```

Example S3 bucket policy that allows read and write access for a CloudFront distribution with OAC enabled

```
{
    "Version": "2012-10-17",
    "Statement": {
        "Sid": "AllowCloudFrontServicePrincipalReadWrite",
        "Effect": "Allow",
        "Principal": {
            "Service": "cloudfront.amazonaws.com"
        },
        "Action": [
            "s3:GetObject",
            "s3:PutObject"
        ],
        "Resource": "arn:aws:s3:::<S3 bucket name>/*",
        "Condition": {
            "StringEquals": {
                "AWS:SourceArn": "arn:aws:cloudfront::111122223333:distribution/<CloudFront distribution ID>"
            }
        }
    }
}
```

via - https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html

Incorrect options:

Create a CloudFront distribution with an S3 static website endpoint as the origin and enable upload operations - This is incorrect because S3 static website endpoints only support GET and HEAD requests and are intended for public read access. They do not support PUT/POST operations required for user uploads. Therefore, they are not suitable for the secure upload workflow described.

Set up a custom origin request policy in CloudFront that includes all viewer headers and query strings. Enable S3 Object Ownership to allow CloudFront to assume control of uploaded files via a signed URL - While customizing origin request policies and enabling S3 Object Ownership might help with some access controls, they do not provide a mechanism to upload content directly via CloudFront. Additionally, signed URLs are primarily used for download access, not for file uploads via CloudFront.

Request a public certificate from AWS Certificate Manager (ACM) in the eu-west-2 Region and associate it with the CloudFront distribution - This option is incorrect because CloudFront cannot use

ACM public certificates that are created in any Region other than us-east-1. If a certificate is created in eu-west-2, it won't be attachable to the CloudFront distribution.

References:

https://docs.aws.amazon.com/acm/latest/userguide/acm-overview.html

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/Introduction.html

https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-signed-urls.html

Domain

Design Secure Architectures

Question 62Skipped

An enterprise is developing an internal compliance framework for its cloud infrastructure hosted on AWS. The enterprise uses AWS Organizations to group accounts under various organizational units (OUs) based on departmental function. As part of its governance controls, the security team mandates that all Amazon EC2 instances must be tagged to indicate the level of data classification — either 'confidential' or 'public'. Additionally, the organization must ensure that IAM users cannot launch EC2 instances without assigning a classification tag, nor should they be able to remove the tag from running instances. A solutions architect must design a solution to meet these compliance controls while minimizing operational overhead.

Which combination of steps will meet these requirements? (Select two)

Create a tag enforcement Lambda function that runs on a schedule to identify EC2 instances without the required tag. The function sends a notification to administrators and optionally shuts down noncompliant resources

Enable AWS Config rules to detect noncompliant EC2 instances. Trigger an AWS Systems Manager Automation runbook to reapply missing tags automatically when noncompliance is detected

Use AWS Identity and Access Management (IAM) permission boundaries to restrict EC2-related actions unless the dataClassification tag is present. Apply these boundaries to all IAM roles used for EC2 provisioning

Correct selection

Define a tag policy in AWS Organizations that enforces the dataClassification key and restricts values to 'confidential' and 'public'. Attach this tag policy to the applicable organizational unit (OU) to enforce uniform tagging behavior across accounts

Correct selection

Create a service control policy (SCP) that denies the ec2:RunInstances API action unless the required tag key is present in the request. Create a second SCP that denies the ec2:DeleteTags action for EC2 resources. Attach both SCPs to the relevant OU in AWS Organizations

Overall explanation

Correct options:

Define a tag policy in AWS Organizations that enforces the dataClassification key and restricts values to 'confidential' and 'public'. Attach this tag policy to the applicable organizational unit (OU) to enforce uniform tagging behavior across accounts

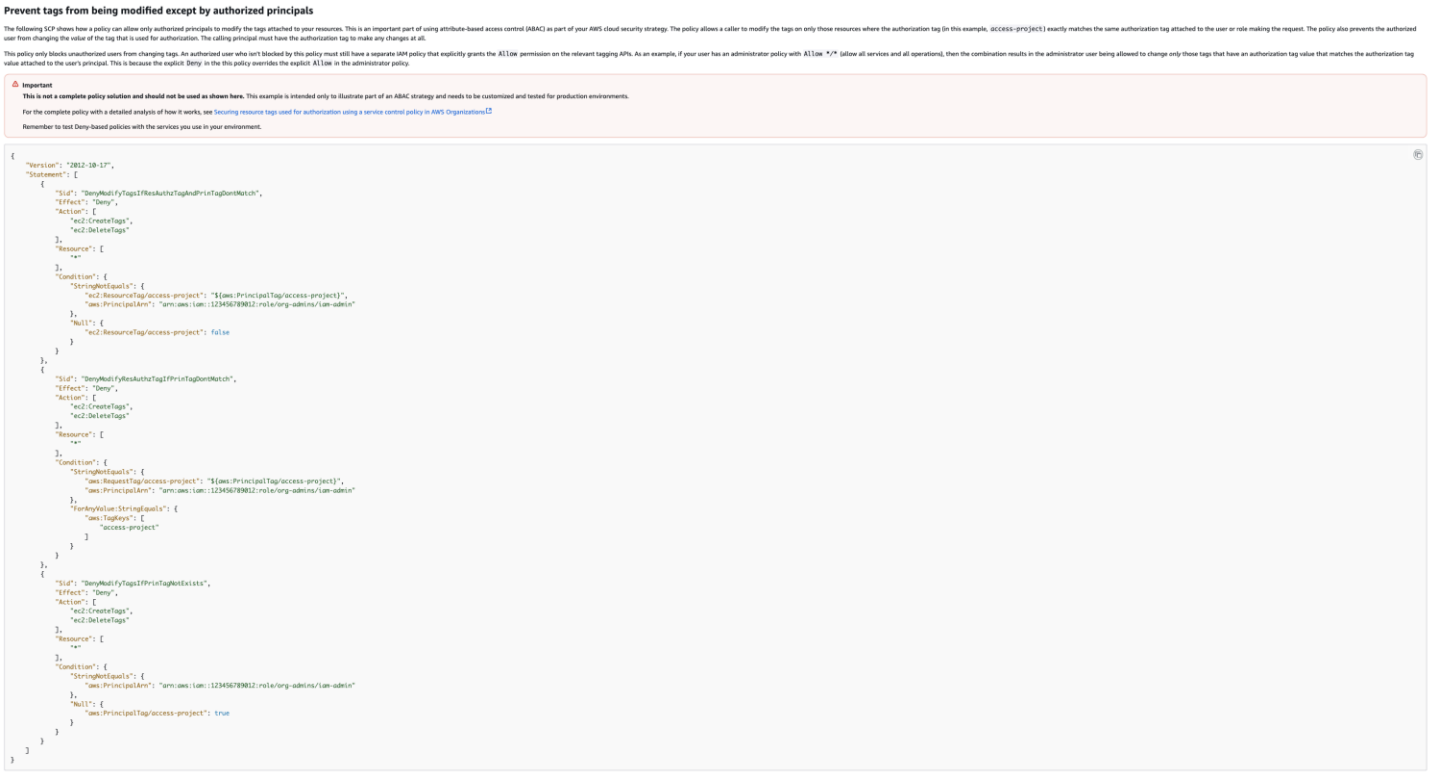
AWS Tag Policies, managed through AWS Organizations, allow administrators to define rules for tag keys and values. These policies do not directly prevent resource creation, but they help enforce tag consistency and visibility by flagging noncompliant resources.

In this case, the solutions architect creates a tag policy that requires the dataClassification key and restricts its values to 'confidential' and 'public'. By attaching this policy to the relevant Organizational Unit (OU), the tag rules automatically apply to all member accounts. Tag policies work across AWS services that support tagging compliance checks. While they don't block actions like RunInstances, they play a governance and auditing role, helping organizations monitor and enforce consistent tagging.

Create a service control policy (SCP) that denies the ec2:RunInstances API action unless the required tag key is present in the request. Create a second SCP that denies the ec2:DeleteTags action for EC2 resources. Attach both SCPs to the relevant OU in AWS Organizations

Service Control Policies (SCPs) allow you to centrally govern permissions for accounts in AWS Organizations. An SCP defines the maximum set of permissions available to IAM users and roles in an account. When combined with condition keys, SCPs can enforce fine-grained controls on specific operations.

To meet the requirements - one SCP is created to deny the ec2:RunInstances action unless the request includes a dataClassification tag key and a second SCP explicitly denies the ec2:DeleteTags action on EC2 resources to ensure tags cannot be removed. These SCPs are then attached to the OU, thereby applying the policies to all member accounts. This ensures that - EC2 instances cannot be launched without the required tag and tags like dataClassification cannot be deleted, preserving compliance integrity.



Prevent tags from being modified except by authorized principals

The following SCP shows how a policy can allow only authorized principals to modify the tags attached to your resources. This is an important part of using attribute-based access control (ABAC) as part of your AWS cloud security strategy. The policy allows a caller to modify the tags on only those resources where the authorization tag (in this example, `access-project`) exactly matches the same authorization tag attached to the user or role making the request. The policy also prevents the authorized user from changing the value of the tag that is used for authorization. The calling principal must have the authorization tag to make any changes at all.

This policy only blocks unauthorized users from changing tags. An authorized user who isn't blocked by this policy must still have a separate IAM policy that explicitly grants the `Allow` permission on the relevant tagging APIs. As an example, if your user has an administrator policy with `Allow */*` (allow all services and all operations), then the combination results in the administrator user being allowed to change only those tags that have an authorization tag value that matches the authorization tag value attached to the user's principal. This is because the explicit `Deny` in the this policy overrides the explicit `Allow` in the administrator policy.

Important

This is not a complete policy solution and should not be used as shown here. This example is intended only to illustrate part of an ABAC strategy and needs to be customized and tested for production environments.

For the complete policy with a detailed analysis of how it works, see Securing resource tags used for authorization using a service control policy in AWS Organizations

Remember to test Deny-based policies with the services you use in your environment.

```
{
    "Version": "2012-10-17",
    "Statement": [
        {
            "Sid": "DenyModifyTagsIfResAuthzTagAndPrinTagDontMatch",
            "Effect": "Deny",
            "Action": [
                "ec2:CreateTags",
                "ec2:DeleteTags"
            ],
            "Resource": [
                "*"
            ],
            "Condition": {
                "StringNotEquals": {
                    "ec2:ResourceTag/access-project": "${aws:PrincipalTag/access-project}",
                    "aws:PrincipalArn": "arn:aws:iam::123456789012:role/org-admins/iam-admin"
                },
                "Null": {
                    "ec2:ResourceTag/access-project": false
                }
            }
        },
        {
            "Sid": "DenyModifyResAuthzTagIfPrinTagDontMatch",
            "Effect": "Deny",
            "Action": [
                "ec2:CreateTags",
                "ec2:DeleteTags"
            ],
            "Resource": [
                "*"
            ],
            "Condition": {
                "StringNotEquals": {
                    "aws:RequestTag/access-project": "${aws:PrincipalTag/access-project}",
                    "aws:PrincipalArn": "arn:aws:iam::123456789012:role/org-admins/iam-admin"
                },
                "ForAnyValue:StringEquals": {
                    "aws:TagKeys": [
                        "access-project"
                    ]
                }
            }
        },
        {
            "Sid": "DenyModifyTagsIfPrinTagNotExists",
            "Effect": "Deny",
            "Action": [
                "ec2:CreateTags",
                "ec2:DeleteTags"
            ],
            "Resource": [
                "*"
            ],
            "Condition": {
                "StringNotEquals": {
                    "aws:PrincipalArn": "arn:aws:iam::123456789012:role/org-admins/iam-admin"
                },
                "Null": {
                    "aws:PrincipalTag/access-project": true
                }
            }
        }
    ]
}
```

via

- https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps_examples_tagging.html

Incorrect options:

Create a tag enforcement Lambda function that runs on a schedule to identify EC2 instances without the required tag. The function sends a notification to administrators and optionally shuts down noncompliant resources - This reactive approach relies on periodic audits rather than enforcing tagging at the time of resource creation. It increases operational overhead and cannot guarantee immediate compliance, especially for short-lived resources.

Use AWS Identity and Access Management (IAM) permission boundaries to restrict EC2-related actions unless the dataClassification tag is present. Apply these boundaries to all IAM roles used for EC2 provisioning - While IAM permission boundaries can restrict actions within a boundary, they are not designed to enforce tag-based controls across all users or to prevent tag deletion. SCPs and tag policies are more appropriate at the organizational level.

Enable AWS Config rules to detect noncompliant EC2 instances. Trigger an AWS Systems Manager Automation runbook to reapply missing tags automatically when noncompliance is detected - AWS Config and automation runbooks offer remediation, but do not prevent noncompliant resources from being created in the first place. This violates the requirement for proactive enforcement and minimal public exposure of misclassified data.

References:

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_tag-policies.html

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps.html

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_manage_policies_scps_examples_tagging.html

https://docs.aws.amazon.com/IAM/latest/UserGuide/access_policies_boundaries.html

https://docs.aws.amazon.com/organizations/latest/userguide/orgs_introduction.html

Domain

Design Secure Architectures

Question 63Skipped

A medical devices company uses Amazon S3 buckets to store critical data. Hundreds of buckets are used to keep the data segregated and well organized. Recently, the development team noticed that the lifecycle policies on the Amazon S3 buckets have not been applied optimally, resulting in higher costs.

As a Solutions Architect, can you recommend a solution to reduce storage costs on Amazon S3 while keeping the IT team's involvement to a minimum?

Use Amazon S3 One Zone-Infrequent Access, to reduce the costs on Amazon S3 storage

Configure Amazon EFS to provide a fast, cost-effective and sharable storage service

Use Amazon S3 Outposts storage class to reduce the costs on Amazon S3 storage by storing the data on-premises

Correct answer

Use Amazon S3 Intelligent-Tiering storage class to optimize the Amazon S3 storage costs

Overall explanation

Correct option:

Use Amazon S3 Intelligent-Tiering storage class to optimize the Amazon S3 storage costs

The Amazon S3 Intelligent-Tiering storage class is designed to optimize costs by automatically moving data to the most cost-effective access tier, without performance impact or operational overhead. It works by storing objects in two access tiers: one tier that is optimized for frequent access and another lower-cost tier that is optimized for infrequent access.

For a small monthly monitoring and automation fee per object, Amazon S3 monitors access patterns of the objects in Amazon S3 Intelligent-Tiering and moves the ones that have not been accessed for 30 consecutive days to the infrequent access tier. If an object in the infrequent access tier is accessed, it is automatically moved back to the frequent access tier. There are no retrieval fees when using the Amazon S3 Intelligent-Tiering storage class, and no additional tiering fees when objects are moved between access tiers. It is the ideal storage class for long-lived data with access patterns that are unknown or unpredictable.

Amazon S3 Storage Classes can be configured at the object level and a single bucket can contain objects stored in Amazon S3 Standard, Amazon S3 Intelligent-Tiering, Amazon S3 Standard-IA, and Amazon S3 One Zone-IA. You can upload objects directly to Amazon S3 Intelligent-Tiering, or use S3 Lifecycle policies to transfer objects from Amazon S3 Standard and Amazon S3 Standard-IA to Amazon S3 Intelligent-Tiering. You can also archive objects from Amazon S3 Intelligent-Tiering to Amazon S3 Glacier.

Incorrect options:

Configure Amazon EFS to provide a fast, cost-effective and sharable storage service - Amazon Elastic File System (Amazon EFS) provides a simple, scalable, fully managed elastic NFS file system for use with AWS Cloud services and on-premises resources. Amazon EFS offers sharable service, unlike Amazon Elastic Block Storage (EBS) that cannot be shared by instances. Amazon EFS is costlier than storing data in Amazon S3. Also, Amazon EFS needs an Amazon EC2 instance or an AWS Direct Connect network connection. Hence, this is not the correct option.

Use Amazon S3 One Zone-Infrequent Access, to reduce the costs on Amazon S3 storage - Amazon S3 One Zone-IA is for data that is accessed less frequently but requires rapid access when needed. Unlike other Amazon S3 Storage Classes which store data in a minimum of three Availability Zones (AZs), Amazon S3 One Zone-IA stores data in a single AZ and costs 20% less than Amazon S3 Standard-IA. Amazon S3 One Zone-IA is ideal for customers who want a lower-cost option for infrequently accessed data but do not require the availability and resilience of Amazon S3 Standard or Amazon S3 Standard-IA. Not a right option, since data stored is business-critical and cannot be risked by using Amazon S3 One Zone-IA.

Use Amazon S3 Outposts storage class to reduce the costs on Amazon S3 storage by storing the data on-premises - This is a distractor as Amazon S3 on Outposts (S3 Outposts) delivers object storage to your on-premises AWS Outposts environment. It is used in conjunction with AWS Outposts and has no relevance to the current use case.

Reference:

https://aws.amazon.com/s3/storage-classes/

Domain

Design Cost-Optimized Architectures

Question 64Skipped

You have built an application that is deployed with Elastic Load Balancing and an Auto Scaling Group. As a Solutions Architect, you have configured aggressive Amazon CloudWatch alarms, making your Auto Scaling Group (ASG) scale in and out very quickly, renewing your fleet of Amazon EC2 instances on a daily basis. A production bug appeared two days ago, but the team is unable to SSH into the instance to debug the issue, because the instance has already been terminated by the Auto Scaling Group. The log files are saved on the Amazon EC2 instance.

How will you resolve the issue and make sure it doesn't happen again?

Make a snapshot of the Amazon EC2 instance just before it gets terminated

Disable the Termination from the Auto Scaling Group any time a user reports an issue

Use AWS Lambda to regularly SSH into the Amazon EC2 instances and copy the log files to Amazon S3

Correct answer

Install an Amazon CloudWatch Logs agents on the Amazon EC2 instances to send logs to Amazon CloudWatch

Overall explanation

Correct option:

Install an Amazon CloudWatch Logs agents on the Amazon EC2 instances to send logs to Amazon CloudWatch

You can use the Amazon CloudWatch Logs agent installer on an existing Amazon EC2 instance to install and configure the Amazon CloudWatch Logs agent. After installation is complete, logs automatically flow from the instance to the log stream you create while installing the agent. The agent confirms that it has started and it stays running until you disable it.

Here, the natural and by far the easiest solution would be to use the Amazon CloudWatch Logs agents on the Amazon EC2 instances to automatically send log files into Amazon CloudWatch, so we can analyze them in the future easily should any problem arise.

To control whether an Auto Scaling group can terminate a particular instance when scaling in, use instance scale-in protection. You can enable the instance scale-in protection setting on an Auto Scaling group or on an individual Auto Scaling instance. When the Auto Scaling group launches an instance, it inherits the instance scale-in protection setting of the Auto Scaling group. You can change the instance scale-in protection setting for an Auto Scaling group or an Auto Scaling instance at any time.

Incorrect options:

Disable the Termination from the Auto Scaling Group any time a user reports an issue - Disabling the Termination from the Auto Scaling Group would prevent our Auto Scaling Group from being Elastic and impact our costs. Therefore this option is incorrect.

Make a snapshot of the Amazon EC2 instance just before it gets terminated - Making a snapshot of the Amazon EC2 instance before it gets terminated could work but it's tedious, not elastic and very expensive, since our interest is just the log files. Therefore this option is not the best fit for the given use-case.

You can back up the data on your Amazon EBS volumes to Amazon S3 by taking point-in-time snapshots. Snapshots are incremental backups, which means that only the blocks on the device that have changed after your most recent snapshot are saved. This minimizes the time required to create the snapshot and saves on storage costs by not duplicating data.

Use AWS Lambda to regularly SSH into the Amazon EC2 instances and copy the log files to Amazon S3 - AWS Lambda lets you run code without provisioning or managing servers. It cannot be used for production-grade serverless log analytics. Using AWS Lambda would be extremely hard to use for this task. Therefore this option is not the best fit for the given use-case.

Reference:

https://docs.aws.amazon.com/AmazonCloudWatch/latest/logs/QuickStartEC2Instance.html

Domain

Design High-Performing Architectures

Question 65Skipped

A mobile-based e-learning platform is migrating its backend storage layer to Amazon DynamoDB to support a rapidly increasing number of student users and learning transactions. The platform must ensure seamless availability and minimal disruption for a global user base. The DynamoDB design must provide low-latency performance, high availability, and automatic fault tolerance across geographies with the lowest possible operational overhead and cost.

Which solution will fulfill these needs in the most cost-efficient manner?

Create separate DynamoDB tables in multiple Regions. Use AWS Data Pipeline to synchronize data periodically between Regions to maintain availability

Enable DynamoDB Accelerator (DAX) to reduce response time for read operations. Deploy DAX in one Region, and use scheduled Lambda functions to replicate data to other Regions

Correct answer

Use DynamoDB global tables for automatic multi-Region replication. Enable provisioned capacity mode with auto scaling to optimize cost and ensure consistent availability

Deploy separate DynamoDB tables in each required AWS Region using on-demand capacity mode. Implement a custom cross-Region replication mechanism by streaming data changes with DynamoDB Streams and processing them through AWS Lambda functions

Overall explanation

Correct option:

Use DynamoDB global tables for automatic multi-Region replication. Enable provisioned capacity mode with auto scaling to optimize cost and ensure consistent availability

DynamoDB global tables offer a native, multi-Region solution that provides high availability and resiliency by replicating data automatically across AWS Regions. This ensures users around the world experience low-latency read and write operations. By using provisioned capacity mode with auto scaling, the company can optimize cost based on actual throughput requirements rather than overprovisioning resources. This approach eliminates the need for manual replication or custom logic, and it ensures continuous availability in the event of a Regional disruption.

Incorrect options:

Enable DynamoDB Accelerator (DAX) to reduce response time for read operations. Deploy DAX in one Region, and use scheduled Lambda functions to replicate data to other Regions - DAX improves read performance through in-memory caching, but it does not help with write scalability, cross-Region replication, or resiliency. Scheduled Lambda functions for replication are not real-time, can fail under high load, and add complexity to the architecture. This setup does not fulfill the requirement for continuous availability and resilience for a global user base.

Deploy separate DynamoDB tables in each required AWS Region using on-demand capacity mode. Implement a custom cross-Region replication mechanism by streaming data changes with DynamoDB Streams and processing them through AWS Lambda functions - While DynamoDB Streams can capture real-time data modifications in a table, using them for manual cross-Region replication requires building and maintaining custom replication logic—typically with AWS Lambda functions or AWS Glue jobs. This approach introduces significant complexity, potential latency, and increased operational burden. It also lacks automated conflict resolution, version control, and automatic failover capabilities. Additionally, this setup doesn't guarantee strong consistency across Regions, and managing data conflicts or write ordering becomes challenging.

Create separate DynamoDB tables in multiple Regions. Use AWS Data Pipeline to synchronize data periodically between Regions to maintain availability - Manually managing multiple DynamoDB tables across Regions using AWS Data Pipeline introduces significant operational complexity and replication delays. This method does not ensure real-time synchronization or automatic failover, making it unsuitable for high-throughput, latency-sensitive applications like online gaming. It also requires custom conflict resolution and lacks the seamless integration offered by global tables.

References:

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/GlobalTables.html

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/V2globaltables_HowItWorks.html

https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Streams.html

Domain

Design High-Performing Architectures